Quorus: Efficient, Scalable Threshold ML-DSA Signatures from MPC*

Alexander Bienstock Leo de Castro✉ Daniel Escudero
Antigoni Polychroniadou Akira Takahashi

JPMC AlgoCRYPT CoE & JPMC AI Research
{firstname}.{lastname}@jpmchase.com

Abstract

A threshold signature protocol divides a secret signing key among multiple parties, enabling any subset above a threshold to jointly create a signature. While post-quantum (PQ) threshold signatures are being studied, especially following NIST's call for threshold schemes, most solutions focus on specially designed, threshold-friendly signature schemes. However, real-world applications like distributed certificate authorities and digital currencies require signatures verifiable under existing standardized procedures. With NIST's standardization of PQ signatures and ongoing industry deployment, designing an efficient threshold scheme compatible with NIST-standardized verification remains a critical challenge.

In this work, we present the first efficient and scalable solution for multi-party generation of the module-lattice digital signature algorithm (ML-DSA), one of NIST's PQ signature standards. Our contributions are two-fold. First, we present a variant of the ML-DSA signing algorithm that is amenable to efficient multi-party computation (MPC) and prove that this variant achieves the same security as the original ML-DSA scheme. Second, we present several efficient & scalable MPC protocols to instantiate the threshold signing functionality. Our protocols can produce threshold signatures with as little as 150 KB (per party) of online communication per rejection-sampling round. In addition, we instantiate our protocols in the honest-majority setting, which allows us to avoid any additional public key assumptions.

Our signatures verify under the same ML-DSA implementation for all security levels, with signature and verification key sizes matching ML-DSA; previous lattice-based threshold schemes could not match both of these sizes. Our solution provides the first method for producing threshold post-quantum signatures compatible with NIST-standardized verification, scalable to any number of parties, without new assumptions.

*This is the full version of the paper accepted at USENIX Security 2026.

1 Introduction

In an increasingly interconnected digital world, ensuring the authenticity, integrity, and non-repudiation of electronic communications and transactions is critical. Digital signatures have emerged as a foundational technology in achieving these goals, playing a vital role in securing online communications, enabling trusted software distribution, and underpinning modern cryptographic protocols. By leveraging public key cryptography, digital signatures provide a means to ensure that a given message has not been altered during transmission. Their applications span a wide range of domains, including e-commerce, e-governance, secure email, blockchain, and legal documentation, among others.

Modern digital signatures like ECDSA [Pub23] and EdDSA [BDL+12] are vulnerable to attacks given a sufficiently large quantum computer [Sho94]. To address this risk, NIST initiated a series of standardization efforts to construct cryptographic primitives based on quantum-safe assumptions, which are problems that are computationally hard even for quantum computers. This effort resulted in the standardization of a signature scheme based on lattice problems, namely *Dilithium* [BDK+21], which was renamed *ML-DSA* [Pub24] in the standard. This scheme is based on the module-lattice learning with errors (MLWE) and short integer solution (MSIS) problems [LS15], which are believed to be hard even for a quantum machine.

As one of the first standardized post-quantum signature schemes, there is huge motivation to integrate ML-DSA into applications currently using pre-quantum signatures like ECDSA. One core application is *threshold signing*, which delegates the signing authority to n parties. This is a particularly important feature for signing keys that are sensitive and long-lived. Threshold signing balances security and availability by allowing any set of more than t parties to construct a valid signature while preventing any set of t or fewer parties from generating any valid signature. Use-cases of threshold signatures include securing master secrets in key management systems or software update services. In these settings, a secret

key can be split across multiple nodes that can still jointly produce signatures, and yet an adversary would need to compromise at least $t+1$ of these nodes in order to access the secret-key.

Despite many efficient constructions of threshold signature protocols for pre-quantum signatures like ECDSA (e.g., [DKLs24]) and Schnorr (e.g., [KG20, Lin24]), post-quantum signature schemes tend to be much less "threshold-friendly." In fact, except a few very recent works [DKLS25, BCdP+25] (see Section 1.2), there does not exist threshold signature scheme that can interoperate with *NIST-standardized* PQ signature verification, such as ML-DSA. Instead, it is common in the literature (cf. [DOTT22, ASY22, TPCZ23, GKS24, ENP24, DKM+24, EKT24, BKL+25, PN25, CATZ24, ALLW25]) to design lattice-based schemes specifically tailored to the threshold setting, which includes custom verification algorithms that differ from the standardized verification procedures. These signatures have downsides beyond non-standardization: they are typically larger than their non-threshold counterparts, and it is not uncommon that their size and other performance aspects scale with the number of signing parties.

Difficulties in designing threshold ML-DSA. Unfortunately, designing a threshold version of the standardized ML-DSA remains an elusive open problem. Even though in principle one could use generic secure multiparty computation (MPC) to distributively evaluate the ML-DSA signing algorithm, doing so naively would lead to prohibitive costs in practice. This is primarily due to the fact that ML-DSA's native description is highly non-MPC-friendly, since the signing algorithm requires computing a cryptographic hash function (e.g., SHA3) on inputs that must be *hidden* from the adversary. Without additional analysis and modifications to the signing algorithm, this would require running SHA3 within the MPC protocol itself, which is a circuit comprising of millions of arithmetic gates.

1.1 Our Contribution

In this work, we present the *first*[1] threshold signature protocol compatible with the *NIST-standardized* ML-DSA scheme. Our protocol produces signatures that pass exactly the same verification implementation of the standardized, non-threshold ML-DSA scheme. Our protocol supports an arbitrary number of parties, and the distribution of signatures produced by our protocol does not change with the number of parties. One of the main contributions of this work is a new "MPC-friendly" variant of the ML-DSA signing algorithm, where partial signatures can be safely revealed. This allows the expensive hash function in the signing algorithm to be run *locally* by the parties (i.e., outside of the MPC functionality). We provide a security proof for this variant to show that the security level of our variant is essentially identical to the original ML-DSA signing algorithm, and we expect this MPC-friendly variant to find applications in other MPC implementations of ML-DSA (beyond simple threshold access structures). Additionally, we prove that this variant even satisfies a stronger unforgeability notion that we call *unforgeability under chosen message attacks with incomplete signatures* ($\mathsf{UF\text{-}CMA}_{\perp}$), which may be of independent interest.

With an MPC-friendly variant of ML-DSA in place, we present our second contribution, which is a careful synthesis of several MPC building blocks in order to securely evaluate our ML-DSA signing algorithm. The modularity of our approach enables us to leverage state-of-the-art techniques from general-purpose MPC, which allows us to achieve the best performance and trade-offs in terms of communication complexity and round count. In terms of security, we prove that our protocol UC-realizes an ideal functionality, $\mathcal{F}_{\mathsf{T\text{-}MLDSA}}$, which computes ordinary ML-DSA signatures *exactly*, instead of realizing an abstract threshold signature functionality or proving game-based unforgeability in isolation. Furthermore, even though we focus on the setting of honest majority with active security, the generality of our techniques allows for extensions to other contexts as well. When taken together, these contributions yield the first efficient, scalable solution for threshold ML-DSA signature generation.

A conceptual shift away from Schnorr. We emphasize that the use of general-purpose MPC to implement PQ threshold signing represents a major conceptual shift in the design of these protocols. Essentially all prior works on lattice-based threshold signatures based on the Fiat-Shamir transform either rely on threshold homomorphic encryption or attempted to leverage the similarities between Lyubashevsky-like signatures [Lyu12] and Schnorr. In threshold Schnorr, it is common for threshold shares of the signature to *themselves* be valid Schnorr signatures under shares of the signing key, and the linearity of Schnorr verification allows these signature shares to be publicly combined. However, locally generating ML-DSA signature shares and running a public, "Schnorr-style" combiner is *fundamentally* limited by the additional requirement on the norm of the ML-DSA signature. In other words, since an ML-DSA signature must have small norm, combining many signature shares increases the norm and degrades the security of the final signature. Therefore, despite the apparent similarities between ML-DSA and Schnorr, our work implicitly argues for a radically different approach in the design of the threshold versions of these signatures. We view this work as identifying the correct MPC subroutines that must be evaluated to obtain a threshold ML-DSA protocol that is simultaneously efficient, secure, and scalable. We expect future works to develop optimized methods for these subroutines in various application settings.

[1] As a concurrent and independent work, Borin et al. [BCdP+25] presented a different approach to threshold ML-DSA. See Section 1.2 for a comparison.

1.2 Related Work

Constructions from Threshold-friendly Schemes. Recent works on threshold lattice-based signatures follow the paradigm of threshold Schnorr signatures [KG20, BCK+22, CKM23], while simultaneously designing "Schnorr-like" signatures with threshold-friendly properties. Damgård et al. [DOTT22] designed a 2-round, n-out-of-n signing protocol for a modified version of the Lyubashevsky signature [Lyu12]. This scheme has been improved in several ways, by reducing the number of aborts [ADP24], improving the communication complexity [Che23], or turning it into t-out-of-n threshold schemes with FHE [GKS24] or MPC [TPCZ23]. The two-round scheme proposed in [GKS24] obtains signatures of size 46.6 KB and public keys of size 13.6 KB for $(t,n) = (3,5)$. Note that [TPCZ23] uses generic *dishonest majority* MPC, and like us they rely on the comparison protocol from Rabbit [MRVW21]. However, they do not instantiate ML-DSA exactly, and in fact they avoid evaluating hashes in MPC by using commitments as in [DOTT22], which is different from our novel approach.

Chairattana-Apirom, Tessaro and Zhu [CATZ24] proposed a 2-round threshold scheme based on (a rejection-free variant of) the Lyubashevsky signature with the signature size of [CATZ24] is over 100 KB for up to 32 parties [ZT25]. Threshold Raccoon [DKM+24] is an efficient 3-round threshold signature scheme based on Raccoon [dPKPR24], which is itself a threshold-friendly variant of ML-DSA. The scheme naturally supports integration of Shamir secret sharing, allowing efficient instantiations, e.g., signatures of size 12.7 KB and public keys of size 3.8 KB for up to 1024 parties. The follow-up works of threshold Raccoon reduced the number of rounds to 2 [EKT24, BKL+25, ZT25], proved the adaptive security [KRT24], and realized the security with identifiable aborts [dPKN+25, dPENP25]. The recent Raccoon-based construction due to del Pino and Niot [PN25] enjoys the most compact instantiation with signatures of size 2.7 KB and public keys of size 2.6 KB for up to 8 parties by limiting the number of signatures produced to 2^{64}. However, it is unclear whether the same technique carries over to scalable threshold signatures compatible with the standardized ML-DSA verification process.

Two-party ML-DSA signature generation. To the best of our knowledge, Trilithium [DKLS25] is the only multi-party scheme outputting a signature compliant with NIST-standardized ML-DSA verification. However, Trilithium is specialized for only two parties, and it requires both parties to generate a valid signature. Moreover, Trilithium heuristically assumes that a rejected, partial signature can be simulated with the uniform distribution, while we introduce an important tweak to the signing algorithm to enable a provable simulation of partial signatures.

Concurrent work. Concurrently with our work, Borin et al. [BCdP+25] also proposed a threshold signature scheme interoperable with the ML-DSA verification process, optimized for up to 6 parties. Compared to our scheme, their protocol achieves game-based unforgeability in the dishonest-majority setting, requires fewer rounds, and incurs lower communication costs. In contrast, our protocol is proven UC-secure in the honest-majority setting, involves more rounds, and has higher communication overhead for the same number of parties. However, our scheme is scalable to an arbitrary number of parties, whereas their protocol relies on replicated secret sharing and local rejection sampling on individual shares, which seem to limit scalability beyond a small constant number of parties. We compare our protocol to this concurrent work in Section 5.2.

1.3 Technical Overview

In this section we present a brief overview of our techniques. We begin with a high-level description of ML-DSA.[2] This scheme is defined over a standard power-of-two cyclotomic ring $\mathcal{R} = \mathbb{Z}[x]/(x^N+1)$ and its quotient $\mathcal{R}_q = \mathcal{R}/q\mathcal{R}$, which means every polynomial of $\mathcal{R}_q$ consists of N coefficients in $\mathbb{Z}_q$. The secret key consists of two vectors $\mathbf{s} \in \mathcal{R}^\ell$ and $\mathbf{e} \in \mathcal{R}^k$ of polynomials, each having small coefficients. The public key is an MLWE instance $\mathbf{t} = \mathbf{A}\cdot\mathbf{s}+\mathbf{e} \bmod q$, where $\mathbf{A} \in \mathcal{R}_q^{\ell\times k}$ is a *public* matrix. For signing a message m, the signer follows the standard *Fiat-Shamir with Aborts* approach [Lyu12], summarized as follows:

1. Samples $\mathbf{y} \in \mathcal{R}^\ell$ at random with small entries, and then computes $\mathbf{w} = \mathbf{A}\mathbf{y} \bmod q$. Then (each coefficient of) $\mathbf{w}$ is decomposed as $\mathbf{w} = (2\gamma_2)\cdot\mathbf{w}_1 + \mathbf{w}_0$ for certain public parameter γ_2, where $\mathbf{w}_1$ is *high-bits* of $\mathbf{w}$, while $\mathbf{w}_0$ is *low-bits* of $\mathbf{w}$, respectively.

2. Derive a *challenge* $c = \mathsf{H}(\mathsf{m}, \mathbf{w}_1)$

3. Compute a *response* $\mathbf{z} = c\cdot\mathbf{s}+\mathbf{y}$ (over $\mathcal{R}$).

4. Finally, before outputting the signature, a *rejection sampling* step checks that $\|\mathbf{z}\|_\infty$ and $\|\mathbf{w}_0 - c\cdot\mathbf{e}\|_\infty$ are within certain specified public bounds, aborting by outputting $\perp$ if this is not the case. If the check passes, then the final signature is $(\mathbf{w}_1, c, \mathbf{z})$.

Let us denote by $[\cdot]$ a linear secret-sharing scheme over $\mathcal{R}_q$. A natural approach to thresholdizing ML-DSA would be to let the parties begin with shares of the secret-key $[\mathbf{s}], [\mathbf{e}]$, and run the steps from the signing algorithm, replacing all operations by *generic MPC* calls that operate on shares. The first blocker one would face is evaluating $\mathsf{c} \leftarrow \mathsf{H}(\mathsf{m}, [\mathbf{w}_1])$ in MPC, which is prohibitively expensive due to the complexity of SHA-3 hash function used in ML-DSA. Note that, *for a valid signature*, $\mathbf{w}_1$ is ultimately public since $\mathbf{w}$ can be approximately computed from the signature as $\mathbf{w} \approx \mathbf{A}\mathbf{z} - c\cdot\mathbf{t}$ (and hence $\mathbf{w}_1$, the upper bits, can be *exactly* derived). This leads to the intuition that

[2] This omits some Dilithium-specific optimizations that are irrelevant to highlight our techniques. We provide a detailed description of actual ML-DSA in Section 2.1 and A.1.

it is acceptable to let the parties first reconstruct $\mathbf{w}_1 \leftarrow [\mathbf{w}_1]$, and evaluate $c = \mathsf{H}(\mathsf{m}, \mathbf{w}_1)$ *in the clear*.

Security of ML-DSA with Revealed Incomplete Signatures. However, notice that in Step 4 of the (non-threshold) signing algorithm above, the signer outputs nothing *if one of the norm checks fails*. Is it still secure to reveal $(\mathbf{w}_1, c, \perp)$ when an ML-DSA signature is rejected? We answer this question affirmatively by introducing a tweak to the ML-DSA signing algorithm and go further to prove that the resulting scheme satisfies a stronger version of the standard unforgeability notion.

First, a subtle technicality with rejection-sampling-based signatures [Lyu12], including ML-DSA, is that the underlying three-pass identification protocol often lacks a proof of standard *honest verifier zero knowledge* (HVZK). Instead, the standard analysis of ML-DSA [BBD+23, KLS18] relies on weaker variants of HVZK, which only requires the simulatability of the tuple $(\mathbf{w}_1, c, \mathbf{z})$ *conditioned on* $\mathbf{z} \neq \perp$—that is, assuming the transcript has been accepted. While recent works have made progress by showing that rejected transcripts in certain schemes can be simulated statistically [BTT22, DFPS23] or even computationally [DFPS23, PN25], these techniques do not directly carry over to ML-DSA due to several optimizations and its parameter configurations.[3] Notably, these techniques are developed for identification schemes in which the prover sends $\mathbf{w} = \mathbf{A}\mathbf{y} + \mathbf{e}_w$ as a commitment. They then rely on the leftover hash lemma [BTT22, DFPS23] (viewing $h_{\mathbf{A}} : (\mathbf{y}, \mathbf{e}_w) \mapsto \mathbf{w}$ as a *surjective* universal hash) or the LWE assumption [DFPS23, PN25] to argue that the rejected $\mathbf{w}$ looks pseudorandom conditioned on $\mathbf{z} = \perp$. However, the ML-DSA identification only sends the higher order bits of $\mathbf{w} = \mathbf{A}\mathbf{y}$ without an error, where the map $h_{\mathbf{A}}$ is not surjective since ML-DSA's $\mathbf{A}$ is either square or tall. To bridge this gap, we tweak the ML-DSA signing operation by bringing back the noise $\mathbf{e}_w$ in such a way that the resulting signature remains verifiable under the standardized ML-DSA verification. The challenging part is to determine a suitable norm bound on $\mathbf{e}_w$ that preserves the standardized ML-DSA parameters while minimizing the correctness error. Through careful analysis, we set $\max \|\mathbf{e}_w\|_\infty = \max \|\mathbf{e}\|_\infty \ll \max \|\mathbf{y}\|_\infty$ to balance these requirements. We empirically confirm that this configuration only introduces a negligible correctness error in practice. As a result, we prove *strong computational HVZK* (scHVZK) [DFPS23] for the modified identification under the standard MLWE assumption, without changing the sizes of the output signatures.

With scHVZK in place, it is now straightforward to prove standard unforgeability under chosen-message attacks (UF-CMA) following the analysis of [DFPS23]. We also observe that our scHVZK simulator enables us to go beyond standard UF-CMA security. Since a rejected transcript $(\mathbf{w}_1, c, \perp)$ is essentially an *incomplete signature*, revealing such a transcript on queried message m should not count as having signed m. As such, an adversary that only obtains incomplete signatures on some messages should still not be able to forge a valid signature on any of these messages. To capture this intuition, we define the $\mathsf{UF\text{-}CMA}_\perp$ notion tailored to signature schemes that may output incomplete signatures, and prove that our modified ML-DSA satisfies it.

[3] This is a well-known open problem [CGL+24]

Our Threshold Signing Protocol. Having removed the challenging part of running H in MPC, what remains is efficiently instantiating the signing steps in MPC. We focus on the setting of *honest majority* with *active security*, for which we use Shamir secret-sharing $[\cdot]_q$ over the ML-DSA modulus q. The parties start with shares $[\mathbf{s}]_q, [\mathbf{e}]_q$ and compute locally $[\mathbf{t}]_q = \mathbf{A} \cdot [\mathbf{s}]_q + [\mathbf{e}]_q$. Most of our more elaborate primitives rely on *edabits* [EGK+20]. We use them to sample small shared random values $[\mathbf{y}]_q$ and $[\mathbf{e}_w]_q$, and also to perform the modular reduction to obtain shares of $\mathbf{w}_0$ and $\mathbf{w}_1$, as well as instantiating the secure comparisons needed for rejection sampling. Thanks to our unforgeability analysis sketched above, the parties can safely reconstruct $\mathbf{w}_1$, *even if the signature ends up being rejected*. Because of this, the parties can compute the challenge $c = \mathsf{H}(\mathsf{m}, \mathbf{w}_1)$ *in the clear*. For this rejection sampling part, we need to see if at least one of the bound checks failed, for which we present a method for *Batched OR* computation (see Protocol 3). Assuming rejection sampling passes, the rest of the signature consists of opening $\mathbf{z} = c \cdot \mathbf{s} + \mathbf{y}$. We consider concrete instantiations for these primitives: Escudero et al. [EGK+20] for edabits, Rabbit [MRVW21] for comparison, Nishide-Ohta [NO07] for bit decomposition (needed for obtaining $\mathbf{w}_0, \mathbf{w}_1$ from $\mathbf{w}$), and for openings and multiplications we consider both round-optimized (all-to-all openings) and communication-optimized ("king-based") variants (see Appendix A.3).

2 Preliminaries

Notations For integers a and b such that $a < b$, $[a, b]$ denotes a set $\{a, a+1, \ldots, b\}$. For an algorithm $\mathcal{A}$, $x \leftarrow \mathcal{A}$ means $\mathcal{A}$ outputs a value x. For a set S, $x \xleftarrow{\$} S$ means a value x is sampled uniformly from S.

2.1 Module Lattices and ML-DSA

In this section, we recall the most basic operations in ML-DSA. As we have already sketched the ML-DSA signing function in Section 1.3, we focus on the description of the parameters and computational assumptions here. For the reader's convenience, Appendix A.1 provides a self-contained summary of the original ML-DSA scheme and the complete list of parameters (Table 2).

Basic Operations. For a positive integer α and any $x \in \mathbb{Z}$, the centered modular reduction operation $x' := x \bmod^{\pm} \alpha$ maps x to a unique integer $-\lceil \alpha/2 \rceil < x' \leq \lfloor \alpha/2 \rfloor$ such that

$x \equiv x' \bmod \alpha$. ML-DSA operates on a power-of-two cyclotomic ring $\mathcal{R} := \mathbb{Z}[x]/(x^N+1)$ and its quotient $\mathcal{R}_q := \mathcal{R}/q\mathcal{R}$. For a ring element $z = \sum_{i=0}^{N-1} z_i \cdot x^i \in \mathcal{R}_q$, the infinity norm of z is defined as $\|z\|_\infty := \max_i |z_i \mod^{\pm} q|$. For a module element $\mathbf{z} = (z^{(1)}, \ldots, z^{(\ell)}) \in \mathcal{R}_q^\ell$, its infinity norm is defined analogously: $\|\mathbf{z}\|_\infty := \max_j \left\|z^{(j)}\right\|_\infty$. The Hamming weight $\mathsf{HW}(\mathbf{z})$ is defined as $||\mathbf{z}||_1 = \sum_j ||z^{(j)}||_1$.

We will write S_η to denote all elements $x \in \mathcal{R}_q$ such that $\|x\|_\infty \leq \eta$. We will write $\tilde{S}_\eta$ to denote the set $\{x \mod^{\pm} 2\eta : x \in \mathcal{R}\}$. Note that $\tilde{S}_\eta$ does not contain any polynomials with $-\eta$ coefficients.

Hash Functions. ML-DSA uses SHAKE-256 H essentially in three ways and maps its output to appropriated codomains: (1) public key digest $tr = \mathsf{H}(\rho, \mathbf{t}_1) \in \{0,1\}^{256}$, (2) message digest $\mu = \mathsf{H}(tr, \mathsf{m}) \in \{0,1\}^{512}$, and (3) Fiat-Shamir challenge $c = \mathsf{H}(\mu, \mathbf{w}_1) \in C = \{c \in \mathcal{R} : ||c||_1 = \tau, ||c||_\infty = 1\}$. As the details of these mappings are not relevant to our protocol, we use the same notation H and refer the reader to [BDK+21, §5.3] for more details.

Security Levels & Common Parameters. The ML-DSA FIPS standard [Pub24, Table 1, page 15] lists the various parameters for the scheme. In Table 1, we give the parameters that are unchanged for all security levels of ML-DSA. Note that $q := 2^{23} - (2^{13} - 1)$ just under 2^{23}, and elements in $\mathbb{Z}_q$ fit in 23 bits.

Supporting Functions. ML-DSA relies on the following supporting functions. We assume that if the function takes a module element $\mathbf{x} \in \mathcal{R}_q^k$ as input, it applies the operation coefficient-wise to each polynomial in $\mathbf{x}$. See Algorithm 9) for the full details:

$\mathbf{A} = \mathsf{ExpandA}(\rho)$: Expands a uniformly random 256-bit seed ρ into a larger matrix $\mathbf{A} \in \mathcal{R}_q^{k \times \ell}$.

$(r_1, r_0) = \mathsf{Power2Round}_q(r, d)$: Decomposes $r \in \mathbb{Z}_q$ into (r_1, r_0) such that $r = 2^d \cdot r_1 + r_0$

$(r_1, r_0) = \mathsf{DecomposeMod}(r, \alpha)$: [CGTZ23] Decomposes $r \in \mathbb{Z}_q$ into unique (r_1, r_0) such that $r = \alpha \cdot r_1 + r_0$, where $r_1 \in [0, (q-1)/\alpha]$ and $r_0 \in [-\alpha/2+1, \alpha/2]$ if $r_1 \neq 0$. It is an MPC-friendly and equivalent variant of Decompose [Pub24, Algorithm 36], and we use them interchangeably.

$r_1 = \mathsf{HighBits}_q(r, \alpha)$: Extracts the high bits r_1 of r by internally running $\mathsf{DecomposeMod}(r, \alpha)$.

$r_0 = \mathsf{LowBits}_q(r, \alpha)$: Extracts the low bits r_0 of r by internally running $\mathsf{DecomposeMod}(r, \alpha)$.

$h = \mathsf{MakeHint}_q(z, r, \alpha)$: Creates a hint bit h, indicating whether adding z to r changes the high bits of r.

$r_1 = \mathsf{UseHint}_q(h, r, \alpha)$: Uses the hint vector h to obtain the corrected high bits r_1 of r.

Computational Assumption. We recall the standard module LWE assumption, required by the security of ML-DSA. Note that the unforgeability of ML-DSA also relies on (SelfTarget)MSIS, which we omit here.

Definition 1 (Module Learning with Errors (MLWE))**.** *For $q \in \mathbb{N}$ and N a power of two, define $\mathcal{R}_q := \mathbb{Z}_q[x]/(x^N+1)$. For $k, \ell, \eta \in \mathbb{N}$, let $\mathcal{D}$ be some distribution over $\mathcal{R}_q^\ell$. The decision-MLWE problem $\mathsf{MLWE}_{q,N,k,\ell,\mathcal{D},\eta}$ is the task of distinguishing $(\mathbf{A}, \mathbf{A}\mathbf{y}+\mathbf{e})$ from $(\mathbf{A}, \mathbf{u})$ with non-negligible advantage, where $\mathbf{A}$ is uniform in $\mathcal{R}_q^{k \times \ell}$, $\mathbf{y} \leftarrow \mathcal{D}$ and $\mathbf{e}$ is sampled coefficient-wise uniformly from $[-\eta, \eta]$, and $\mathbf{u}$ is uniform in $\mathcal{R}_q^k$. If $\mathcal{D}$ is a uniform distribution over a set S, we write $\mathsf{MLWE}_{q,N,k,\ell,S,\eta}$.*

Parameter =	Value	Description
q	8380417	Polynomial coefficient modulus
N	256	Degree of the polynomial modulus
d	13	Number of dropped bits from the public key

Table 1: Common parameters of ML-DSA.

2.2 Secure Multiparty Computation

We make use of MPC to design our threshold ML-DSA signing protocol. To define and argue security of our protocol we use the Universal Composability (UC) framework [Can01] and refer interested readers to the original works for further details. We also write an introductory self-contained description of UC in Appendix A.2 in the Appendix. In this model, we define *ideal functionalities* (denoted here by $\mathcal{F}_{\mathsf{name}}$) that may receive inputs from the parties, perform a given computation, and may return some outputs. These functionalities are then instantiated by *protocols* (denoted here by Π_{name}) where the parties communicate with each other without any trusted third party. In words, security is defined by means of *simulation*: a given protocol Π_{name} instantiates a functionality $\mathcal{F}_{\mathsf{name}}$ if the interaction of the adversary in the *real world*, where Π_{name} is executed and the honest parties send messages that depend on their (sensitive) inputs, can be simulated in the *ideal world*, where $\mathcal{F}_{\mathsf{name}}$ runs and all the adversary sees is the output.

Our work is set in the *actively secure* model where an adversary can deviate arbitrarily from the protocol execution and yet security is maintained. We do not write this explicitly in our functionalities, but the adversary can cause an abort at any point of the execution—this is called *security with abort*.[4] We assume a model with n parties among which the adversary controls t, where $n = 2t+1$. We achieve *computational security*, but we remark this is only for using simple cryptographic primitives such as PRFs, and other than that our protocols are mostly information-theoretic secure, and in particular are simple and concretely efficient.

[4] For the sake of simplicity we also omit other UC-related details in our functionality descriptions, like the use of sid and ssid's.

Parameter	ML-DSA-44	ML-DSA-65	ML-DSA-87	Description
λ	128	192	256	Security level
(k,ℓ)	$(4,4)$	$(6,5)$	$(8,7)$	Public matrix dimensions
η	2	4	2	Coefficient bound of the secret and error
τ	39	49	60	Hamming weight of the challenge c
γ_1	2^{17}	2^{19}	2^{19}	Coefficient range of $\mathbf{y}$.
γ_2	$95232 = \frac{q-1}{88}$	$261888 = \frac{q-1}{32}$	$261888 = \frac{q-1}{32}$	Truncation parameter
ω	80	55	75	Max hamming weight of hint $\mathbf{h}$
$\beta := \eta \cdot \tau$	78	196	120	Bound on $\|\|c\mathbf{s}\|\|_\infty$ and $\|\|c\mathbf{e}\|\|$.
M_{rep}	4.25	5.09	3.85	Expected # of reps in rej. samp. loop (original)

Table 2: ML-DSA parameters that vary with the security level.

Remark 1 (On the number of parties)**.** *In threshold signing it is common to have a set of N parties that run DKG, among which a given number $n \ll N$ of them must get together to sign, and the adversary cannot sign even when controlling t of the parties. In this work we obviate this, and assume we already have n parties, all of which will sign a message, and the adversary corrupts t of them where $n = 2t+1$. We emphasize this is merely for the sake of exposition, and we can* trivially *accommodate the setting above.*

Remark 2 (On generality)**.** *We describe our functionalities in terms of Shamir secret-sharing, but our core ideas do not rely on any specifics of this scheme, or even the honest majority setting. An alternative take could be to use the so-called* arithmetic black-box *(ABB) model, where all is assumed is a secret-sharing scheme that allows for basic operations such as reconstructions, additions and multiplications (cf. [CEN25]). Such approach would enable different instantiations beyond Shamir secret-sharing and even beyond honest majority. Our core ideas are generic enough to accommodate for this model, and we hope future work in threshold ML-DSA for other settings will benefit from them, but here we chose concreteness and simplicity over generality for the sake of exposition and to better present experimental results.*

2.3 Some Core Functionalities

We consider two finite fields: $\mathbb{F}_q$ where q is the ML-DSA modulus (see Table 1), and $\mathbb{F}_{2^k}$ (degree-k extension of $\mathbb{F}_2$) where $k = \lceil \log n \rceil + 1$.

We use $[\![x]\!]$ to denote Shamir secret-sharing over $\mathbb{F}_q$, and $[\![b]\!]_2$ to denote Shamir secret-sharing over $\mathbb{F}_{2^k}$. For the latter we think of the secret b as being in $\mathbb{F}_2$, and we only use the degree-k extension to get enough interpolation points to define Shamir shares over $\mathbb{F}_2$.

We make use of the following functionalities. These are standard building blocks in MPC and we discuss different instantiations in Appendix Appendix A.3.

- $\mathcal{F}_{\mathsf{Open}}$: receives shares of a secret $[\![x]\!]$ (or $[\![b]\!]_2$). If the shares lie on a degree-t polynomial, reconstructs x (or b) and sends it back to the parties.
- $\mathcal{F}_{\mathsf{Mult}}$: receives shares of two values $[\![x]\!]$ and $[\![y]\!]$ (or $[\![x]\!]_2$ and $[\![y]\!]_2$) and ouputs shares of $[\![x \cdot y]\!]$ (or $[\![x \cdot y]\!]_2$) to the parties.
- $\mathcal{F}_{\mathsf{Coin}}[D]$: samples $x \in D$ at random and sends x to all the parties.
- $\mathcal{F}_{\mathsf{Rand}}[D]$: samples $r \in D$ and sends shares $[\![r]\!]$ to the parties. We write $\mathcal{F}_{\mathsf{Rand}}$ if $D = \mathbb{F}_q$. Our other type of domain D is sampling bounded values.
- $\mathcal{F}_{\mathsf{edabits}}[m]$ [EGK+20]: samples $r \in \mathbb{F}_q$, bit-decomposes it as $r = r_1 + r_2 \cdot 2 + \cdots + r_m \cdot 2^{m-1}$, and distributes shares $([\![r]\!]; [\![r_1]\!]_2, \ldots, [\![r_m]\!]_2)$.
- $\mathcal{F}_{\mathsf{dabits}}[k]$ [RW19]: samples random bits $\mathbf{r} \in \{0,1\}^k$ and distributes shares $([\![\mathbf{r}]\!]; [\![\mathbf{r}]\!]_2)$; i.e., edabits of length-*one*.

3 MPC-friendly ML-DSA Signing

In this section, we present our MPC-friendly variant of ML-DSA in Algorithm 1. The purpose of this section is to prove the extended unforgeability of Algorithm 1 (Theorem 1) and thus justify $\mathcal{F}_{\mathsf{T\text{-}MLDSA}}$ (Appendix C.1) which computes Gen and Sign of Algorithm 1 inside the ideal functionality. We then present protocols that UC-realize $\mathcal{F}_{\mathsf{T\text{-}MLDSA}}$ in Section 4. As mentioned in Section 1.3, a short error vector $\mathbf{e}_w$ with coefficients in $[-\eta, \eta]$ is added to $\mathbf{Ay}$ before decomposition occurs. The description also simultaneously defines the underlying identification scheme $\mathsf{ID} = (\mathsf{Gen}, \mathsf{P}, \mathsf{V})$, in which the prover algorithm consists of $\mathsf{P} = (\mathsf{P}_1, \mathsf{P}_2)$, and the verifier V returns a uniformly sampled $c \in C$ and then accepts if and only if $\mathbf{w}_1 = \mathsf{UseHint}_q(\mathbf{h}, \mathbf{Az} - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2)$, $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$ and $\mathsf{HW}(\mathbf{h}) \le \omega$.

Correctness. We first prove the correctness of a variant of the scheme presented in Algorithm 1, which replaces Line 5 of P_2 with $\mathbf{h} := \mathsf{MakeHint}_q(-c\mathbf{t}_0 + \mathbf{e}_w, \mathbf{Az} - c\mathbf{t} + c\mathbf{t}_0, 2\gamma_2)$ and the first check of Line 6 with $\|c\mathbf{t}_0 - \mathbf{e}_w\|_\infty \ge \gamma_2$, respectively.

Algorithm 1: MPC-friendly ML-DSA (changes from original ML-DSA are highlighted)

$\underline{\mathsf{Gen}()}$

1: $\rho \xleftarrow{\$} \{0,1\}^{256}$
2: $\mathbf{A} := \mathsf{ExpandA}(\rho)$
3: $(\mathbf{s},\mathbf{e}) \xleftarrow{\$} S_\eta^\ell \times S_\eta^k$
4: $\mathbf{t} := \mathbf{A}\mathbf{s}+\mathbf{e} \bmod q$
5: $(\mathbf{t}_1,\mathbf{t}_0) := \mathsf{Power2Round}_q(\mathbf{t},d)$
6: $tr := \mathsf{H}(\rho,\mathbf{t}_1)$
7: $\mathsf{pk} := (\rho,\mathbf{t}_1)$
8: $\mathsf{sk} := (\rho,tr,\mathbf{s},\mathbf{e},\mathbf{t}_1,\mathbf{t}_0)$
9: **return** $(\mathsf{pk},\mathsf{sk})$

$\underline{\mathsf{Sign}(\mathsf{sk}=(\rho,tr,\mathbf{s},\mathbf{e},\mathbf{t}_1,\mathbf{t}_0),\mathsf{m})}$

1: $(\mathbf{w}_1,\mathsf{st}) \leftarrow \mathsf{P}_1(\mathsf{sk})$
2: $\mu := \mathsf{H}(tr,\mathsf{m})$; $c = \mathsf{H}(\mu,\mathbf{w}_1)$
3: $(\mathbf{z},\mathbf{h}) \leftarrow \mathsf{P}_2(\mathsf{st},c)$
4: **return** $(\sigma=(c,\mathbf{z},\mathbf{h}),\mathbf{w}_1)$

$\underline{\mathsf{Ver}(\mathsf{pk}=(\rho,\mathbf{t}_1),\mathsf{m},\sigma=(c,\mathbf{z},\mathbf{h}))}$

1: $\mathbf{A} := \mathsf{ExpandA}(\rho)$
2: $\mu := \mathsf{H}(\mathsf{H}(\rho,\mathbf{t}_1),\mathsf{m})$
3: $\mathbf{w}' := \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d \bmod q$
4: $\mathbf{w}_1' := \mathsf{UseHint}_q(\mathbf{h},\mathbf{w}',2\gamma_2)$
5: **assert**:
$$c = \mathsf{H}(\mu,\mathbf{w}_1') \wedge \|\mathbf{z}\|_\infty < \gamma_1-\beta \wedge \mathsf{HW}(\mathbf{h}) \le \omega$$

$\underline{\mathsf{P}_1(\mathsf{sk}=(\rho,tr,\mathbf{s},\mathbf{e},\mathbf{t}_1,\mathbf{t}_0))}$

1: $\mathbf{A} := \mathsf{ExpandA}(\rho)$
2: $\mathbf{y} \xleftarrow{\$} \tilde{S}_{\gamma_1}^\ell$; $\mathbf{e}_w \xleftarrow{\$} S_\eta^k$
3: $\mathbf{w} := \mathbf{A}\mathbf{y}+\mathbf{e}_w \bmod q$
4: $(\mathbf{w}_1,\mathbf{w}_0) := \mathsf{DecomposeMod}_q(\mathbf{w},2\gamma_2)$
5: $(\mathbf{w}_1,\mathsf{st} = (\mathbf{w}_0,\mathsf{sk},\mathbf{y},\mathbf{e}_w))$

$\underline{\mathsf{P}_2(\mathsf{st},c)}$

1: $\mathbf{z} := c\mathbf{s}+\mathbf{y}$
2: $\bar{\mathbf{w}}_0 := \mathbf{w}_0 - c\mathbf{e}$
3: **if** $\|\mathbf{z}\|_\infty \ge \gamma_1-\beta$ **or** $\|\bar{\mathbf{w}}_0\|_\infty \ge \gamma_2-\beta$ **then return** $(\perp,\perp)$
4: **else**
5: $\quad \mathbf{h} := \mathsf{MakeHint}_q(-c\mathbf{t}_0, \mathbf{A}\mathbf{z}-c\mathbf{t}+c\mathbf{t}_0, 2\gamma_2)$
6: $\quad$ **if** $\|c\mathbf{t}_0\|_\infty \ge \gamma_2$ **or** $\mathsf{HW}(\mathbf{h}) > \omega$ **then** $\mathbf{h} = \perp$
7: $\quad$ **return** $(\mathbf{z},\mathbf{h})$

This variant is already MPC-friendly, as our proof in the next subsection implies that $\mathbf{e}_w$ is not sensitive once both checks in Line 3 of P_2 have passed, allowing us to release $\mathbf{e}_w$ in the clear.

We show that, conditioned on $\mathbf{z} \neq \perp$ and $\mathbf{h} \neq \perp$, the modified scheme is perfectly correct. Recall that Line 4 of Ver satisfies $\mathbf{w}' := \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d \equiv \mathbf{A}\mathbf{z} - c\mathbf{t} + c\mathbf{t}_0 \equiv \mathbf{w} - c\mathbf{e} + c\mathbf{t}_0 - \mathbf{e}_w \pmod q$. By Lemma 3, we have that

$$\begin{aligned}&\mathsf{UseHint}_q(\mathsf{MakeHint}_q(-c\mathbf{t}_0+\mathbf{e}_w,\mathbf{w}',2\gamma_2),\mathbf{w}',2\gamma_2)\\ &\quad= \mathsf{HighBits}_q(\mathbf{w}'-c\mathbf{t}_0+\mathbf{e}_w,2\gamma_2)\\ &\quad= \mathsf{HighBits}_q(\mathbf{w}-c\mathbf{e},2\gamma_2).\end{aligned}$$

For the verification condition to hold, it remains to be shown that $\mathsf{HighBits}_q(\mathbf{w}-c\mathbf{e},2\gamma_2) = \mathsf{HighBits}_q(\mathbf{w},2\gamma_2) = \mathbf{w}_1$. This is handled by the second rejection condition of Line 3 of Sign. By Lemma 5, if $\|\mathbf{w}_0 - c\mathbf{e}\|_\infty < \gamma_2 - \beta$ (as it is guaranteed by Sign outputting a passing transcript) and $\|c\mathbf{e}\|_\infty \le \beta$, then we have that $\mathsf{LowBits}_q(\mathbf{w}-c\mathbf{e},2\gamma_2) < \gamma_2 - \beta$. Then Lemma 4 guarantees $\mathsf{HighBits}_q(\mathbf{w}-c\mathbf{e},2\gamma_2) = \mathsf{HighBits}_q(\mathbf{w},2\gamma_2) = \mathbf{w}_1$. The verification conditions on $\mathbf{z}$ and $\mathbf{h}$ are clearly satisfied due to the corresponding checks on Line 3 and Line 6 of P_2, respectively.

The expected number of iterations caused by Line 3 is identical to that of the original ML-DSA. Let $p_{\mathbf{z}}$ the probability that $\mathbf{z}$ passes the check, and $p_{\bar{\mathbf{w}}_0}$ the probability that $\bar{\mathbf{w}}_0$ passes the check conditioned on $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$. Following §3.4 of the Dilithium specification [BDK+21], the probability that the checks in Line 3 of P_2 succeed is

$$p_{\mathbf{z}} \cdot p_{\bar{\mathbf{w}}_0} \approx e^{-N\cdot(\beta k/\gamma_2 + \beta\ell/\gamma_1)}. \tag{1}$$

In Table 2, we provide the expected numbers of repetitions (denoted by $M_{\mathsf{rep}} = 1/p_{\mathsf{pass}}$) due to the checks in Line 3.

The probability that the checks in Line 6 of P_2 pass changes in theory due to the additional error $\mathbf{e}_w$. However, since we set the parameter such that $\|\mathbf{e}_w\|_\infty \le \eta \ll \|c\mathbf{t}_0\|_\infty$, the empirical probability that Line 6 fails is around 0.02, which remains almost unchanged compared to the ML-DSA scheme [BDK+21].

Optimization: skipping addition by $\mathbf{e}_w$ when computing hint. While the correctness analysis above assumes that MakeHint takes into account the overflow caused by $\mathbf{e}_w$, thanks to our parameter configuration, we can further optimize the construction by dropping $\mathbf{e}_w$ when computing a hint, as presented in Algorithm 1. In our empirical experiments, this optimization does not noticeably affect the probability of the hint growing beyond the size limit, and therefore this optimization does not sacrifice correctness in practice.

3.1 Defining Unforgeability

UF-CMA$_\perp$: Unforgeability under Chosen Message Attacks and Incomplete Signatures. We first discuss our extended version of the unforgeability notion tailored to our modified ML-DSA (or, more generally, signature schemes that sometimes output incomplete signatures), which we call *unforgeability under chosen message attack and incomplete signatures* (UF-CMA$_\perp$). Recall that the standard UF-CMA security game requires that no PPT adversary can produce a valid signature on a message that has never been queried to the signing oracle. While ML-DSA in its original form always outputs a valid signature after repeating the signing process several times, our modified version runs the signing process *only once*, and sometimes outputs an incomplete signature $(\mathbf{w}_1,c,\perp)$ whenever the norm check fails. We opted for this design choice to model an adversary who can potentially obtain an incomplete signature after a single signing attempt, and then leverage this information to adaptively choose messages to query the signing oracle. As the unforgeability game already allows an adversary to query the signing oracle with

Algorithm 2: $\mathsf{UF\text{-}CMA}_\perp$ **Experiment**

$\underline{\mathsf{Exp}^{\mathsf{cma}_\perp}_{\mathcal{A}}(1^\lambda)}$

$M := \varnothing$
$(\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{Gen}(1^\lambda)$
$(\mathsf{m}^*,\sigma^*) \leftarrow \mathcal{A}^{O,|\mathsf{H}\rangle}(\mathsf{pk})$
return $\mathsf{Ver}(\mathsf{pk},\mathsf{m}^*,\sigma^*) = 1 \wedge \mathsf{m}^* \notin M$

$\underline{O(\mathsf{m})}$

$\mu := \mathsf{H}(tr,\mathsf{m})$;
$(\mathbf{w}_1, c, (\mathbf{z},\mathbf{h})) \leftarrow \mathsf{GetTrans}(\mu)$
if $\mathbf{z} \neq \perp$ **then**
    $M := M \cup \{\mathsf{m}\}$
return $(\mathbf{w}_1, c, (\mathbf{z},\mathbf{h}))$

$\underline{\mathsf{GetTrans}(\mu)}$

$(\mathbf{w}_1, \mathsf{st}) \leftarrow \mathsf{P}_1(\mathsf{sk})$
$c = \mathsf{H}(\mu, \mathbf{w}_1)$
$(\mathbf{z},\mathbf{h}) \leftarrow \mathsf{P}_2(\mathsf{st}, c)$
return $(\mathbf{w}_1, c, (\mathbf{z},\mathbf{h}))$

arbitrary messages, this also naturally models the repeated signing process on the same message.

Now, note that variants of the security game can be defined based on when a forgery is considered trivial. In the standard $\mathsf{UF\text{-}CMA}$ notion, any forgery on a message m previously queried to the oracle is deemed trivial, regardless of whether the oracle returned a complete or incomplete signature. However, if the oracle only returns an incomplete signature on m, intuitively, it should be still difficult to produce a valid signature on m. To formalize this intuition, we introduce the stronger $\mathsf{UF\text{-}CMA}_\perp$ security, which allows an adversary to win the game by querying the signing oracle with m^*, obtaining an incomplete signature on m^*, and later outputting a valid signature on m^*. We define the following notion in the quantum random oracle model (QROM) [BDF+11], which is recalled in Appendix B.2.

Definition 2 ($\mathsf{UF\text{-}CMA}_\perp$/$\mathsf{UF\text{-}NMA}$ Security)**.** *A modified ML-DSA scheme is* $\mathsf{UF\text{-}CMA}_\perp$ *secure in the quantum random oracle model if for every polynomial-time adversary* $\mathcal{A}$ *that makes classical queries to the signing oracle* O *and quantum queries to the random oracle* H*, there exists a negligible function* ε *such that* $\Pr[\mathsf{Exp}^{\mathsf{cma}_\perp}_{\mathcal{A}}(1^\lambda) = 1] \leq \varepsilon(\lambda)$*, where* $\mathsf{Exp}^{\mathsf{cma}_\perp}_{\mathcal{A}}$ *is defined in Algorithm 2. Moreover, if* $\mathcal{A}$ *does not have access to the signing oracle* O*, then the scheme is* $\mathsf{UF\text{-}NMA}$ *secure.*

3.2 $\mathsf{UF\text{-}CMA}_\perp$ Security of Modified ML-DSA

We now present our main theorem regarding our modified ML-DSA scheme. We sketch the proof idea here and defer the formal proof to Appendix B.2.

Theorem 1. *The signature scheme presented in Algorithm 1 is* $\mathsf{UF\text{-}CMA}_\perp$ *secure in the (quantum) random oracle model, assuming the* $\mathsf{UF\text{-}NMA}$ *security of the original ML-DSA and under the* $\mathsf{MLWE}_{q,N,k,\ell,\tilde{S}^\ell_{\gamma_1},\eta}$ *and* $\mathsf{MLWE}_{q,N,k,\ell,S^\ell_{\gamma_1-\beta-1},\eta}$ *assumptions.*

Security of Fiat-Shamir with Aborts. Since ML-DSA is constructed by applying the Fiat-Shamir transform to the underlying three-pass identification scheme ID, we first recall standard security proofs for Fiat-Shamir signatures in the QROM (cf. [DFPS23, Theorem 9-10] [GHHM21, Theorem 3]). Instead of proving $\mathsf{UF\text{-}CMA}_\perp$ from scratch, we reduce the $\mathsf{UF\text{-}CMA}_\perp$ security to a weaker notion known as the $\mathsf{UF\text{-}NMA}$ *(unforgeability under no message attack)* security, which only considers adversaries that do not have access to the signing oracle. As the verification algorithm of our modified ML-DSA in Algorithm 1 is identical to that of the original ML-DSA, the $\mathsf{UF\text{-}NMA}$ security of our modified ML-DSA follows from that of the original ML-DSA [BDK+21]. The reduction from $\mathsf{UF\text{-}CMA}_\perp$ to $\mathsf{UF\text{-}NMA}$ boils down to the ability to simulate the signing oracle's reponses without knowing the secret key, which is typically shown by proving that the underlying ID is *strong computational honest-verifier zero-knowledge (scHVZK)* [DFPS23], defined below.

Definition 3 (Strong Computational Honest-Verifier Zero-Knowledge (scHVZK))**.** *An identification protocol* $(\mathsf{Gen},\mathsf{P},\mathsf{V})$ *satisfies the scHVZK property if there exists a polynomial-time simulator* Sim *such that for all probabilistic polynomial-time distinguisher* $\mathcal{A}$*, the following advantage is negligible in* λ*:*

$$\varepsilon_{\mathcal{A}} := \left| \Pr\left[\mathcal{A}(\mathsf{trs},\mathsf{sk}) = 1 : \begin{matrix}(\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{Gen}(1^\lambda)\\ \mathsf{trs} \leftarrow \mathsf{Trans}(\mathsf{sk})\end{matrix}\right] - \Pr\left[\mathcal{A}(\mathsf{trs},\mathsf{sk}) = 1 : \begin{matrix}(\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{Gen}(1^\lambda)\\ \mathsf{trs} \leftarrow \mathsf{Sim}()\end{matrix}\right] \right|$$

where pk *is an implicit argument to* $\mathsf{Trans},\mathsf{Sim},\mathcal{A}$ *above, and* $\mathsf{Trans}(\mathsf{sk})$ *denotes the following procedure:* $(w,\mathsf{st}) \leftarrow \mathsf{P}_1(\mathsf{sk})$*;* $c \xleftarrow{\$} C$*;* $z \leftarrow \mathsf{P}_2(\mathsf{st},c)$*; output* $\mathsf{trs} = (w,c,z)$*.*

Putting it all together. Once scHVZK is proved, we take advantage of the *adaptive reprogramming* lemma by [GHHM21] to reduce the $\mathsf{UF\text{-}CMA}_\perp$ security to the $\mathsf{UF\text{-}NMA}$ security of ML-DSA. As a result, we obtain Theorem 1. Here, to formally prove that incomplete signatures on queried m do not help the adversary to come up with a complete signature on m, we rely on the ability to simulate rejected transcripts *without knowing challenge* c *in advance*. This will essentially allow us to skip reprogramming the random oracle whenever a signing oracle outputs an incomplete signature.

Proving scHVZK. At the heart of Theorem 1 is the ability to simulate both accepting and rejected transcripts. We provide

a sketch of the simulation and proof here and defer the full proof with a concrete reduction loss to Appendix B.1.

Theorem 2. *The modified ML-DSA identification implicit in Algorithm 1 is strongly computational HVZK under the* $\mathsf{MLWE}_{q,N,k,\ell,\tilde{S}^{\ell}_{\gamma_1},\eta}$ *and* $\mathsf{MLWE}_{q,N,k,\ell,S^{\ell}_{\gamma_1-\beta-1},\eta}$ *assumptions. That is, there exists a polynomial-time simulator* Sim *such that for any scHVZK distinguisher* $\mathcal{A}$*, there exist adversaries breaking either* $\mathsf{MLWE}_{q,N,k,\ell,\tilde{S}^{\ell}_{\gamma_1},\eta}$ *or* $\mathsf{MLWE}_{q,N,k,\ell,S^{\ell}_{\gamma_1-\beta-1},\eta}$.

Proof Sketch. The first challenge is that the error we are adding to $\mathbf{w} := \mathbf{A}\mathbf{y} + \mathbf{e}_w$ is too small to show zero-knowledge for the interactive protocol with the first message as $\mathbf{w}$. This is because the verifier can compute $\mathbf{w} + \mathbf{c}\mathbf{t} - \mathbf{A}\mathbf{z} = \mathbf{e}_w + c\mathbf{e}$, where the public key $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}$. If the first message was the full value $\mathbf{w}$, the error $\mathbf{e}_w$ would need to be the size of a flooding term, or an additional rejection sampling must be performed on $c\mathbf{e} + \mathbf{e}_w$ in order to show that the public key error is hidden. Thus, we cannot naively rely on the existing scHVZK proof [DFPS23, PN25] that aims to simulate $(\mathbf{w}, c, \mathbf{z})$.

On the one hand, we have that the accepting transcripts for ML-DSA signatures are *perfectly* simulatable even *without* any error in $\mathbf{w}$ (i.e. $\mathbf{e}_w = \mathbf{0}$). On the other hand, we must show the simulatability of *rejected* transcripts $(\mathbf{w}_1, \mathbf{c}, \perp)$.

The detailed description of the simulator is presented in Algorithm 11. While it is somewhat similar the generic Fiat-Shamir with Aborts (FSwA) simulator of [DFPS23], our simulator is more involved due to the optimizations unique to ML-DSA. This simulator takes in the public key $\mathbf{t}$,[5] and it is parametrized by $p_{\mathbf{z}}$, i.e., the probability that $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$. At the beginning, the simulator samples uniform challenge $c \in C$. The simulator has two modes: With probability $p_{\mathbf{z}}$ (resp. $1 - p_{\mathbf{z}}$), it enters Mode 1 (resp. Mode 2) to simulate the case where $\mathbf{z}$ passed the norm check (resp. where $\mathbf{z}$ failed the norm check). Within Mode 1, the simulator further enters one of the two modes as follows. The simulator in Mode 1 first samples a uniform random $\mathbf{w}' \in \mathcal{R}_q^k$, computes its lower bits $\mathbf{w}'_0$, and checks its norm as in the real prover. If $\mathbf{w}'_0$ fails the norm check, then it enters Mode 1-A; else, it enters Mode 1-B.

Mode 1-A: Simulating $(\mathbf{w}_1, c, \perp)$ for passing $\mathbf{z}$ and rejected $\bar{\mathbf{w}}_0$: In this mode, the simulator samples every element of $\mathbf{w}_1$ "almost" uniformly from $W_1 := \{0, 1, \ldots, (q-1)/(2\gamma_2) - 1\}$, i.e., the range of $\mathsf{Decompose}$. Note that $\mathbf{w}_1$ is not entirely uniform due to the edge case of $\mathsf{Decompose}$, slightly biasing towards 0. At a high-level, we can prove this case is computationally indistinguishable by the MLWE assumption, since $\mathbf{w}' = \mathbf{A}\mathbf{z} + \mathbf{e}_w - c\mathbf{t}$ is pseudorandom thanks to the added noise $\mathbf{e}_w$.

Mode 1-B: Simulating $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}))$ for passing $\mathbf{z}$ and passing $\bar{\mathbf{w}}_0$: For passing transcripts, we maintain the same perfect HVZK property as the original ML-DSA identification. In this mode, the simulator enters the following "loop mode" to freshly sample a passing transcript: sample a uniformly random $\mathbf{z}$ such that $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$, sample a uniform error $\mathbf{e}_w \in S_\eta^k$, compute $\mathbf{w}' = \mathbf{A}\mathbf{z} - c\mathbf{t} + \mathbf{e}_w$, until $\|\mathbf{w}'_0\|_\infty < \gamma_2 - \beta$. This loop ends in expectation after $1/p_{\bar{\mathbf{w}}_0}$ steps, which is a polynomial in λ.[6] As in the standard Fiat-Shamir with aborts proofs, $\mathbf{z}$ is perfectly simulated. We also obtain an identical distribution of $\mathbf{w}_1 = \mathsf{HighBits}_q(\mathbf{A}\mathbf{y} + \mathbf{e}_w, 2\gamma_2)$, since the simulator is able to reproduce *exactly* the same check as in the real prover. This is because $\mathbf{w}'$ computed by the simulator is identically distributed to $\mathbf{w}' = \mathbf{w} - c\mathbf{e}$ in the prover, since

$$\mathbf{A}\mathbf{z} - c\mathbf{t} + \mathbf{e}_w = \mathbf{A}\mathbf{y} + \mathbf{e}_w - c\mathbf{e} = \mathbf{w} - c\mathbf{e}.$$

By Lemma 4, as long as the check $\mathsf{LowBits}(\mathbf{w}', 2\gamma_2) < \gamma_2 - \beta$ passes and by setting $\beta = \tau\eta = \max_{c,\mathbf{e}} \|c\mathbf{e}\|_\infty$, we have $\mathsf{HighBits}_q(\mathbf{w}') = \mathsf{HighBits}_q(\mathbf{w}) = \mathbf{w}_1$, so the overall transcript is perfectly simulated.

Mode 2: Simulating $(\mathbf{w}_1, c, \perp)$ for rejected $\mathbf{z}$: In this mode, the simulator samples a uniform $\mathbf{w} \in \mathcal{R}_q^k$ and outputs its higher bits $\mathbf{w}_1$. Thanks to the additional noise $\mathbf{e}_w$, this mode computationally simulates a real transcript by the MLWE assumption. Note that the the distribution of real $\mathbf{w} = \mathbf{A}\mathbf{y} + \mathbf{e}_w$ is not exactly equivalent to the standard MLWE sample, because it is conditioned on $c\mathbf{s} + \mathbf{y}$ failing the norm check. However, by invoking the tight reduction recently presented by del Pino and Niot [PN25], we can reduce this variant of MLWE to standard MLWE assumptions with bounded uniform secrets.

4 Threshold ML-DSA Protocol

Now we describe our threshold signing protocol for ML-DSA, starting in this section with the online phase. We discuss the instantiation of the offline phase in Section 4.3. Our functionality that captures threshold ML-DSA signing, Functionality $\mathcal{F}_{\mathsf{T\text{-}MLDSA}}$, is presented in Appendix C.1. In essence, $\mathcal{F}_{\mathsf{T\text{-}MLDSA}}$ allows for two commands: $(\textsc{Gen})$, which enables the parties to obtain shares of a secret-key together with its respective public key, and $(\textsc{Sign}, \mathsf{m})$, which allows the parties to obtain the respective signature. $\mathcal{F}_{\mathsf{T\text{-}MLDSA}}$ runs our MPC-friendly description of ML-DSA signing from Appendix 3, since this is the one our protocols will instantiate.

To instantiate $\mathcal{F}_{\mathsf{T\text{-}MLDSA}}$, we rely on a preprocessing functionality that outputs shares of the "commitment message" $\mathbf{w}_1, \mathbf{w}_0$. We model this as Functionality $\mathcal{F}_{\mathsf{Prep}}$ below, which is instantiated in Section 4.3. Note that the functionality outputs the shares of $\mathbf{y}$, $\mathbf{w}_0$, and $\mathbf{w}_1$ over the arithmetic domain.

[5] We assume that all parties hold the full public key $\mathbf{t}$, since providing only $\mathbf{t}_1$ to the verifier is strictly a performance optimization.

[6] To realize a strict polynomial time simulator, we can bound the number of loops by $O(\lambda)$, leading to a negligible statistical loss.

Functionality 1: $\mathcal{F}_{\mathsf{Prep}}$

The functionality proceeds as follows:

1: If the adversary inputs abort, send abort to all parties. Otherwise, continue.
2: $\mathbf{y} \xleftarrow{\$} \tilde{S}^{\ell}_{\gamma_1}$ // $\tilde{S}_{\gamma_1} = S_{\gamma_1} \setminus \{x \in S_{\gamma_1} : \exists x_i \in x, x_i = -\gamma_1\}$
3: $\mathbf{e}_w \xleftarrow{\$} S^{k}_{\eta}$
4: $\mathbf{w} = \mathbf{A}\mathbf{y} + \mathbf{e}_w \bmod q$
5: $(\mathbf{w}_1, \mathbf{w}_0) = \mathsf{DecomposeMod}_q(\mathbf{w}, 2\gamma_2)$
6: Receive $3t$ shares from the adversary, complete $[\![\mathbf{w}_0]\!]$, $[\![\mathbf{w}_1]\!]$, and $[\![\mathbf{y}]\!]$ from these, and send shares to the honest parties.

In addition to $\mathcal{F}_{\mathsf{Prep}}$, the parties also need a functionality for *secure comparison*, which enables performing rejection sampling securely in MPC. This functionality, modeled as $\mathcal{F}_{\mathsf{LTC}}$ (which stands for *less than constant*) takes as input shares of a value $[\![x]\!]$, a public bound B, and outputs $[\![b]\!]$ where $b = (x < B)$. Functionality $\mathcal{F}_{\mathsf{LTC}}$ appears in Appendix C.1. $\mathcal{F}_{\mathsf{LTC}}$ is a fundamental building block in many MPC applications, and it admits many different instantiations depending on a range of trade-offs. We discuss concrete instantiations $\mathcal{F}_{\mathsf{LTC}}$ in Appendix C.2.

4.1 Performing Rejection Sampling Securely

One of the core operations in ML-DSA signing is *rejection sampling*, which consists of comparing certain values against some given bounds, and restarting the signing process if any of the values fall out-of-bounds. We present Functionality $\mathcal{F}_{\mathsf{RejSamp}}$ in Appendix C.1, which takes in secret sharings, compares their secret values to the relevant bounds, and outputs either 1 (fail) or 0 (pass) to the parties.

Below, we present Protocol Π_{RejSamp}, which realizes this functionality. Overall, the parties call $\mathcal{F}_{\mathsf{LTC}}$ to perform the comparisons, and the problem is reduced to a functionality $\mathcal{F}_{\mathsf{BatchedOR}}$ that checks whether an array of shared bits contains at least one 1, which here indicates at least one sample was rejected. We instantiate this via Protocol $\Pi_{\mathsf{BatchedOR}}$, which works by taking κ random linear combinations, opening them, and verifying all of them are zero. Unfortunately, this approach on its own does not work since it leaks non-trivial information on which values failed the check (to see this, consider the extreme example where many linear combinations are open: it would eventually leak the vector of checks itself), and this leakage is not allowed by our signature functionality. In fact, in our case, hiding the position of out-of-bound coefficients in $\mathbf{z}$ is crucial especially because we reveal challenge c corresponding to a rejected signature. Combining this information together, it has been shown that a key recovery attack is possible [ZWSY25]. To securely verify these κ "check bits" b^i shared over $\mathbb{F}_2$ efficiently with no leakage, the parties will first lift them to a different, large field $\mathbb{F}_p$ ($|\mathbb{F}_p| \geq 2^{\kappa}$) using *dabits* [RW19]. They then combine these bits by simply adding them together over $\mathbb{F}_p$ and finally multiply the sum by a random element $u \in \mathbb{F}_p$ to re-randomize. Finally, they open $y = (\sum_{i=1}^{\kappa} b^{(i)}) \cdot u$, and if $y \neq 0$, then we know $v^{(j)} = 1$ for at least one $j \in [\ell]$ with overwhelming probability and hence the rejection sampling failed.

Functionality 2: $\mathcal{F}_{\mathsf{BatchedOR}}$

Input: On receiving shares $[\![\mathbf{v}]\!]_2$ of a vector $\mathbf{v} \in \{0,1\}^{\ell}$ from the honest parties:

1: Reconstruct $\mathbf{v}$ alongside the corrupt parties' shares, and send the shares to the adversary.
2: If all entries of $\mathbf{v}$ are 0, set $b \leftarrow 0$.
3: Else, set $b \leftarrow 1$ to all parties.
4: Send bit b to the adversary.
5: Once the adversary replies with continue, output b to the parties.

Protocol 3: $\Pi_{\mathsf{BatchedOR}}$

Parameters: Prime p such that $p \geq 2^{\kappa}$.
Input: Secret-shared $[\![\mathbf{v}]\!]_2$ of length ℓ. Proceed as follows:

1: **for** i from 1 to κ **do** $(s_1^{(i)}, \ldots, s_{\ell}^{(i)})_{i=1}^{\kappa} \leftarrow \mathcal{F}_{\mathsf{Coin}}[\{0,1\}^{\ell}]$
2: **for** i from 1 to κ **do** compute $[\![b^{(i)}]\!]_2 \leftarrow \left(\bigoplus_{j=1}^{\ell} s_j^{(i)} \cdot [\![v_j]\!]_2\right)$
3: Call $\mathcal{F}_{\mathsf{dabits}}[1]$, κ times, to receive $([\![r^{(1)}]\!]_p; [\![r^{(1)}]\!]_2), \ldots, ([\![r^{(\kappa)}]\!]_p; [\![r^{(\kappa)}]\!]_2)$
4: Parties compute and open $c^{(i)} \leftarrow \mathcal{F}_{\mathsf{Open}}([\![b^{(i)}]\!]_2 \oplus [\![r^{(i)}]\!]_2)$ for $i \in [\kappa]$
5: Parties compute $[\![b^{(i)}]\!]_p \leftarrow c^{(i)} + (1 - 2 \cdot c^{(i)})[\![r^{(i)}]\!]_p$ for $i \in [\kappa]$
6: Parties compute $[\![b]\!]_p \leftarrow \sum_{i=1}^{\kappa} [\![b^{(i)}]\!]_p$
7: Parties call $[\![u]\!]_p \leftarrow \mathcal{F}_{\mathsf{Rand}}$
8: Parties compute $[\![y]\!]_p \leftarrow \mathcal{F}_{\mathsf{Mult}}([\![b]\!]_p, [\![u]\!]_p)$
9: Open $y \leftarrow \mathcal{F}_{\mathsf{Open}}([\![y]\!]_p)$
10: **if** $y \neq 0$ **then return** 1. // indicates $\exists j \in [\ell] : v_j = 1$.
11: **else return** 0.

The following is proved in Section C.5 in the Appendix.

Lemma 1. *$\Pi_{\mathsf{BatchedOR}}$ UC-realizes $\mathcal{F}_{\mathsf{BatchedOR}}$ in the $(\mathcal{F}_{\mathsf{Coin}}, \mathcal{F}_{\mathsf{Rand}}, \mathcal{F}_{\mathsf{Mult}}, \mathcal{F}_{\mathsf{Open}})$-hybrid model.*

For Π_{RejSamp}, note that we actually check bounds on an *absolute value* $\|z\| < B$. Over integers this would correspond to $-B < z < B$, but since negative values are represented in $\mathbb{F}_q$ by integers in the range $[(q+1)/2, q)$, we have that $\|z\| < B$ iff $0 \leq z < B$ or $q - B < z < q$. We can shift this by $B - 1$ to equivalently get

$$\|z\| < B \iff \begin{cases} B - 1 \leq z + B - 1 < 2B - 1 \\ \textbf{or} \\ q - 1 < z + B - 1 < q + B - 1. \end{cases}$$

Since z is represented in $\mathbb{F}_q := \{0, 1, \ldots, q-1\}$, the latter inequality wraps around and becomes $0 \leq (z + B - 1) \bmod$

$q < B$, so the two inequalities together translate to $(z + B - 1) \bmod q < 2B - 1$, which is the strict-less-than operation that we will compute. We do so using the functionality $\mathcal{F}_{\mathsf{LTC}}$.

Protocol 4: $\Pi_{\mathsf{RejSamp}}([\![\mathbf{z}]\!], [\![\bar{\mathbf{w}}_0]\!])$

Inputs: Secret-shared $[\![\mathbf{z}]\!]$ (length $\ell \cdot N$) and $[\![\bar{\mathbf{w}}_0]\!]$ (length $k \cdot N$). Proceed as follows:

1: Define $k_x := \ell N$ and $k_y := kN$. Define $B_x := \gamma_1 - \beta$ and $B_y := \gamma_2 - \beta$.
2: **for** i from 1 to k_x **do**
3: $\quad [\![u_i]\!]_2 \leftarrow 1 - \mathcal{F}_{\mathsf{LTC}}[2B_x - 1]([\![\mathbf{z}_i + B_x - 1]\!])$ // $u_i = 0$ if $\mathbf{z}_i + B_x - 1 < 2B_x - 1$; $u_i = 1$ o.w.
4: **for** i from 1 to k_y **do**
5: $\quad [\![u'_i]\!]_2 \leftarrow 1 - \mathcal{F}_{\mathsf{LTC}}[2B_y - 1]([\![\bar{\mathbf{w}}_{0,i} + B_y - 1]\!])$
6: Concatenate $(u_i)_{i=1}^{k_x}$ and $(u'_i)_{i=1}^{k_y}$ as $\mathbf{v} \in \mathbb{F}_q^{k_x + k_y}$
7: Let $b \leftarrow \mathcal{F}_{\mathsf{BatchedOR}}([\![\mathbf{v}]\!]_2)$. // 1 means at least one check failed
8: **return** b // 1 means rejection sampling failed

The following is proved in Section C.5 in the Appendix.

Lemma 2. Π_{RejSamp} *UC-realizes* $\mathcal{F}_{\mathsf{RejSamp}}$ *in the* $(\mathcal{F}_{\mathsf{LTC}}, \mathcal{F}_{\mathsf{BatchedOR}})$*-hybrid model.*

4.2 Threshold ML-DSA: Online Phase

Now we are ready to present our threshold ML-DSA signing protocol, assuming the functionality $\mathcal{F}_{\mathsf{RejSamp}}$, and also the preprocessing functionality $\mathcal{F}_{\mathsf{Prep}}$.

Protocol 5: $\Pi_{\mathsf{T\text{-}MLDSA}}$

Key generation: Upon receiving (GEN) as input:

1: All parties call $\rho \leftarrow \mathcal{F}_{\mathsf{Coin}}[\{0,1\}^{256}]$, $[\![\mathbf{s}]\!] \leftarrow \mathcal{F}_{\mathsf{Rand}}[S_\eta^\ell]$ and $[\![\mathbf{e}]\!] \leftarrow \mathcal{F}_{\mathsf{Rand}}[S_\eta^k]$
2: $\mathbf{A} := \mathsf{ExpandA}(\rho)$
3: Locally compute $[\![\mathbf{t}]\!] = \mathbf{A}[\![\mathbf{s}]\!] + [\![\mathbf{e}]\!]$
4: Open $\mathbf{t} \leftarrow \mathcal{F}_{\mathsf{Open}}([\![\mathbf{t}]\!])$
5: Parties locally compute $(\mathbf{t}_1, \mathbf{t}_0) = \mathsf{Power2Round}_q(\mathbf{t}, d)$ // $\mathbf{t}$ is public
6: Compute $tr := \mathsf{H}(\rho, \mathbf{t}_1)$
7: Let $\mathsf{pk} = (\rho, \mathbf{t}_1)$
8: Parties store $(\rho, tr, \mathbf{t}, \mathbf{t}_1, \mathbf{t}_0, [\![\mathbf{s}]\!], [\![\mathbf{e}]\!])$ and **return** pk.

Preprocessing: Upon receiving (PREP) as input, the parties call $([\![\mathbf{w}_0]\!], [\![\mathbf{w}_1]\!], [\![\mathbf{y}]\!]) \leftarrow \mathcal{F}_{\mathsf{Prep}}$ and store the output.

Sign: Upon receiving $(\mathrm{SIGN}, \mathsf{m})$ as input, proceed as follows:

1: Compute $\mu := \mathsf{H}(tr, \mathsf{m})$
2: Open $\mathbf{w}_1 \leftarrow \mathcal{F}_{\mathsf{Open}}([\![\mathbf{w}_1]\!])$, and locally compute $c := \mathsf{H}(\mu, \mathbf{w}_1)$
3: Locally compute $[\![\mathbf{z}]\!] = c \cdot [\![\mathbf{s}]\!] + [\![\mathbf{y}]\!]$ and $[\![\bar{\mathbf{w}}_0]\!] = [\![\mathbf{w}_0]\!] - c \cdot [\![\mathbf{e}]\!]$
4: $b \leftarrow \mathcal{F}_{\mathsf{RejSamp}}([\![\mathbf{z}]\!], [\![\bar{\mathbf{w}}_0]\!])$.
5: **if** $b = 1$ **then return** $(\mathbf{w}_1, c, \perp, \perp)$.
6: **else** open $\mathbf{z} \leftarrow \mathcal{F}_{\mathsf{Open}}([\![\mathbf{z}]\!])$ and compute hint $\mathbf{h} = \mathsf{MakeHint}_q(-c\mathbf{t}_0, \mathbf{Az} - c\mathbf{t} + c\mathbf{t}_0, 2\gamma_2)$.
7: **if** $\|c\mathbf{t}_0\|_\infty \geq \gamma_2$ or $\mathsf{HW}(\mathbf{h}) > \omega$ **then return** $(\mathbf{w}_1, c, \mathbf{z}, \perp)$
8: **else return** $(\mathbf{w}_1, c, \mathbf{z}, \mathbf{h})$.

The following is proved in Section C.5 in the Appendix.

Theorem 3. $\Pi_{\mathsf{T\text{-}MLDSA}}$ *UC-realizes* $\mathcal{F}_{\mathsf{T\text{-}MLDSA}}$ *in the* $(\mathcal{F}_{\mathsf{Coin}}, \mathcal{F}_{\mathsf{Rand}}, \mathcal{F}_{\mathsf{Open}}, \mathcal{F}_{\mathsf{Prep}}, \mathcal{F}_{\mathsf{RejSamp}})$*-hybrid model.*

4.3 Threshold ML-DSA: Offline Phase

We now describe the instantiation of the $\mathcal{F}_{\mathsf{Prep}}$ functionality, which is part of the offline (*i.e.* message-independent phase).[7] Due to space constraints, we will only present Π_{Prep} for ML-DSA-65 and ML-DSA-87 in the main body; ML-DSA-44 requires a different instantiation due to the fact that δ in DecomposeMod for this case is not a power-of-two, unlike in the other cases. We present $\Pi_{\mathsf{Prep\text{-}44}}$ in Appendix 4.3.1.

For instantiating $\mathcal{F}_{\mathsf{Prep}}$ for ML-DSA-65 and ML-DSA-87, we will make use of a bit decomposition functionality, $\mathcal{F}_{\mathsf{BitDec}}$, that takes as input sharings $w \in \mathbb{F}_q$ and returns binary sharings of their bit-decomposition. We describe it as Functionality $\mathcal{F}_{\mathsf{BitDec}}$ in Section C.1 in the Appendix. This functionality appears in several applications of MPC where direct access to the bits is needed. Several instantiations appear in the literature, and we discuss this in Section C.4 in the Appendix.

Lines 3-9 in Π_{Prep} below capture our MPC instantiation of DecomposeMod for ML-DSA 65 and 87, where $\delta = (q-1)/(2\gamma_2)$ is a power-of-2. The following theorem is proved in Section C.5 in the Appendix.

Theorem 4. Π_{Prep} *UC-realizes* $\mathcal{F}_{\mathsf{Prep}}$ *in the* $(\mathcal{F}_{\mathsf{Rand}}, \mathcal{F}_{\mathsf{BitDec}}, \mathcal{F}_{\mathsf{dabits}}, \mathcal{F}_{\mathsf{Open}})$*-hybrid model.*

Protocol 6: Π_{Prep}

Parameters The bit-length $m := \lceil \log q \rceil$. A power-of-2 modulus $\delta := \frac{q-1}{2\gamma_2}$ and $d = \log \delta$.

1: Call $[\![\mathbf{y}]\!] \leftarrow \mathcal{F}_{\mathsf{Rand}}[\tilde{S}_{\gamma_1}^\ell]$ and $[\![\mathbf{e}_w]\!] \leftarrow \mathcal{F}_{\mathsf{Rand}}[S_\eta^k]$.
2: Locally compute $[\![\mathbf{w}]\!] = \mathbf{A}[\![\mathbf{y}]\!] + [\![\mathbf{e}_w]\!]$
3: Locally compute $[\![\mathbf{s}]\!] \leftarrow -\delta \cdot [\![\mathbf{w}]\!] + (q-1)/2$
4: Call $([\![\mathbf{s}_1]\!]_2, \ldots, [\![\mathbf{s}_m]\!]_2) \leftarrow \mathcal{F}_{\mathsf{BitDec}}([\![\mathbf{s}]\!])$
5: Call $\mathcal{F}_{\mathsf{dabits}}[k]$, d times, to receive $([\![\mathbf{r}_1]\!]; [\![\mathbf{r}_1]\!]_2), \ldots, ([\![\mathbf{r}_d]\!]; [\![\mathbf{r}_d]\!]_2)$
6: Compute and open $\mathbf{c}_i \leftarrow \mathcal{F}_{\mathsf{Open}}([\![\mathbf{s}_i]\!]_2 \oplus [\![\mathbf{r}_i]\!]_2)$ for $i \in [d]$
7: Locally compute $[\![\mathbf{s}_i]\!] \leftarrow \mathbf{c}_i + (1 - 2 \cdot \mathbf{c}_i)[\![\mathbf{r}_i]\!]$ for $i \in [d]$
8: Locally compute $[\![\mathbf{w}_1]\!] \leftarrow \sum_{i=1}^d 2^{i-1} \cdot [\![\mathbf{s}_i]\!]$
9: Locally compute $[\![\mathbf{w}_0]\!] \leftarrow [\![\mathbf{w}]\!] - (2\gamma_2) \cdot [\![\mathbf{w}_1]\!]$
10: **return** $[\![\mathbf{w}_0]\!], [\![\mathbf{w}_1]\!], [\![\mathbf{y}]\!]$

[7] We remark that the offline phase of our end-to-end protocol is not restricted to Π_{Prep}: instantiating $\mathcal{F}_{\mathsf{LTC}}$ also requires some preprocessed data in the form of dabits and edabits. See Section C.2 in the Appendix.

4.3.1 ML-DSA-44 Offline Phase

The protocol Π_{Prep} from Section 4.3 only captures the case where the operation "$s \bmod^{+} \delta$" in DecomposeMod is equivalent to the extraction of $d = \log \delta$ LSBs. As $\delta = 44$ is not a power of two in ML-DSA-44, we need a separate protocol to handle this case. The essential building block is $\pi_{\mathsf{Mod\text{-}44}}$, which takes as input (a vector of) sharings $[\![r]\!]$ of $r \in \mathbb{Z}_q$ together with its bit-wise sharings over the binary domain, and outputs a sharing $[\![r \bmod \delta]\!]$. Intuitively, $\pi_{\mathsf{Mod\text{-}44}}$ exploits the fact that $\mathbb{Z}_q^*$ has a multiplicative (cyclic) subgroup $G = \langle g \rangle$ of order δ, as δ divides $q-1$. Therefore, for $r \in \mathbb{Z}_q$, one can obtain $r \bmod \delta$ by computing the discrete logarithm of $g^r \bmod q$ with respect to the generator g. To extract the discrete logarithm, we define a polynomial $F(\mathrm{x}) = a_0 + a_1 \cdot \mathrm{x} + \cdots + a_\delta \cdot \mathrm{x}^\delta$, such that $F(g^i \bmod q) = i$ for $i \in [0, \delta - 1]$, and thus we have that $F(g^r \bmod q) = r \bmod \delta$. Thus, computing $[\![r \bmod \delta]\!]$ boils down to evaluating the public polynomial F at the point $g^r \bmod q$ in a multi-party fashion.

The procedure $\pi_{\mathsf{Mod\text{-}44}}$ is then used in $\Pi_{\mathsf{Prep\text{-}44}}$ to instantiate the preprocessing for ML-DSA-44. Both are formally presented below. The proof of the following theorem is deferred to Appendix C.5.

Theorem 5. *$\Pi_{\mathsf{Prep\text{-}44}}$ UC-realizes $\mathcal{F}_{\mathsf{Prep}}$ in the $(\mathcal{F}_{\mathsf{Rand}}, \mathcal{F}_{\mathsf{BitLTC}}, \mathcal{F}_{\mathsf{edabits}}, \mathcal{F}_{\mathsf{dabits}}, \mathcal{F}_{\mathsf{Mult}}, \mathcal{F}_{\mathsf{Open}})$-hybrid model.*

Procedure 7: $\pi_{\mathsf{Mod\text{-}44}}([\![\mathbf{r}]\!]; [\![\mathbf{r}_1]\!]_2, \ldots, [\![\mathbf{r}_m]\!]_2)$

Parameters:
- Generator g of order 44 subgroup $G \subset \mathbb{Z}_q^*$ and elements $h_1 = g^1, h_2 = g^2, h_3 = g^{2^2}, \ldots, h_m = g^{2^{m-1}}$.
- Polynomial $F(\mathbf{x}) = a_0 + a_1 \cdot \mathbf{x} + \cdots + a_{43} \cdot \mathbf{x}^{43}$, mapping elements of G injectively to $[44]$.

// Compute shares of g^r

Call $\mathcal{F}_{\mathsf{dabits}}[k]$, m times, to receive $([\![\mathbf{s}_1]\!]; [\![\mathbf{s}_1]\!]_2), \ldots, ([\![\mathbf{s}_m]\!]; [\![\mathbf{s}_m]\!]_2)$
Compute and open $\mathbf{c}_i \leftarrow \mathcal{F}_{\mathsf{Open}}([\![\mathbf{r}_i]\!]_2 \oplus [\![\mathbf{s}_i]\!]_2)$ for $i \in [m]$
Locally compute $[\![\mathbf{r}_i]\!] \leftarrow \mathbf{c}_i + (1 - 2 \cdot \mathbf{c}_i)[\![\mathbf{s}_i]\!]$ for $i \in [m]$
Locally compute $[\![\mathbf{u}_i]\!] \leftarrow [\![\mathbf{r}_i]\!] \cdot h_i + (1 - [\![\mathbf{r}_i]\!])$
for i from 1 to $\log(m)$ **do**
  for j from 1 to $\log(m)/2^i$ **do** // in parallel
    Compute $[\![\mathbf{u}_j]\!] \leftarrow \mathcal{F}_{\mathsf{Mult}}([\![\mathbf{u}_{2j-1}]\!], [\![\mathbf{u}_{2j}]\!])$

// Compute r mod 44 based on $F(\mathbf{x})$

Let $[\![\mathbf{x}_0]\!] \leftarrow 1$ and $[\![\mathbf{x}_1]\!] \leftarrow [\![\mathbf{u}_1]\!]$
for i from 0 to $\lfloor \log(44) \rfloor$ **do**
  for j from 0 to $2^i - 1$ **do**
    Let $[\![\mathbf{x}_{2^{i+1}-j}]\!] \leftarrow \mathcal{F}_{\mathsf{Mult}}([\![\mathbf{x}_{2^i}]\!], [\![\mathbf{x}_{2^i-j}]\!])$
Let $[\![\mathbf{r} \bmod 44]\!] \leftarrow 0$
for i from 0 to 44 **do**
  Compute $[\![\mathbf{r} \bmod 44]\!] \leftarrow [\![\mathbf{r} \bmod 44]\!] + a_i \cdot [\![\mathbf{x}_i]\!]$
return $[\![\mathbf{r} \bmod 44]\!]$

Protocol 8: $\Pi_{\mathsf{Prep\text{-}44}}$

Parameters The bit-length $m := \lceil \log q \rceil$. A modulus $\delta = 44$.

Call $[\![\mathbf{y}]\!] \leftarrow \mathcal{F}_{\mathsf{Rand}}[\tilde{S}_{\gamma_1}^{\ell}]$ and $[\![\mathbf{e}_w]\!] \leftarrow \mathcal{F}_{\mathsf{Rand}}[S_\eta^k]$.
Locally compute $[\![\mathbf{w}]\!] = \mathbf{A}[\![\mathbf{y}]\!] + [\![\mathbf{e}_w]\!]$
Locally compute $[\![\mathbf{s}]\!] \leftarrow -\delta \cdot [\![\mathbf{w}]\!] + (q-1)/2$
Call $\mathcal{F}_{\mathsf{edabits}}[m]$, k times, to receive $([\![r^1]\!]; [\![r_1^1]\!]_2, \ldots, [\![r_m^1]\!]_2), \ldots ([\![r^k]\!]; [\![r_1^k]\!]_2, \ldots, [\![r_m^k]\!]_2)$. Represent this as a vector of edabits $([\![\mathbf{r}]\!]; [\![\mathbf{r}_1]\!]_2, \ldots, [\![\mathbf{r}_m]\!]_2)$
Compute and open $\mathbf{a} \leftarrow \mathcal{F}_{\mathsf{Open}}([\![\mathbf{s}]\!] + [\![\mathbf{r}]\!])$
Compute $[\![\mathbf{b}]\!]_2 \leftarrow \mathcal{F}_{\mathsf{BitLTC}}[\mathbf{a}]([\![\mathbf{r}_1]\!]_2, \ldots, [\![\mathbf{r}_m]\!]_2)$
Call $([\![\mathbf{u}]\!]; [\![\mathbf{u}]\!]_2) \leftarrow \mathcal{F}_{\mathsf{dabits}}[k]$
Compute and open $\mathbf{d} \leftarrow \mathcal{F}_{\mathsf{Open}}([\![\mathbf{b}]\!]_2 + [\![\mathbf{u}]\!]_2)$
Locally compute $[\![\mathbf{b}]\!] \leftarrow \mathbf{d} + (1 - 2 \cdot \mathbf{d})[\![\mathbf{u}]\!]$
Call $[\![\mathbf{r} \bmod \delta]\!] \leftarrow \pi_{\mathsf{Mod\text{-}44}}([\![\mathbf{r}]\!]; [\![\mathbf{r}_1]\!]_2, \ldots, [\![\mathbf{r}_m]\!]_2)$
Locally compute $[\![\mathbf{w}_1]\!] \leftarrow \mathbf{a} \bmod \delta + (1 - [\![\mathbf{b}]\!]) * (q \bmod \delta) - [\![\mathbf{r} \bmod \delta]\!]$
Locally compute $[\![\mathbf{w}_0]\!] \leftarrow [\![\mathbf{w}]\!] - (2\gamma_2) \cdot [\![\mathbf{w}_1]\!]$
return $[\![\mathbf{w}_0]\!], [\![\mathbf{w}_1]\!], [\![\mathbf{y}]\!]$

5 Performance and Experiments

We now discuss the concrete performance of our threshold signing protocol. For the online phase, we report both communication and runtimes derived from an implementation of our protocol. For the offline phase, we only report communication and round costs. For $\mathcal{F}_{\mathsf{Mult}}$ we first generate triples using DN07 [DN07], which costs $9.5n$ field elements in total, and then for the online phase two calls to $\mathcal{F}_{\mathsf{Open}}$ are required. We instantiate $\mathcal{F}_{\mathsf{Open}}$ in two possible ways: (1) via all-to-all reconstruction, which has a cost of $n \cdot (n-1)$ field elements in one round, and (2) via "multiple kings" as in [DN07], which costs $4n$ elements in two rounds. Recall that, due to limitations with Shamir secret-sharing, shares of binary secrets are actually defined over $\mathbb{F}_{2^k}$. We use a simple "packing optimization" that allows reconstruction of a batch of k secrets over $\mathbb{F}_2$, at the cost of one element in $\mathbb{F}_{2^k}$. We refer the reader to Section A.3 for details on these instantiations and the derivation of their costs. We set the statistical security parameter κ to 40.

5.1 Communication, Rounds, and Benchmarks

We present communication and rounds for both offline phase (i.e., PREP command in $\Pi_{\mathsf{T\text{-}MLDSA}}$) and online phase (i.e., SIGN command in $\Pi_{\mathsf{T\text{-}MLDSA}}$) of our protocol. We also provide runtime benchmarks for the online phase. See Table 3 and Table 4 for communication costs. We consider two variants of our protocol that differ on the method used for reconstruction: either all-to-all or king-based, as discussed above. SIGN involves:

1. One arithmetic opening,

2. One call to $\mathcal{F}_{\mathsf{RejSamp}}$,

3. In case of *accept*, one more opening.

Note that, for the communication-optimized version, the communication per party remains *constant*, whereas for the round-optimized variant this communication grows linearly with the number of parties. However, for a small number of parties, we observe that the round-optimized approach is likely the better option, since only a slight increase in the communication is required to halve the number of rounds. As the number of parties increases, the linear scaling in the round-optimized communication begins to diverge from the communication-optimized version. Applications with a large number of parties will likely favor the communication-optimized version, and we discuss below how to minimize the total number of expected rounds when generating a signature.

# of parties		3	7	15	31	63	Rounds
MLDSA-44	C	0.31	0.31	0.31	0.31	0.31	29
	R	0.15	0.47	1.1	2.36	4.87	16
MLDSA-65	C	0.43	0.43	0.43	0.43	0.43	29
	R	0.21	0.65	1.5	3.24	6.7	16
MLDSA-87	C	0.59	0.59	0.59	0.59	0.59	29
	R	0.29	0.88	2.06	4.42	9.14	16

Table 3: Online communication in MB per party for a successful SIGN command. For n parties, we set $t = (n-1)/2$. The C rows are communication-optimized and the R rows are round-optimized.

# of parties	3	7	15	31	63	Rounds
MLDSA-44	6	15.9	40.6	110.4	330.1	86 – 108
MLDSA-65	4.1	11.9	34.4	108.4	369.9	81 – 103
MLDSA-87	5.5	16	46.3	146	499	81 – 103

Table 4: Offline communication in MB per party for a successful SIGN command. For n parties, we set $t = (n-1)/2$. We also report round-count, given as a range since for the offline phase the number of rounds increases as the number of parties grow (this is due to the edabits protocol).

Full signature generation: parallel rejection sampling. As in the original ML-DSA scheme, the majority of invocations of the SIGN command will fail to output a complete signature. Table 2 gives the expected number of times the SIGN command must be run before a full signature is produced. Our variant of the ML-DSA signing algorithm has essentially the same failure rate, so parties can expect to invoke the SIGN command about four or five times before a signature is successfully produced.

For some applications, it may be acceptable to evaluate each SIGN command sequentially and only invoke the next SIGN command when the rejection sampling test fails. However, for most applications, this high variability in the number of rounds could cause unacceptable latency. To address this, we consider an approach[8] where the parties run several independent iterations of SIGN in parallel and open the first $\mathbf{z}$ value that passes the rejection sampling test. These parallel invocations of SIGN dramatically reduce the variance in the number of rounds while only slightly increasing the expected communication bandwidth of the protocol.

In Table 5, we give communication benchmarks for parallel invocations of the SIGN command. Note that we only perform the final opening of $\mathbf{z}$ on a *single* passing invocation; any other passing invocations must be discarded without opening the full signature. The table reports the expected performance of running multiple SIGN commands in parallel. Observe in Table 5 that evaluating more SIGN commands in parallel decreases the expected round-complexity but increases the expected communication.

	# parallel SIGN	1	2	4	8
	$\mathbb{E}$[# of Π_{RejSamp} iters]	5.09	2.78	1.69	1.20
Comm. opt.	$\mathbb{E}$[# of rounds]	149	82	50	35
	$\mathbb{E}$[per party comm.]	2.2	2.4	2.9	4.2
Round opt.	$\mathbb{E}$[# of rounds]	81	45	27	19
	$\mathbb{E}$[per party comm.]	7.6	8.5	10.3	14.5

Table 5: Expected number of rounds and communication in MB per party to generate an ML-DSA signature. We analyze this for both the round and communication-optimized versions of our online phase, in a setting with 15 parties generating ML-DSA-65 signatures.

Runtimes. We implemented[9] our circuits using Dalskov's C++ secure computation library (SCL) [Dal22]. In Table 6, we give the runtimes for one round of rejection sampling ignoring network effects. Our benchmarks were run on a single thread of a machine with 32 GB of RAM with an Intel i7 chip running at 2.5 GHz. Additionally, we do not consider pipelining optimizations that would likely remove the computation from the overall wall-clock times. For example, in a WAN setting with a round-trip-time of 20 milliseconds, the overhead from network latency would be about 160 milliseconds for the round-optimized version and about 290 milliseconds for the communication-optimized version. In a pipelined implementation, the network operations would run in parallel to the computation, so the overall wall-clock time would be heavily dominated by the network.

[8] Note that our approach is different from the one considered in [DOTT22, PN25], in which a signature is generated only if all parties simultaneously succeed in local rejection sampling on *each share of* $\mathbf{z}$. In contrast, our protocol allows the parties to *jointly* perform rejection sampling on the shared $\mathbf{z}$, thereby preserving the expected number of restarts of the original ML-DSA scheme.

[9] https://doi.org/10.5281/zenodo.17888654

# of parties	3	7	15	31	63
ML-DSA-44	6.3	8.9	18.7	52.2	196.4
ML-DSA-65	8.7	12.5	25.7	70.4	261.2
ML-DSA-87	11.9	16.9	34.8	94.6	343.1

Table 6: Online compute times in ms ignoring network effects. All parties are evaluated in parallel.

5.2 Comparison to Related Works

In Table 7, we compare our protocol to similar related works, including the concurrent work by Borin et al. [BCdP+25]. As noted in Section 1.2, while [BCdP+25] supports dishonest majority and is more efficient in terms of communication and rounds for a small number of parties, ours achieves universal composability assuming honest majority and supports an arbitrary number of parties. To illustrate the cost of achieving arbitrary scalability and UC-security while remaining compatible with the ML-DSA verification process, we compare ours to (threshold) Raccoon [DKM+24], EKT [EKT24], Ringtail [BKL+25], and Finally! [PN25]. These protocols are all based on the Raccoon signature scheme [dPKPR24] and are proven secure under variants of game-based unforgeability notions [BTZ22] in the dishonest majority setting. While our communication and round counts are significantly higher, the signature and verification sizes remain the smallest thanks to the ML-DSA parameters.

Acknowledgments

We thank Anders Dalskov for answering our questions regarding the SCL library [Dal22], Katharina Boudgoust for discussions on the security of ML-DSA, and the anonymous reviewers of USENIX Security 2026 for their valuable feedback and suggestions.

Disclaimer

References

[ACE+21] Mark Abspoel, Ronald Cramer, Daniel Escudero, Ivan Damgård, and Chaoping Xing. Improved single-round secure multiplication using regenerating codes. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part II*, volume 13091 of *LNCS*, pages 222–244. Springer, Cham, December 2021.

[ADP24] Nabil Alkeilani Alkadri, Nico Döttling, and Sihang Pu. Practical lattice-based distributed signatures for a small number of signers. In Christina Pöpper and Lejla Batina, editors, *ACNS 2024, Part I*, volume 14583 of *LNCS*, pages 376–402. Springer, Cham, March 2024.

[ALLW25] Martin R. Albrecht, Russell W. F. Lai, Oleksandra Lapiha, and Ivy K. Y. Woo. Partial lattice trapdoors: How to split lattice trapdoors, literally. Cryptology ePrint Archive, Report 2025/367, 2025.

[ASY22] Shweta Agrawal, Damien Stehlé, and Anshu Yadav. Round-optimal lattice-based threshold signatures, revisited. In Mikolaj Bojanczyk, Emanuela Merelli, and David P. Woodruff, editors, *ICALP 2022*, volume 229 of *LIPIcs*, pages 8:1–8:20. Schloss Dagstuhl, July 2022.

[BBC+19] Dan Boneh, Elette Boyle, Henry Corrigan-Gibbs, Niv Gilboa, and Yuval Ishai. Zero-knowledge proofs on secret-shared data via fully linear PCPs. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 67–97. Springer, Cham, August 2019.

[BBD+23] Manuel Barbosa, Gilles Barthe, Christian Doczkal, Jelle Don, Serge Fehr, Benjamin Grégoire, Yu-Hsuan Huang, Andreas Hülsing, Yi Lee, and Xiaodi Wu. Fixing and mechanizing the security proof of Fiat-Shamir with aborts and Dilithium. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 358–389. Springer, Cham, August 2023.

[BCdP+25] Giacomo Borin, Sofía Celi, Rafael del Pino, Thomas Espitau, Guilhem Niot, and Thomas Prest. Threshold signatures reloaded: ML-DSA and enhanced raccoon with identifiable aborts.

Scheme	ML-DSA	Security	Rounds (Online)	Comm. (Online)	$\lvert pk \rvert$	$\lvert sig \rvert$	t	n	N
This work (C)	✓	**UC-realize** $\mathcal{F}_{\text{T-MLDSA}}$	29	952	1.3	2.4	$< n/2$	$\leq \boldsymbol{N}$	**Any**
This work (R)	✓	**UC-realize** $\mathcal{F}_{\text{T-MLDSA}}$	16	1444	1.3	2.4	$< n/2$	$\leq \boldsymbol{N}$	**Any**
Borin et al. [BCdP+25]	✓	Unforgeable	2	≤ 525	1.3	2.4	$< n$	$\leq N$	≤ 6
Finally! [PN25]	✗	Unforgeable	2	22	2.5	2.7	$< n$	$\leq N$	≤ 8
Ringtail [BKL+25]	✗	Unforgeable	1	11	4.5	13.4	$< n$	≤ 1024	Any
EKT [EKT24]	✗	Unforgeable	1	14	5.5	11.1	$< n$	≤ 1024	Any
T-Raccoon [DKM+24]	✗	Unforgeable	3	40	3.8	12.4	$< n$	≤ 1024	Any

Table 7: Comparison with other lattice-based threshold signatures with 128-bit security. The column **ML-DSA** indicates whether the scheme produces signatures compatible with the standardized ML-DSA verification. Following Remark 1, N denotes the total number of parties participating in the DKG; n denotes the number of signing parties producing a signature; t denotes the corruption threshold, i.e., the security is guaranteed as long as up to t parties are malicious. Online communication is in KB per party, and public key and signature sizes are in KB. Per-party communication for [DKM+24, BKL+25, EKT24] is derived by setting $n = 1024$, and for [PN25] by setting $n = 8$. The rounds and communication for ours and [BCdP+25] are set to achieve a success probability > 0.5. We set $n = 7$ for our round-optimized protocol (R) to compute the communication. Per-party communication of our communication-optimized protocol (C) remains constant regardless of the number of parties n. We note that ours has an offline phase incurring varying communication costs depending on the number of parties (see Table 4).

Cryptology ePrint Archive, Paper 2025/1166, 2025.

[BCK+22] Mihir Bellare, Elizabeth C. Crites, Chelsea Komlo, Mary Maller, Stefano Tessaro, and Chenzhi Zhu. Better than advertised security for non-interactive threshold signatures. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 517–550. Springer, Cham, August 2022.

[BDF+11] Dan Boneh, Özgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. Random oracles in a quantum world. In Dong Hoon Lee and Xiaoyun Wang, editors, *ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 41–69. Springer, Berlin, Heidelberg, December 2011.

[BDK+21] Shi Bai, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Dilithium: Algorithm specifications and supporting documentation (version 3.1). https://pq-crystals.org/dilithium/, 2021.

[BDL+12] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, September 2012.

[BGW88] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In *20th ACM STOC*, pages 1–10. ACM Press, May 1988.

[BKL+25] Cecilia Boschini, Darya Kaviani, Russell Lai, Giulio Malavolta, Akira Takahashi, and Mehdi Tibouchi. Ringtail: Practical Two-Round Threshold Signatures from Learning with Errors . In *2025 IEEE Symposium on Security and Privacy*, pages 149–164. IEEE Computer Society, May 2025.

[BTT22] Cecilia Boschini, Akira Takahashi, and Mehdi Tibouchi. MuSig-L: Lattice-based multi-signature with single-round online phase. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 276–305. Springer, Cham, August 2022.

[BTZ22] Mihir Bellare, Stefano Tessaro, and Chenzhi Zhu. Stronger security for non-interactive threshold signatures: BLS and FROST. Cryptology ePrint Archive, Report 2022/833, 2022.

[Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001.

[CATZ24] Rutchathon Chairattana-Apirom, Stefano Tessaro, and Chenzhi Zhu. Partially non-interactive two-round lattice-based threshold signatures. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part IV*, volume 15487 of *LNCS*, pages 268–302. Springer, Singapore, December 2024.

[CEN25] Sofía Celi, Daniel Escudero, and Guilhem Niot. Share the MAYO: thresholdizing MAYO. In Ruben Niederhagen and Markku-Juhani O. Saarinen, editors, *PQCrypto 2025*, volume 15577 of *LNCS*, pages 165–198. Springer, 2025.

[CGH+18] Koji Chida, Daniel Genkin, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Yehuda Lindell, and Ariel Nof. Fast large-scale honest-majority MPC for malicious adversaries. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part III*, volume 10993 of *LNCS*, pages 34–64. Springer, Cham, August 2018.

[CGL+24] Jean-Sébastien Coron, François Gérard, Tancrède Lepoint, Matthias Trannoy, and Rina Zeitoun. Improved high-order masked generation of masking vector and rejection sampling in Dilithium. *IACR TCHES*, 2024(4):335–354, 2024.

[CGTZ23] Jean-Sébastien Coron, François Gérard, Matthias Trannoy, and Rina Zeitoun. Improved gadgets for the high-order masking of Dilithium. *IACR TCHES*, 2023(4):110–145, 2023.

[CH10] Octavian Catrina and Sebastiaan Hoogh. Improved primitives for secure multiparty integer computation. In *Lecture Notes in Computer Science*, volume LNCS 6280, pages 182–199, 09 2010.

[Che23] Yanbo Chen. DualMS: Efficient lattice-based two-round multi-signature with trapdoor-free simulation. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 716–747. Springer, Cham, August 2023.

[CKM23] Elizabeth C. Crites, Chelsea Komlo, and Mary Maller. Fully adaptive Schnorr threshold signatures. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 678–709. Springer, Cham, August 2023.

[Dal22] Anders Dalskov. SCL (Secure Computation Library)—utility library for prototyping MPC applications. https://github.com/anderspkd/secure-computation-library, 2022.

[DFPS23] Julien Devevey, Pouria Fallahpour, Alain Passelègue, and Damien Stehlé. A detailed analysis of Fiat-Shamir with aborts. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 327–357. Springer, Cham, August 2023.

[DKLs24] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. Threshold ECDSA in three rounds. In *2024 IEEE Symposium on Security and Privacy*, pages 3053–3071. IEEE Computer Society Press, May 2024.

[DKLS25] Antonín Dufka, Semjon Kravtšenko, Peeter Laud, and Nikita Snetkov. Trilithium: Efficient and universally composable distributed ML-DSA signing. Cryptology ePrint Archive, Paper 2025/675, 2025.

[DKM+24] Rafaël Del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Markku-Juhani O. Saarinen. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 219–248. Springer, Cham, May 2024.

[DN07] Ivan Damgård and Jesper Buus Nielsen. Scalable and unconditionally secure multiparty computation. In Alfred Menezes, editor, *CRYPTO 2007*, volume 4622 of *LNCS*, pages 572–590. Springer, Berlin, Heidelberg, August 2007.

[DOTT22] Ivan Damgård, Claudio Orlandi, Akira Takahashi, and Mehdi Tibouchi. Two-round n-out-of-n and multi-signatures and trapdoor commitment from lattices. *Journal of Cryptology*, 35(2):14, April 2022.

[dPENP25] Rafael del Pino, Thomas Espitau, Guilhem Niot, and Thomas Prest. Simple and efficient lattice threshold signatures with identifiable aborts. Cryptology ePrint Archive, Paper 2025/871, 2025.

[dPKN+25] Rafael del Pino, Shuichi Katsumata, Guilhem Niot, Michael Reichle, and Kaoru Takemure. Unmasking TRaccoon: A lattice-based threshold signature with an efficient identifiable abort protocol. Cryptology ePrint Archive, Paper 2025/849, 2025.

[dPKPR24] Rafaël del Pino, Shuichi Katsumata, Thomas Prest, and Mélissa Rossi. Raccoon: A masking-friendly signature proven in the probing model. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part I*, volume 14920 of *LNCS*, pages 409–444. Springer, Cham, August 2024.

[EGK+20] Daniel Escudero, Satrajit Ghosh, Marcel Keller, Rahul Rachuri, and Peter Scholl. Improved primitives for MPC over mixed arithmetic-binary circuits. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 823–852. Springer, Cham, August 2020.

[EKT24] Thomas Espitau, Shuichi Katsumata, and Kaoru Takemure. Two-round threshold signature from algebraic one-more learning with errors. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 387–424. Springer, Cham, August 2024.

[EL03] Milo D. Ercegovac and Toms Lang. *Digital Arithmetic*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1st edition, 2003.

[ENP24] Thomas Espitau, Guilhem Niot, and Thomas Prest. Flood and submerse: Distributed key generation and robust threshold signature from lattices. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 425–458. Springer, Cham, August 2024.

[GHHM21] Alex B. Grilo, Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz. Tight adaptive reprogramming in the QROM. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part I*, volume 13090 of *LNCS*, pages 637–667. Springer, Cham, December 2021.

[GKS24] Kamil Doruk Gür, Jonathan Katz, and Tjerand Silde. Two-round threshold lattice-based signatures from threshold homomorphic encryption. In Markku-Juhani Saarinen and Daniel Smith-Tone, editors, *Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part II*, pages 266–300. Springer, Cham, June 2024.

[GRR98] Rosario Gennaro, Michael O. Rabin, and Tal Rabin. Simplified VSS and fast-track multiparty computations with applications to threshold cryptography. In Brian A. Coan and Yehuda Afek, editors, *17th ACM PODC*, pages 101–111. ACM, June / July 1998.

[GSZ20] Vipul Goyal, Yifan Song, and Chenzhi Zhu. Guaranteed output delivery comes free in honest majority MPC. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 618–646. Springer, Cham, August 2020.

[KG20] Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In Orr Dunkelman, Michael J. Jacobson, Jr., and Colin O'Flynn, editors, *SAC 2020*, volume 12804 of *LNCS*, pages 34–65. Springer, Cham, October 2020.

[KLS18] Eike Kiltz, Vadim Lyubashevsky, and Christian Schaffner. A concrete treatment of Fiat-Shamir signatures in the quantum random-oracle model. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 552–586. Springer, Cham, April / May 2018.

[KRT24] Shuichi Katsumata, Michael Reichle, and Kaoru Takemure. Adaptively secure 5 round threshold signatures from MLWE/MSIS and DL with rewinding. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 459–491. Springer, Cham, August 2024.

[Lin24] Yehuda Lindell. Simple three-round multiparty Schnorr signing with full simulatability. *CiC*, 1(1):25, 2024.

[LS15] Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *DCC*, 75(3):565–599, 2015.

[Lyu12] Vadim Lyubashevsky. Lattice signatures without trapdoors. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 738–755. Springer, Berlin, Heidelberg, April 2012.

[MRVW21] Eleftheria Makri, Dragos Rotaru, Frederik Vercauteren, and Sameer Wagh. Rabbit: Efficient comparison for secure multi-party computation. In Nikita Borisov and Claudia Díaz, editors, *FC 2021, Part I*, volume 12674 of *LNCS*, pages 249–270. Springer, Berlin, Heidelberg, March 2021.

[NO07] Takashi Nishide and Kazuo Ohta. Multiparty computation for interval, equality, and comparison without bit-decomposition protocol. In Tatsuaki Okamoto and Xiaoyun Wang, editors, *PKC 2007*, volume 4450 of *LNCS*, pages 343–360. Springer, Berlin, Heidelberg, April 2007.

[PN25] Rafaël Del Pino and Guilhem Niot. Finally! A compact lattice-based threshold signature. In Tibor Jager and Jiaxin Pan, editors, *PKC 2025,*

Part III, volume 15676 of *LNCS*, pages 169–199. Springer, Cham, May 2025.

[Pub23] Federal Information Processing Standards Publication. FIPS 186-5: Digital Signature Standard (DSS). Federal Information Processing Standards Publication https://doi.org/10.6028/NIST.FIPS.186-5, 2023.

[Pub24] Federal Information Processing Standards Publication. FIPS 204: Module-Lattice-Based Digital Signature Standard. Federal Information Processing Standards Publication https://doi.org/10.6028/NIST.FIPS.204, 2024.

[RW19] Dragos Rotaru and Tim Wood. MArBled circuits: Mixing arithmetic and Boolean circuits with active security. In Feng Hao, Sushmita Ruj, and Sourav Sen Gupta, editors, *INDOCRYPT 2019*, volume 11898 of *LNCS*, pages 227–249. Springer, Cham, December 2019.

[Sho94] Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *35th FOCS*, pages 124–134. IEEE Computer Society Press, November 1994.

[TPCZ23] Guofeng Tang, Bo Pang, Long Chen, and Zhenfeng Zhang. Efficient lattice-based threshold signatures with functional interchangeability. *IEEE Trans. Inf. Forensics Secur.*, 18:4173–4187, 2023.

[WS58] Arnold Weinberger and JL Smith. A logic for high-speed addition. *Nat. Bur. Stand. Circ*, 591:3–12, 1958.

[Zha19] Mark Zhandry. How to record quantum queries, and applications to quantum indifferentiability. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 239–268. Springer, Cham, August 2019.

[ZT25] Chenzhi Zhu and Stefano Tessaro. The algebraic one-more MISIS problem and applications to threshold signatures. Cryptology ePrint Archive, Paper 2025/436, 2025.

[ZWSY25] Yuanyuan Zhou, Weijia Wang, Yiteng Sun, and Yu Yu. Rejected signatures' challenges pose new challenges: Key recovery of CRYSTALS-dilithium via side-channel attacks. Cryptology ePrint Archive, Report 2025/214, 2025.

List of Boxed Items

Functionalities

Protocols

Procedures

Algorithms

A Extended Preliminaries

A.1 Details of ML-DSA

In this section, we provide additional details on the ML-DSA scheme [BDK+21]. The list of complete parameters is provided in Table 2. In the description below, we replace some of the de-randomized operations in Gen and Sign in the spec by fully randomized ones. This modification does not impact interoperability with the standardized verification procedure.

Basic Operations. For a positive integer α and any $x \in \mathbb{Z}$, the centered modular reduction operation $x' := x \bmod^{\pm} \alpha$ maps x to a unique integer $-\lceil \alpha/2 \rceil < x' \leq \lfloor \alpha/2 \rfloor$ such that $x \equiv x' \bmod \alpha$. ML-DSA operates on a power-of-two cyclotomic ring $\mathcal{R} := \mathbb{Z}[x]/(x^N+1)$ and its quotient $\mathcal{R}_q := \mathcal{R}/q\mathcal{R}$. For a ring element $z = \sum_{i=0}^{N-1} z_i \cdot x^i \in \mathcal{R}_q$, the infinity norm of z is defined as $\|z\|_\infty := \max_i |z_i \bmod^{\pm} q|$. For a module element $\mathbf{z} = (z^{(1)}, \ldots, z^{(\ell)}) \in \mathcal{R}_q^\ell$, its infinity norm is defined analogously: $\|\mathbf{z}\|_\infty := \max_j \left\|z^{(j)}\right\|_\infty$. The Hamming weight $\mathsf{HW}(\mathbf{z})$ is defined as $||\mathbf{z}||_1 = \sum_j ||z^{(j)}||_1$.

We will write S_η to denote all elements $x \in \mathcal{R}_q$ such that $\|x\|_\infty \leq \eta$. We will write $\tilde{S}_\eta$ to denote the set $\{x \bmod^{\pm} 2\eta : x \in \mathcal{R}\}$. Note that $\tilde{S}_\eta$ does not contain any polynomials with $-\eta$ coefficients.

Hash Functions. ML-DSA uses SHAKE-256 H essentially in three ways and maps its output to appropriated codomains: (1) public key digest $tr = \mathsf{H}(\rho, \mathbf{t}_1) \in \{0,1\}^{256}$, (2) message digest $\mu = \mathsf{H}(tr, \mathsf{m}) \in \{0,1\}^{512}$, and (3) Fiat-Shamir challenge $c = \mathsf{H}(\mu, \mathbf{w}_1) \in C = \{c \in \mathcal{R} : ||c||_1 = \tau, ||c||_\infty = 1\}$. As the details of these mappings are not relevant to our protocol, we use the same notation H and refer the reader to [BDK+21, §5.3] for more details.

Security Levels & Common Parameters. The ML-DSA FIPS standard [Pub24, Table 1, page 15] lists the various parameters for the scheme. In Table 1, we give the parameters that are unchanged for all security levels of ML-DSA. Note that $q := 2^{23} - (2^{13}-1)$ just under 2^{23}, and elements in $\mathbb{Z}_q$ fit in 23 bits.

Supporting Functions. ML-DSA relies on the following supporting functions. We assume that if the function takes a module element $\mathbf{x} \in \mathcal{R}_q^k$ as input, it applies the operation coefficient-wise to each polynomial in $\mathbf{x}$. See Algorithm 9) for the full details:

$\mathbf{A} = \mathsf{ExpandA}(\rho)$: Expands a uniformly random 256-bit seed ρ into a larger matrix $\mathbf{A} \in \mathcal{R}_q^{k \times \ell}$.

$(r_1, r_0) = \mathsf{Power2Round}_q(r, d)$: Decomposes $r \in \mathbb{Z}_q$ into (r_1, r_0) such that $r = 2^d \cdot r_1 + r_0$

$(r_1, r_0) = \mathsf{DecomposeMod}(r, \alpha)$: [CGTZ23] Decomposes $r \in \mathbb{Z}_q$ into unique (r_1, r_0) such that $r = \alpha \cdot r_1 + r_0$, where $r_1 \in [0, (q-1)/\alpha]$ and $r_0 \in [-\alpha/2+1, \alpha/2]$ if $r_1 \neq 0$. It is an MPC-friendly and equivalent variant of Decompose [Pub24, Algorithm 36], and we use them interchangeably.

$r_1 = \mathsf{HighBits}_q(r, \alpha)$: Extracts the high bits r_1 of r by internally running $\mathsf{DecomposeMod}(r, \alpha)$.

$r_0 = \mathsf{LowBits}_q(r, \alpha)$: Extracts the low bits r_0 of r by internally running $\mathsf{DecomposeMod}(r, \alpha)$.

$h = \mathsf{MakeHint}_q(z, r, \alpha)$: Creates a hint bit h, indicating whether adding z to r changes the high bits of r.

$r_1 = \mathsf{UseHint}_q(h, r, \alpha)$: Uses the hint vector h to obtain the corrected high bits r_1 of r.

Computational Assumption. We recall the standard module LWE assumption, required by the security of ML-DSA. Note that the unforgeability of ML-DSA also relies on (SelfTarget)MSIS, which we omit here.

Definition 4 (Module Learning with Errors (MLWE))**.** *For $q \in \mathbb{N}$ and N a power of two, define $\mathcal{R}_q := \mathbb{Z}_q[x]/(x^N+1)$. For $k, \ell, \eta \in \mathbb{N}$, let $\mathcal{D}$ be some distribution over $\mathcal{R}_q^\ell$. The decision-MLWE problem $\mathsf{MLWE}_{q,N,k,\ell,\mathcal{D},\eta}$ is the task of distinguishing $(\mathbf{A}, \mathbf{A}\mathbf{y}+\mathbf{e})$ from $(\mathbf{A}, \mathbf{u})$ with non-negligible advantage, where $\mathbf{A}$ is uniform in $\mathcal{R}_q^{k \times \ell}$, $\mathbf{y} \leftarrow \mathcal{D}$ and $\mathbf{e}$ is sampled coefficient-wise uniformly from $[-\eta, \eta]$, and $\mathbf{u}$ is uniform in $\mathcal{R}_q^k$. If $\mathcal{D}$ is a uniform distribution over a set S, we write $\mathsf{MLWE}_{q,N,k,\ell,S,\eta}$.*

Secret Key & Public Key Distributions. The public matrix is $\mathbf{A} \in \mathcal{R}_q^{k \times \ell}$, which is obtained by expanding a random seed $\rho \in \{0,1\}^{256}$ via ExpandA function, internally calling SHAKE-128 (see §5.3 of [BDK+21] and Alg.32 of [Pub24]). The security level in ML-DSA is adjusted by growing the dimensions k and ℓ. The public key for ML-DSA consists of ρ and the top bits of $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e} = \mathbf{t}_1 \cdot 2^d + \mathbf{t}_0$ where the coefficients of both $\mathbf{s}$ and $\mathbf{e}$ are uniformly sampled from $[-\eta, \eta]$. The actual public key is $\mathbf{t}_1 := \mathsf{Power2Round}_q(\mathbf{t}, d)$.

ML-DSA Signing & Rejection Sampling. The coefficients of the mask $\mathbf{y} \in \mathcal{R}_q^\ell$ are uniform in $[-\gamma_1 + 1, \gamma_1]$. The rejection sampling loop begins by sampling a mask $\mathbf{y}$, then computes $\mathbf{w} = \mathbf{A}\mathbf{y}$. Let $\mathbf{w}_1 := \mathsf{HighBits}_q(\mathbf{w}, 2\gamma_2)$. Let us denote $\delta = \frac{q-1}{2\gamma_2}$. Then the $\mathsf{HighBits}_q$ operation (see Algorithm 9) internally decomposes $\mathbf{w}$ into unique $(\mathbf{w}_1, \mathbf{w}_0)$ such that $\mathbf{w} = 2\gamma_2 \cdot \mathbf{w}_1 + \mathbf{w}_0$, where coefficients of $\mathbf{w}_1$ are in canonical representation modulo δ (i.e., $\mathbf{w}_1 \in \{0, 1, \ldots, \delta-1\}^{kN}$) and coefficient of $\mathbf{w}_0$ are in centered representation modulo $2\gamma_2$ (i.e., $\mathbf{w}_0 \in [-\gamma_2 + 1, \gamma_2]^{kN}$ if $\mathbf{w}_1 \neq 0$ and $\mathbf{w}_0 \in [-\gamma_2, \gamma_2]^{kN}$ if $\mathbf{w}_1 = 0$), respectively. Note that this decomposition is carefully defined so that $2\gamma_2 \cdot \mathbf{w}_1 \neq q-1$ and thus the distance (modulo q) between any two distinct $2\gamma_2 \cdot \mathbf{w}_1$ and $2\gamma_2 \cdot \mathbf{w}_1'$ is at least $2\gamma_2$. In concrete instantiations of ML-DSA, $\mathbf{w}_1$ is very small since δ is either 44 or 16.

Next, $\mathbf{w}_1$ is hashed to $c \in C \subset \mathcal{R}_q$ together with $\mu \in \{0,1\}^{512}$, where C consists of polynomials with coefficients in $\{-1, 0, 1\}$ with hamming weight τ, and μ is a hash of the message m to be signed and $tr = \mathsf{H}(\rho, \mathbf{t}_1)$. Observe that τ sets c to be quite sparse, which is highly restrictive for the verification norm. Define $\mathbf{z} := \mathbf{y} + c \cdot \mathbf{s}$, and define

$\bar{\mathbf{w}}_0 := \mathsf{LowBits}_q(\mathbf{w}, 2\gamma_2) - c \cdot \mathbf{e}$. The rejection sampling consists of two checks: $||\mathbf{z}||_\infty < \gamma_1 - \beta$, and $||\bar{\mathbf{w}}_0||_\infty < \gamma_2 - \beta$. The latter check is for the error term, where γ_2 is nearly the modulus q.

Once both rejection sampling tests pass, a final hint $\mathbf{h}$ is computed, which is a series of bits marking when

$$\mathsf{HighBits}_q(\mathbf{w} - c \cdot \mathbf{e}, 2\gamma_2) \neq \mathsf{HighBits}_q(\mathbf{w} - c \cdot \mathbf{e} + c \cdot \mathbf{t}_0, 2\gamma_2).$$

If $||c \cdot \mathbf{t}_0||_\infty \geq \gamma_2$ or the hamming weight of $\mathbf{h}$ (denoted by $\mathsf{HW}(\mathbf{h})$) is beyond ω, rejection sampling restarts. The final signature is $\sigma = (c, \mathbf{z}, \mathbf{h})$.

ML-DSA Verification. The verifier takes the public key $\mathbf{t}_1$ and computes $\mathbf{w}' = \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d$ and $\mathbf{w}'_1 = \mathsf{HighBits}_q(\mathbf{w}', 2\gamma_2)$. For all locations where $\mathbf{h}$ is 1, the verification corrects $\mathbf{w}'_1$ by adding 1 if $\mathbf{w}'_0 > 0$ or subtracting 1 if $\mathbf{w}'_0 \leq 0$. Finally verification computes $c' = \mathsf{H}(\mu, \mathbf{w}'_1)$, where $\mu = \mathsf{H}(\mathsf{H}(\rho, \mathbf{t}_1), \mathsf{m})$ and accepts if $c = c'$ and $||\mathbf{z}||_\infty < \gamma_1 - \beta$.

Technical Lemma. We recall the technical lemma from the ML-DSA specification [BDK+21]. Lemma 5 is a direct consequence of Lemma 2 and 3 of [BDK+21].

Lemma 3 (Lemma 1 of [BDK+21])**.** *Supposed that q and α are positive integers satisfying $q > 2\alpha$, $q \equiv 1 \pmod{\alpha}$ and α even. Let $\mathbf{r}$ and $\mathbf{z}$ be vectors of elements in $\mathcal{R}_q$ where $\|\mathbf{z}\|_\infty \leq \alpha/2$, and let $\mathbf{h}, \mathbf{h}'$ be vectors of bits. Then the $\mathsf{HighBits}_q, \mathsf{MakeHint}_q$, and $\mathsf{UseHint}_q$ algorithms satisfy the following properties:*

1. $\mathsf{UseHint}_q(\mathsf{MakeHint}_q(\mathbf{z}, \mathbf{r}, \alpha), \mathbf{r}, \alpha) = \mathsf{HighBits}_q(\mathbf{r} + \mathbf{z}, \alpha)$
2. *Let $\mathbf{v}_1 = \mathsf{UseHint}_q(\mathbf{h}, \mathbf{r}, \alpha)$. Then $\|\mathbf{r} - \mathbf{v}_1 \cdot \alpha\|_\infty \leq \alpha + 1$. Further, if the number of 1's in $\mathbf{h}$ is ω, then all except at most ω coefficients of $\mathbf{r} - \mathbf{v}_1 \cdot \alpha$ will have magnitude of at most $\alpha/2$ after centered reduction modulo q.*
3. *For any $\mathbf{h}, \mathbf{h}'$, if $\mathsf{UseHint}_q(\mathbf{h}, \mathbf{r}, \alpha) = \mathsf{UseHint}_q(\mathbf{h}', \mathbf{r}, \alpha)$, then $\mathbf{h} = \mathbf{h}'$.*

Lemma 4 (Lemma 2 of [BDK+21])**.** *Let $\|\mathbf{s}\|_\infty \leq \beta$. If $\left\|\mathsf{LowBits}_q(\mathbf{r} - \mathbf{s}, \alpha)\right\|_\infty < \alpha/2 - \beta$, then*

$$\mathsf{HighBits}_q(\mathbf{r} - \mathbf{s}, \alpha) = \mathsf{HighBits}_q(\mathbf{r}, \alpha)$$

Lemma 5 (Lemma 3 of [BDK+21], adapted)**.** *Let $(\mathbf{r}_1, \mathbf{r}_0) = \mathsf{Decompose}_q(\mathbf{r}, \alpha)$, $(\mathbf{r}'_1, \mathbf{r}'_0) = \mathsf{Decompose}_q(\mathbf{r} - \mathbf{s}, \alpha)$, and $\|\mathbf{s}\|_\infty \leq \beta$. Then we have that*

$$\|\mathbf{r}_0 - \mathbf{s}\|_\infty < \alpha/2 - \beta \Leftrightarrow \left\|\mathbf{r}'_0\right\|_\infty < \alpha/2 - \beta.$$

A.2 UC Framework

In the UC framework, protocols are modeled as a system of *Interactive Turing Machines (ITM)*. While ITM itself is just a static piece of code, for each *session identifier* $\mathsf{sid} \in \mathbb{N}$, we consider a collection of *ITM instances (ITI)* sharing the same sid. Each ITI is an instance of some ITM for a specific session and together they form the runtime notion of a protocol session. Each ITI in a given protocol session is also called a *party*.

A protocol Π is executed by a set of parties $\mathcal{P}$, the *environment* $\mathcal{Z}$, and the *adversary* $\mathcal{A}$. The environment controls the flow of execution by giving instructions to the *adversary* $\mathcal{A}$ and choosing inputs to the parties involved in Π and receiving their outputs. The execution terminates when the environment finally terminates with an output 0 or 1. In this paper, we focus on static corruption meaning that $\mathcal{A}$ corrupts a subset of parties $\mathcal{C} \subseteq [n]$ at the beginning of an execution of Π. We denote by $\mathsf{EXEC}_{\Pi,\mathcal{A},\mathcal{Z}}(\lambda)$ the distribution of a binary output by $\mathcal{Z}$ after an execution of Π in the presence of $\mathcal{A}$, where $\lambda \in \mathbb{N}$ is a security parameter, and the randomness for all ITMs are assumed to be sampled uniformly at random.

Recall that two distributions $X(\lambda), Y(\lambda)$ indexed by $\lambda \in \mathbb{N}$ are called *indistinguishable* (denoted $X(\lambda) \approx Y(\lambda)$) if for all $c \in \mathbb{N}$, there exists a $\lambda_0 \in \mathbb{N}$ such that for all $\lambda > \lambda_0$, $|\Pr[X(\lambda) = 1] - \Pr[Y(\lambda) = 1]| < \lambda^{-c}$. Intuitively, we consider that a protocol Π in the presence of an adversary $\mathcal{A}$ successfully UC-emulates another (typically more idealized) protocol Φ if there exists another adversary (aka. *simulator*) $\mathcal{S}$ such that no environment $\mathcal{Z}$ can distinguish the execution of Φ with $\mathcal{S}$ from that of Π with $\mathcal{A}$.

To define the security of protocol Π in the UC framework, one describes an ideal functionality $\mathcal{F}$ which captures the desired functionality of the task in hand in the form of an ITM. One then defines Π UC-secure if Π UC-emulates the *ideal protocol* $\Phi = \mathsf{IDEAL}_\mathcal{F}$. The ideal protocol $\mathsf{IDEAL}_\mathcal{F}$ models an idealized run of protocol execution: the *simulator* $\mathcal{S}$ only interacts with $\mathcal{Z}$ and influences the execution through the prescribed interfaces of $\mathcal{F}$, and the parties $\mathcal{P}$ are replaced with the so-called *dummy parties* $\tilde{\mathcal{P}}$ which merely forward the inputs from $\mathcal{Z}$ to $\mathcal{F}$ and the responses back from $\mathcal{F}$ to $\mathcal{Z}$.

Definition 5 (UC security)**.** *Let $\mathcal{F}$ be an ideal functionality. A protocol Π is said to* UC-relize *$\mathcal{F}$, if for any PPT adversary $\mathcal{A}$, there exists a PPT simulator $\mathcal{S}$ such that for all PPT environment $\mathcal{Z}$,*

$$\mathsf{EXEC}_{\Pi,\mathcal{A},\mathcal{Z}}(\lambda) \approx \mathsf{EXEC}_{\mathsf{IDEAL}_\mathcal{F},\mathcal{S},\mathcal{Z}}(\lambda) .$$

A.3 Instantiation of Core Functionalities

Here we discuss different instantiations of the core functionalities from Section 2.3.

Algorithm 9: Supporting Algorithms for ML-DSA

$\mathsf{Decompose}_q(r,\alpha)$

$r := r \bmod^+ q$
$r_0 := r \bmod^\pm \alpha$
if $r - r_0 = q-1$ **then**
  $r_1 := 0; r_0 := r_0 - 1$
else
  $r_1 := (r-r_0)/\alpha$
return (r_1, r_0)

$\mathsf{Power2Round}_q(r,d)$

$r := r \bmod^+ q$
$r_0 := r \bmod^\pm 2^d$
$r_1 = (r-r_0)/2^d$
return (r_1, r_0)

$\mathsf{DecomposeMod}_q(r,\alpha)$ [CGTZ23]

$\delta := (q-1)/\alpha$
$s := (-\delta \cdot r + (q-1)/2) \bmod^+ q$
$r_1 := s \bmod^+ \delta$
$r_0 := r - r_1\alpha \bmod^\pm q$
return (r_1, r_0)

$\mathsf{HighBits}_q(r,\alpha)$

$(r_1, r_0) := \mathsf{Decompose}_q(r,\alpha)$
return r_1

$\mathsf{LowBits}_q(r,\alpha)$

$(r_1, r_0) := \mathsf{Decompose}_q(r,\alpha)$
return r_0

$\mathsf{MakeHint}_q(z,r,\alpha)$

$r_1 := \mathsf{HighBits}_q(r,\alpha)$
$v_1 := \mathsf{HighBits}_q(r+z,\alpha)$
return $r_1 \neq_? v_1$

$\mathsf{UseHint}_q(h,r,\alpha)$

$m := (q-1)/\alpha$
$(r_1, r_0) := \mathsf{Decompose}_q(r,\alpha)$
if $h = 1$ and $r_0 > 0$ **then**
  return $(r_1 + 1) \bmod^+ m$
if $h = 1$ and $r_0 \leq 0$ **then**
  return $(r_1 - 1) \bmod^+ m$
return r_1

A.3.1 Instantiating $\mathcal{F}_{\mathsf{Open}}$

Reconstructing a degree-t Shamir-shared value $[\![x]\!]$ can be done in two ways.

- (*All-to-all reconstruction*). All parties send their shares to each other, and each party performs error-detection locally. This costs $n \cdot (n-1)$ field elements in total distributed in 1 round.[10]

- (*King-based reconstruction* [DN07]). Suppose the parties will reconstruct $t+1$ secrets $[\![x_1]\!], \ldots, [\![x_{t+1}]\!]$. The parties locally compute $[\![f(i)]\!]$ for $i \in [n]$, where $f(z)$ is the polynomial $x_1 z^0 + \cdots + x_{t+1} \cdot z^t$. Then, the parties send their shares of $[\![f(i)]\!]$ to P_i, who performs error-detection to recover $f(i)$. Finally, each P_i sends $f(i)$ to all parties who once again perform error-detection to recover the polynomial $f(z)$. This costs $n(n-1) + n(n-1) = 2n(n-1)$ messages distributed in 2 rounds. Since this is for $t+1 = (n+1)/2$ reconstructions, the amortized cost per opening is $\approx 4n$, which is *linear*.

Optimization to reconstruct elements in $\mathbb{F}_{2^k}$. In our protocol we use sharings over $\mathbb{F}_{2^k}$, denoted by $[\![b]\!]_2$, where the secret b is in $\mathbb{F}_2$, but the shares are over $\mathbb{F}_{2^k}$. This means that, when reconstructing such secret, we get an overhead of $\times k$ due to the bigger shares. We can avoid this overhead when reconstructing k secrets simultaneously, as follows.

Recall that $\mathbb{F}_{2^k}$ is a k-dimensional $\mathbb{F}_2$-vector space, so there exists a basis $\beta_1, \ldots, \beta_k \in \mathbb{F}_{2^k}$ so that every element in $\mathbb{F}_{2^k}$ can be written uniquely as $b_1\beta_1 + \cdots + b_k\beta_k$. Now, consider k secrets $[\![b_1]\!]_2, \ldots, [\![b_k]\!]_2$, where $b_i \in \mathbb{F}_2$. Instead of reconstructing these k secrets, the parties can locally compute $[\![b]\!]_2 = \sum_{i=1}^k \beta_i \cdot [\![b_i]\!]_2$, and reconstruct this *single sharing* $[\![b]\!]_2$. Due to the uniqueness property from above, the reconstructed value decomposes as $b = \sum_{i=1}^k \beta_i \cdot b_i$, which allows the parties to recover $b_1, \ldots, b_k$, as intended.

[10] One can get $o(n^2)$ reconstruction for the case of $\mathbb{F}_{2^k}$ by using *regenerating codes*. See cf. [ACE+21].

A.3.2 Instantiating $\mathcal{F}_{\mathsf{Coin}}$

The "standard" approach of instantiating $\mathcal{F}_{\mathsf{Coin}}$ is by calling $[\![r]\!] \leftarrow \mathcal{F}_{\mathsf{Rand}}$, followed by $r \leftarrow \mathcal{F}_{\mathsf{Open}}([\![r]\!])$ (and parsing r in the required domain). We can make use of this instantiation, but its cost is proportional to the amount of coins needed (multiplied by n or n^2 depending on the instantiations of $\mathcal{F}_{\mathsf{Rand}}$ and $\mathcal{F}_{\mathsf{Open}}$). To support larger domains without such overhead, a common and widely used "optimization" is to sample a small seed, and use a PRF to expand it to a longer coin as $\mathtt{coin} \leftarrow \mathsf{PRF}(\mathtt{seed}, \mathtt{id})$. Unfortunately, this is *not* a valid instantiation of $\mathcal{F}_{\mathsf{Coin}}$ as it is not simulatable (to simulate an ideal coin in the real world the simulator would need to invert the PRF).

That said, for most uses of $\mathcal{F}_{\mathsf{Coin}}$ this optimization is actually works, but to prove security one must incorporate the PRF in the higher level protocol itself, and prove security of that protocol using the properties of the PRF itself. This is also the case in our protocol, where we use $\mathcal{F}_{\mathsf{Coin}}$ in two places: (1) as coefficients for the linear combinations in Π_{RejSamp}, and (2) to sample the public ρ from the public key. For (1), one can prove using standard hybrid arguments that the rejection sampling check still works when *replacing* the calls to $\mathcal{F}_{\mathsf{Coin}}$ by the PRF expansion approach from above. As for (2), ρ is very small and we can use the secure instantiation of $\mathcal{F}_{\mathsf{Coin}}$ from above.

A.3.3 Instantiating $\mathcal{F}_{\mathsf{Rand}}$

We consider two instantiations of $\mathcal{F}_{\mathsf{Rand}}[D]$, depending on the domain D. If $D = \mathbb{F}$ for $\mathbb{F} \in \{\mathbb{F}_q, \mathbb{F}_{2^k}\}$, we use the randomness extraction protocol from [DN07]. It makes use of a *super-invertible matrix* (*a.k.a.* an MDS matrix) $\boldsymbol{M} \in \mathbb{F}^{n-t \times n}$, where each $n-t \times n-t$ submatrx is invertible. Each party

P_i samples random $s_i \in \mathbb{F}$ and distributes shares $[\![s_i]\!]$ to the other parties. Then, the parties output as the random sharings $([\![r_1]\!], \ldots, [\![r_{n-t}]\!])^\intercal = \boldsymbol{M} \cdot ([\![s_1]\!], \ldots, [\![s_n]\!])^\intercal$, which can be shown to be random and unknown to the adversary. The communication is $n \cdot (n-1)$ elements, but since $n - t = (n+1)/2$ random values are produced, the amortized cost per random sharing is just $\approx 2n$.

Now, the other type of domain D is $\tilde{S}_\gamma = \{x \mod^\pm 2\gamma : x \in \mathcal{R}\}$ and $S_\eta = \{x \in \mathcal{R} : \|x\|_\infty \leq \eta\}$. We obtain these by using Edabits ($\mathcal{F}_{\mathsf{edabits}}$) of bounded size. Since $|\tilde{S}_\gamma|$ will always be a power-of-two, this approach on its own will always suffice. However, $|S_\eta|$ may not be a power-of-two, therefore, we need to sample edabits of size $\lceil \log |S_\eta| \rceil$ and use rejection sampling based on $\mathcal{F}_{\mathsf{BitLTC}}$ to ensure that the arithmetic part r of the edabit satisfies $r < |S_\eta|$. We present this protocol below in $\Pi_{\mathsf{Rand}}[S_\eta]$.

Protocol 10: $\Pi_{\mathsf{Rand}}[S_\eta]$

1: Let $m \leftarrow 2\eta + 1$ and $\ell \leftarrow \lceil \log m \rceil$; set $b \leftarrow 0$
2: **while** $b = 0$ **do**
3: Parties call $([\![r]\!]; [\![r_1]\!]_2, \ldots, [\![r_\ell]\!]_2) \leftarrow \mathcal{F}_{\mathsf{edabits}}[\ell]$.
4: Parties call $[\![d]\!]_2 \leftarrow \mathcal{F}_{\mathsf{BitLTC}}[m-1]([\![r_1]\!]_2, \ldots, [\![r_\ell]\!]_2)$
5: Parties call $d \leftarrow \mathcal{F}_{\mathsf{Open}}([\![d]\!]_2)$ and set $b \leftarrow 1 \oplus d$
6: **return** $[\![r]\!]$

The following lemma follows straighforwardly from the properties of $\mathcal{F}_{\mathsf{edabits}}, \mathcal{F}_{\mathsf{BitLTC}}, \mathcal{F}_{\mathsf{Open}}$.

Lemma 6. *$\Pi_{\mathsf{Rand}}[S_\eta]$ UC-realizes $\mathcal{F}_{\mathsf{Rand}}[S_\eta]$ in the $(\mathcal{F}_{\mathsf{edabits}}, \mathcal{F}_{\mathsf{BitLTC}}, \mathcal{F}_{\mathsf{Open}})$-hybrid model.*

A.3.4 Instantiating $\mathcal{F}_{\mathsf{Mult}}$

For instantiating secure multiplication we have a few options. First, the following approaches instantiate a weaker version of $\mathcal{F}_{\mathsf{Mult}}$ which does not compute $[\![xy]\!]$, but $[\![xy + \varepsilon]\!]$ for an error ε chosen by the adversary. For this section we denote by $\langle x \rangle$ additive shares of x, that is, each party P_i has x_i such that $x = x_1 + \cdots + x_n$.

- (*BGW* [BGW88, GRR98]). The parties locally compute $\langle xy \rangle \leftarrow [\![x]\!] \cdot [\![y]\!]$. Let $\langle xy \rangle = (z_1, \ldots, z_n)$. Each P_i distributes shares $[\![z_i]\!]$, and the parties locally compute $[\![xy]\!] = \sum_{i=1}^n \lambda_i \cdot [\![z_i]\!]$, where λ_i are the Lagrange coefficients. The communication of this approach is $n(n-1)$ elements in one round.

- (*Double-sharings* [DN07]). The parties preprocess a double-sharing $(\langle r \rangle, [\![r]\!])$, by adapting the ideas for $\mathcal{F}_{\mathsf{Rand}}$ (see Section A.3.3). This costs $4n$ field elements of communication. In the online phase, the parties compute locally $\langle d \rangle \leftarrow [\![x]\!] \cdot [\![y]\!] - \langle r \rangle$, send their shares of $\langle d \rangle$ to P_1 who reconstructs d and sends it back to the parties, who compute locally $[\![xy]\!] \leftarrow d + [\![r]\!]$. The communication for the online phase is $2(n-1)$ distributed in two rounds, but with the optimization from [GSZ20] it can be brought down to $\approx 1.5n$.

Assume the parties have performed one of the multiplication protocols above and ended up with $([\![x]\!], [\![y]\!], [\![xy + \varepsilon]\!])$. To instantiate the actual $\mathcal{F}_{\mathsf{Mult}}$ we compose this with a *verification step* that checks that $\varepsilon = 0$, which can be one of two types:

- (*"Sacrificing" with $\geq 2\times$ overhead*). The parties call $\mathcal{F}_{\mathsf{Rand}}$ to get $[\![a]\!], [\![b]\!]$, and use the "error-prone" multiplication from above to get $[\![ab + \delta]\!]$. Then they call $\sigma \leftarrow \mathcal{F}_{\mathsf{Coin}}$, compute locally $[\![d]\!] \leftarrow [\![x]\!] - [\![a]\!]$ and $[\![e]\!] \leftarrow \sigma[\![y]\!] - [\![b]\!]$, reconstruct $d \leftarrow \mathcal{F}_{\mathsf{Open}}([\![d]\!])$ and $e \leftarrow \mathcal{F}_{\mathsf{Open}}([\![e]\!])$, and compute locally

$$d[\![b]\!] + e[\![a]\!] + [\![ab + \delta]\!] + de - \sigma[\![xy + \varepsilon]\!],$$

  which equals $[\![\delta - \sigma\varepsilon]\!]$. The parties call $\mathcal{F}_{\mathsf{Open}}([\![\delta - \sigma\varepsilon]\!])$, and check that $\delta - \sigma\varepsilon = 0$. If this holds, then with probability $1 - 1/q$ it will be the case that $\varepsilon = 0$, as required. For our value of q, repeating this 3 times will ensure > 60 bits of security.[11]

- (*Checks with sublinear overhead* (a.k.a. *distributed ZK*)). One can use fully-linear PCPs [BBC+19] to verify N multiplications at a cost that is sublinear in N. Two variants can be considered: (1) $\approx \log N$ rounds with $O(\log N)$ communication, or (2) constant rounds with $O(\sqrt{N})$ complexity.

Using Multiplication Triples. The above can be used to multiply two sharings $[\![x]\!], [\![y]\!]$, but it requires to check the correctness of the multiplication (either with sacrificing or with distributed ZK), which adds overheads and complexities to the online phase. To make the online phase simpler and more efficient at the expense of slightly increasing the costs in the offline phase, one can alternatively first use the method above to produce *multiplication triples*: the parties call $\mathcal{F}_{\mathsf{Rand}}$ to obtain two random sharings $([\![a]\!], [\![b]\!])$, and use the method above to obtain $[\![a \cdot b]\!]$. Then, with these triples, the parties can multiply two sharings $[\![x]\!], [\![y]\!]$ as follows:

1. Compute locally $[\![d]\!] \leftarrow [\![x]\!] - [\![a]\!]$ and $[\![e]\!] \leftarrow [\![y]\!] - [\![b]\!]$,
2. Reconstruct $d \leftarrow \mathcal{F}_{\mathsf{Open}}([\![d]\!])$ and $e \leftarrow \mathcal{F}_{\mathsf{Open}}([\![e]\!])$,
3. Compute locally $[\![xy]\!] \leftarrow d[\![b]\!] + e[\![a]\!] + [\![ab]\!] + de$.

Since only openings are involved, this can be done with active security without extra checks. In terms of offline costs, the

[11] The approach from [CGH+18] is also worth mentioning: it "runs the circuit twice" to ensure an overall overhead of $2\times$, whereas the approach outlined here incurs in more costs.

two calls to $\mathcal{F}_{\mathsf{Rand}}$ cost $4n$ elements, and multiplying these to get the triple costs $5.5n$ elements, which gives $9.5n$ elements overall. In the online phase two calls to $\mathcal{F}_{\mathsf{Open}}$ are required, whose costs depend on the type of opening as outlined above.

A.3.5 Instantiating $\mathcal{F}_{\mathsf{edabits}}$

Edabits are obtained using the approach from [EGK+20]. We discuss the associated complexities in Section D.

A.3.6 Instantiating $\mathcal{F}_{\mathsf{dabits}}$

Dabits, which are just edabits of length one (*i.e.* a shared bit b as $([\![b]\!], [\![b]\!]_2)$), could be generated using the edabits approach [EGK+20], but as the authors in that work point out one can alternatively use the original *dabits* paper [RW19].

B Supplementary Material for Section 3

B.1 Proof of Theorem 2

We provide a detailed hybrid argument to prove strong computational HVZK of our modified ML-DSA identification.

Theorem 2 (concrete) *The modified ML-DSA identification implicit in Algorithm 1 is strongly computational HVZK under the* $\mathsf{MLWE}_{q,N,k,\ell,\tilde{S}^{\ell}_{\gamma_1},\eta}$ *and* $\mathsf{MLWE}_{q,N,k,\ell,S^{\ell}_{\gamma_1-\beta-1},\eta}$ *assumptions. Concretely, there exists a polynomial time simulator $\mathcal{S}$, such that for any $T_{\mathcal{A}}$-time scHVZK distinguisher $\mathcal{A}$ with advantage $\varepsilon_{\mathcal{A}}$, there exists $(\varepsilon_{\mathcal{B}_1}, T_{\mathcal{B}_1})$-adversary $\mathcal{B}_1$ against* $\mathsf{MLWE}_{q,N,k,\ell,\tilde{S}^{\ell}_{\gamma_1},\eta}$, *$(\varepsilon_{\mathcal{B}_2}, T_{\mathcal{B}_2})$-adversary $\mathcal{B}_2$ against* $\mathsf{MLWE}_{q,N,k,\ell,S^{\ell}_{\gamma_1-\beta-1},\eta}$, *and $(\varepsilon_{\mathcal{B}_3}, T_{\mathcal{B}_3})$-adversary $\mathcal{B}_3$ against* $\mathsf{MLWE}_{q,N,k,\ell,S^{\ell}_{\gamma_1-\beta-1},\eta}$, *respectively, running in time $T_{\mathcal{A}} \approx T_{\mathcal{B}_1} \approx T_{\mathcal{B}_2} \approx T_{\mathcal{B}_3}$ such that,*

$$\varepsilon_{\mathcal{A}} \leq \varepsilon_{\mathcal{B}_1} + p_{\mathbf{z}} \cdot (\varepsilon_{\mathcal{B}_2} + \varepsilon_{\mathcal{B}_3}) + 2^{-2\lambda}$$

where $p_{\mathbf{z}}$ is the probability that $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$ in a real execution.

Proof. We prove that the simulator presented in Algorithm 11 computationally simulates the distribution of real prover's transcript. We denote by $\mathsf{H}_i(1^\lambda)$ a hybrid experiment where an scHVZK adversary $\mathcal{A}$ receives a transcript from the algorithm Trans_i. We summarize hybrid versions of Trans_i in Algorithm 12 and 13. The experiment where the adversary receives a real transcript is equivalent to H_0. Since the check in Line 6 of P_2 (Algorithm 1) is purely for ensuring correctness, we drop this check in the proof.

Hybrid H_1: In this hybrid, the second check on a transcript is performed with respect to $\mathbf{w}'_0 = \mathsf{LowBits}_q(\mathbf{w} - c\mathbf{e}, 2\gamma_2)$ instead of on $\mathbf{w}_0 - c\mathbf{e}$. By Lemma 5, these checks are equivalent and thus the distribution of Trans_1 is identical to that of Trans_0.

Algorithm 11: scHVZK simulator of modified ML-DSA

$\mathsf{Sim}(\mathsf{pk} = (\rho, \mathbf{t}_1, \mathbf{t}_0))$

1: $\mathbf{A} := \mathsf{ExpandA}(\rho)$
2: $c \leftarrow C$
3: $\rho' \leftarrow [0, 1)$
4: **if** $\rho' \leq p_{\mathbf{z}}$ **then** // Mode 1: Passing $\mathbf{z}$
5: $\quad \mathbf{w}' \leftarrow \mathcal{R}_q^k$
6: $\quad (\mathbf{w}'_1, \mathbf{w}'_0) = \mathsf{Decompose}_q(\mathbf{w}', 2\gamma_2)$
7: $\quad$ **if** $\|\mathbf{w}'_0\|_\infty \geq \gamma_2 - \beta$ **then** // Mode 1-A
8: $\quad\quad$ **for** $i \in [kN]$ **do**
9: $\quad\quad\quad$ **if** $|w'_{0,i}| < \gamma_2 - \beta$ **then**
10: $\quad\quad\quad\quad w_{1,i} \leftarrow \mathcal{U}$ // Uniform in $\mathbb{Z}_\delta$
11: $\quad\quad\quad$ **else**
12: $\quad\quad\quad\quad w_{1,i} \leftarrow \tilde{\mathcal{U}}$ // See (2)
13: $\quad\quad \mathbf{w}_1 := (w_{1,1}, \ldots, w_{1,kN})$
14: $\quad\quad$ **return** $(\mathbf{w}_1, c, \perp)$
15: $\quad$ **else** // Mode 1-B
16: $\quad\quad \mathbf{w}'_0 := (\gamma_2 - \beta, \ldots, \gamma_2 - \beta)$
17: $\quad\quad i := 0$
18: $\quad\quad$ **while** $\|\mathbf{w}'_0\|_\infty \geq \gamma_2 - \beta$ **or** $i < 4\lambda$ **do**
19: $\quad\quad\quad i = i + 1$
20: $\quad\quad\quad \mathbf{z} \leftarrow [-\gamma_1 + \beta + 1, \gamma_1 - \beta - 1]^{N\ell}$
21: $\quad\quad\quad \mathbf{e}_w \leftarrow S_\eta^k$
22: $\quad\quad\quad \mathbf{w}' = \mathbf{A}\mathbf{z} - c\mathbf{t} + \mathbf{e}_w$
23: $\quad\quad\quad (\mathbf{w}'_1, \mathbf{w}'_0) = \mathsf{Decompose}_q(\mathbf{w}', 2\gamma_2)$
24: $\quad\quad \mathbf{h} := \mathsf{MakeHint}_q(-c\mathbf{t}_0, \mathbf{A}\mathbf{z} - c\mathbf{t} + c\mathbf{t}_0, 2\gamma_2)$
25: $\quad\quad$ **return** $(\mathbf{w}'_1, c, (\mathbf{z}, \mathbf{h}))$
26: **else** // Mode 2
27: $\quad \mathbf{w} \leftarrow \mathcal{R}_q^k$
28: $\quad (\mathbf{w}_1, \mathbf{w}_0) = \mathsf{Decompose}_q(\mathbf{w}, 2\gamma_2)$
29: $\quad$ **return** $(\mathbf{w}_1, c, \perp)$

Moreover, if $\|\mathbf{w}'_0\|_\infty < \gamma_2 - \beta$, it outputs $\mathbf{w}'_1$ instead of $\mathbf{w}_1$. By Lemma 4, we have that $\mathbf{w}'_1 = \mathbf{w}_1$.

Hybrid H_2: In this hybrid, we rearrange the conditional operations so that if $\mathbf{z}$ passes the check (corresponding to Mode 1 mentioned in Section 3), Trans_2 performs the second check on the low bits of $\mathbf{w}' = \mathbf{A}\mathbf{z} - c\mathbf{t} + \mathbf{e}_w = \mathbf{w} - c\mathbf{e}$. If the second check fails (Mode 1-A), then it outputs the higher order component of $\mathbf{w} = \mathbf{w}' + c\mathbf{e} = \mathbf{A}\mathbf{y} + \mathbf{e}_w$; else (Mode 1-B), then it release an entire transcript together with the hint $\mathbf{h}$. If $\mathbf{z}$ fails the check, it releases $(\mathbf{w}_1, c, \perp)$ as an output, where $\mathbf{w}_1 = \mathsf{DecomposeMod}_q(\mathbf{A}\mathbf{y} + \mathbf{e}_w, 2\gamma_2)$. This is merely a syntactic change, so H_2 is equivalent to H_1.

Hybrid H_3: In this hybrid, we change how passing $\mathbf{z}$ is sampled and how rejected $\mathbf{y}$ is sampled as follows. Trans_3 flips random coins $\rho' \in [0, 1)$, and if $\rho' \leq p_{\mathbf{z}} = \left(\frac{2(\gamma_1 - \beta) - 1}{2\gamma_1 - 1}\right)^{N \cdot \ell}$, $\mathbf{z}$ is sampled from $[-\gamma_1 + \beta + 1, \gamma_1 - \beta - 1]^{k\ell}$; else, $\mathbf{y}$ is sampled uniformly from $\tilde{S}^{\ell}_{\gamma_1}$ conditioned on $\|c\mathbf{s} + \mathbf{y}\|_\infty \geq \gamma_1 - \beta$. In the previous hybrid, the probability that all the coefficients of $\mathbf{z}$ are smaller than $\gamma_1 - \beta$ is $p_{\mathbf{z}}$. Since $\|c\mathbf{s}\|_\infty \leq \beta$, conditioned on $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$, every coefficient of $\mathbf{z}$ is uniformly

distributed in $[-\gamma_1+\beta+1, \gamma_1-\beta-1]$ in the previous hybrid. Thus, the distribution of the transcript remains unchanged for both passing and rejected cases.

Hybrid H_4: In this hybrid, if $\rho' > p_{\mathbf{z}}$ (i.e., rejected $\mathbf{z}$), $\mathbf{w}$ is sampled uniformly from $\mathcal{R}_q^k$. Conditioned on $\rho' \le p_{\mathbf{z}}$, H_4 is identical to the previous hybrid. Conditioned on $\rho' > p_{\mathbf{z}}$, if there exists an adversary $\mathcal{A}$ distinguishing H_4 from H_3, one can construct a reduction $\mathcal{A}'$ solving $\mathsf{MLWE}_{q,N,k,\ell,Q^\perp,\eta}$ with non-negligible advantage, where $Q^\perp$ denotes the uniform distribution over a set $\{\mathbf{y}' \in \tilde{S}^\ell_{\gamma_1} : \|c\mathbf{s}+\mathbf{y}'\|_\infty \ge \gamma_1-\beta\}$. Finally, $\mathsf{MLWE}_{q,N,k,\ell,Q^\perp,\eta}$ reduces to $\mathsf{MLWE}_{q,N,k,\ell,\tilde{S}^\ell_{\gamma_1},\eta}$ and $\mathsf{MLWE}_{q,N,k,\ell,S^\ell_{\gamma_1-\beta-1},\eta}$, thanks to the tight reduction presented by by del Pino and Niot [PN25]. In Lemma 7 below, we adapt their result by tailoring it to uniform secrets and errors. Note that in the reduction below, the distribution of $\mathbf{e}_w$ remains unchanged since no norm check is performed on $\mathbf{e}_w$. Overall, we obtain the following advantage loss due to this jump:

$$\begin{aligned}&\left|\Pr[\mathsf{H}_3(1^\lambda)=1]-\Pr[\mathsf{H}_4(1^\lambda)=1]\right|\\ &\le (1-p_{\mathbf{z}})\cdot\left|\Pr[\mathsf{H}_3(1^\lambda)=1\,|\,\rho'>p_{\mathbf{z}}]-\Pr[\mathsf{H}_4(1^\lambda)=1\,|\,\rho'>p_{\mathbf{z}}]\right|\\ &\le (1-p_{\mathbf{z}})\cdot\varepsilon_{\mathcal{A}'} \le \varepsilon_{\mathcal{B}_1}+p_{\mathbf{z}}\cdot\varepsilon_{\mathcal{B}_2}.\end{aligned}$$

Lemma 7 (Lemma 5 of [PN25], adapted)**.** *Given a secret $\mathbf{s}$, challenge c, note $Q^\perp$ the uniform distribution over $\{\mathbf{y}' \in \tilde{S}^\ell_{\gamma_1} : \|c\mathbf{s}+\mathbf{y}'\|_\infty \ge \gamma_1-\beta\}$. For any $(\varepsilon_{\mathcal{A}'}, T_{\mathcal{A}'})$-adversary $\mathcal{A}'$ against the $\mathsf{MLWE}_{q,N,k,\ell,Q^\perp,\eta}$ problem, there exist $(\varepsilon_{\mathcal{B}_1}, T_{\mathcal{B}_1})$-adversary $\mathcal{B}_1$ and $(\varepsilon_{\mathcal{B}_2}, T_{\mathcal{B}_2})$-adversary $\mathcal{B}_2$ respectively against the $\mathsf{MLWE}_{q,N,k,\ell,\tilde{S}^\ell_{\gamma_1},\eta}$ and $\mathsf{MLWE}_{q,N,k,\ell,S^\ell_{\gamma_1-\beta-1},\eta}$ problems, running in time $T_{\mathcal{A}'} \approx T_{\mathcal{B}_1} \approx T_{\mathcal{B}_2}$ such that*

$$(1-p_{\mathbf{z}})\cdot\varepsilon_{\mathcal{A}'} \le \varepsilon_{\mathcal{B}_1}+p_{\mathbf{z}}\cdot\varepsilon_{\mathcal{B}_2}$$

where $p_{\mathbf{z}} = \Pr\left[\|c\mathbf{s}+\mathbf{y}\|_\infty < \gamma_1-\beta : \mathbf{y} \leftarrow \tilde{S}^\ell_{\gamma_1}\right]$.

Hybrid H_5: In this hybrid, if both checks on $\mathbf{z}$ and $\mathbf{w}'_0$ have passed, Trans_5 samples a passing (simulated) transcript until $\left\|\mathbf{w}'_0\right\|_\infty \ge \gamma_2-\beta$, where $\mathbf{w} = \mathbf{Az}-c\mathbf{t}+\mathbf{e}_w$. Clearly, the output distribution remains unchanged. To evaluate the expected number of iterations until Trans_5 finds a passing transcript, consider the probability $p_{\bar{\mathbf{w}}_0} = \Pr[\left\|\mathsf{LowBits}_q(\mathbf{Az}-c\mathbf{t}+\mathbf{e}_w, 2\gamma_2)\right\|_\infty < \gamma_2-\beta]$ where $\mathbf{z} \leftarrow S^\ell_{\gamma_1-\beta-1}, \mathbf{e}_w \leftarrow S^k_\eta$. Following Lemma 4.4 of [KLS18], $p_{\bar{\mathbf{w}}_0}$ can be approximated by

$$p_{\bar{\mathbf{w}}_0} \approx \frac{|S^k_{\gamma_2-\beta-1}|}{|S^k_{\gamma_2-1}|} \approx e^{-N\cdot\beta k/\gamma_2}$$

In our parameters, $1/p_{\bar{\mathbf{w}}_0}$ is set to a small constant between 2 and 3. Thus, Trans_5 halts in an expected polynomial time. Alternatively, one can achieve a strict polynomial time simulation by bounding the number of iterations to 4λ. Then H_5 and H_4 are statistically indistinguishable with an advantage loss $(1-p_{\bar{\mathbf{w}}_0})^{4\lambda} < 2^{-2\lambda}$ for all ML-DSA parameters. Thus, we get

$$\left|\Pr[\mathsf{H}_4(1^\lambda)=1]-\Pr[\mathsf{H}_5(1^\lambda)=1]\right| \le 2^{-2\lambda}.$$

Hybrid H_6: In this hybrid, if $\rho' \le p_{\mathbf{z}}$ (i.e., passing $\mathbf{z}$), $\mathbf{w}'$ is sampled uniformly from $\mathcal{R}_q^k$, instead of $\mathbf{w}' = \mathbf{Az}-c\mathbf{t}+\mathbf{e}_w$. If there exists an adversary $\mathcal{A}$ distinguishing H_6 from H_5, one can construct a reduction $\mathcal{B}_3$ solving $\mathsf{MLWE}_{q,N,k,\ell,S^\ell_{\gamma_1-\beta-1},\eta}$ with non-negligible advantage. Overall, we obtain the following advantage loss due to this jump:

$$\begin{aligned}&\left|\Pr[\mathsf{H}_5(1^\lambda)=1]-\Pr[\mathsf{H}_6(1^\lambda)=1]\right|\\ &\le p_{\mathbf{z}}\cdot\left|\Pr[\mathsf{H}_5(1^\lambda)=1\,|\,\rho'\le p_{\mathbf{z}}]-\Pr[\mathsf{H}_6(1^\lambda)=1\,|\,\rho'\le p_{\mathbf{z}}]\right|\\ &\le p_{\mathbf{z}}\cdot\varepsilon_{\mathcal{B}_3}.\end{aligned}$$

Hybrid H_7: In this hybrid, if the norm of $\mathbf{w}'_0$ exceeds the bound, Trans_7 returns $\mathbf{w}_1 = (w_{1,1},\ldots,w_{1,kN})$ constructed as follows: for $i = 1,\ldots,kN$, if $|w'_{0,i}| < \gamma_2-\beta$, sample $w_{1,i}$ uniformly from $\mathbb{Z}_\delta$, where $\delta = (q-1)/2\gamma_2$; else, sample $w_{1,i}$ from the distribution $\tilde{\mathcal{U}}$ defined in (2) of Lemma 8 below. To prove the former case perfectly simulates H_6, observe $w_{1,i} = w'_{1,i}$ by Lemma 4 since $\|c\mathbf{e}\|_\infty \le \beta$. Conditioned on $|w'_{0,i}| < \gamma_2-\beta$, $w'_{1,i}$ is uniformly distributed in $\mathbb{Z}_\delta$. By Lemma 8 presented below, the latter case perfectly simulates H_6. As the transcript algorithm in this hybrid is equivalent to the simulator in Algorithm 11, we conclude the proof. □

Lemma 8. *Let q, α, β be positive integers such that α is even, α divides $q-1$, and $\beta < \alpha/2$. Let $\tilde{\mathcal{U}}$ be the following distribution with support $W_1 = \{0,1,\ldots,\delta-1\}$:*

$$\tilde{\mathcal{U}}(a_1) = \begin{cases} \frac{2\beta+1}{2(\beta+1)\delta+1} & \text{if } a_1 \in W_1 \setminus \{0\} \\ \frac{2\beta+2}{2(\beta+1)\delta+1} & \text{if } a_1 = 0 \end{cases} \tag{2}$$

where $\delta = (q-1)/\alpha$. Fix an arbitrary integer s such that $|s| \le \beta < \alpha/2$. $\tilde{\mathcal{U}}$ is perfectly indistinguishable with the following distribution $\tilde{\mathcal{U}}'$: Sample uniform $a' \in \mathbb{Z}_q$ conditioned on $|a'_0| \ge \alpha/2-\beta$, where $(a'_1, a'_0) := \mathsf{DecomposeMod}_q(a', \alpha)$; $(a_1, a_0) := \mathsf{DecomposeMod}_q(a'+s, \alpha)$; output a_1.

Before giving the full proof, we briefly discuss the intuition for this lemma. In order to show that hybrid H_6 is indistinguishable from hybrid H_7, we must show that H_7 can sample the high bits of some value a' that has been shifted by some *secret* value s. The distribution of a' corresponds to the distribution of the coefficients of $\mathbf{w}'$ in H_6 and H_7, which is uniform except for the conditioning on the low bits. If there was no shift, we could simply output the high bits of a uniform value. However, the output of H_6 is not the high bits of a' but rather

the high bits of $a'+s$. Here, the value s corresponds to a coefficient of $c\mathbf{e}$, and the goal of Lemma 8 is to show that H_7 can simulate the high bits of $a'+s$ with a single distribution $\tilde{\mathcal{U}}$ regardless of the value of s.

Proof of Lemma 8. We first analyze the distribution of

$$\begin{aligned}(a'_1, a'_0) &= \mathsf{DecomposeMod}_q(a', \alpha) \\ &= \mathsf{Decompose}_q(a', \alpha)\end{aligned}$$

conditioned on $|a'_0| \geq \alpha/2 - \beta$ for a uniformly sampled $a' \in \mathbb{Z}_q$.

Since $|a'_0| \geq \alpha/2-\beta$, the support of $a' = a'_1\alpha + a'_0$ has size $2(\beta+1)\delta+1$. Thus, we have

$$\Pr[a'_1 = j \,|\, |a'_0| \geq \alpha/2-\beta] = \begin{cases} \frac{2\beta+1}{2(\beta+1)\delta+1} \text{ if } j \neq 0 \\ \frac{2\beta+2}{2(\beta+1)\delta+1} \text{ if } j = 0 \end{cases}$$

where the probability that $a'_1 = 0$ is slightly larger due to the edge case of $\mathsf{Decompose}_q$. Let $W_0 = \{-\alpha/2+1, \ldots, \alpha/2\}$.

We show that for each of the following cases, the distribution of a_1 is identical to a'_1.

Case 1: $s \geq 0$ We consider the following subcases:

- If $-\alpha/2 < a'_0 \leq -\alpha/2+\beta$ or $\alpha/2-\beta \leq a'_0 \leq \alpha/2-s$: In this case, $a_0 := a'_0 + s \in W_0$, and thus $a = a'+s = a'_1\alpha + (a'_0+s) = a_1\alpha + a_0$, where $a_1 = a'_1$.
- If $\alpha/2 - s < a'_0 \leq \alpha/2$: In this case, $a'_0 + s = a_0 + \alpha$ for some $-\alpha/2 < a_0 \leq -\alpha/2+\beta$, and thus $a = a' + s = (a'_1+1)\alpha + a_0$. If $0 \leq a'_1 < \delta - 1$, then a can be decomposed into $a_1 = a'_1 + 1$ and a_0. If $a'_1 = \delta - 1$, then a can be decomposed into $a_1 = 0$ and $a_0 - 1$ due to the edge case of $\mathsf{Decompose}_q$.
- $-\alpha/2 = a'_0 \wedge a'_1 = 0$: In this case, $a_0 := a'_0 + s \in W_0$, and thus $a = a+s = a'_1\alpha + (a'_0+s) = a_1\alpha + a_0$, where $a_1 = a'_1 = 0$.

Therefore, when adding any $0 \leq s \leq \beta$, a'_1 either remains the same or maps to $a_1 = a'_1 + 1 \pmod{\delta}$. Moreover, if $-\alpha/2 = a'_0 \wedge a'_1 = 0$, we have $a_1 = a'_1 = 0$, which preserves the probability that $a_1 = 0$.

Case 2: $s < 0$ We consider the following subcases:

- If $\alpha/2-\beta \leq a'_0 \leq \alpha/2$ or $-\alpha/2-s < a'_0 \leq -\alpha/2+\beta$: In this case, $a_0 := a'_0 + s \in W_0$, and thus $a = a+s = a'_1\alpha + (a'_0+s) = a_1\alpha + a_0$, where $a_1 = a'_1$.
- If $-\alpha/2 \leq a'_0 < -\alpha/2 - s$: In this case, $a'_0 + s = a_0 - \alpha$ for some $\alpha/2-\beta \leq a_0 < \alpha/2$, and thus $a = a+s = (a'_1-1)\alpha + a_0$. If $0 < a'_1 \leq \delta - 1$, then a can be decomposed into $a_1 = a'_1 - 1$ and a_0. If $a'_1 = 0$, then a can be decomposed into $a_1 = \delta - 1$ and $a_0 + 1$ due to the edge case of $\mathsf{Decompose}_q$.
- $-\alpha/2 - s = a'_0 \wedge a'_1 \neq 0$: In this case, $a'_0 + s = a_0 - \alpha$ for $a_0 = \alpha/2$, and thus $a = a+s = (a'_1-1)\alpha + a_0$, where $a_1 = a'_1 - 1$.
- $-\alpha/2 - s = a'_0 \wedge a'_1 = 0$: In this case, $a = a+s = -\alpha/2$, and thus $a_1 = a'_1 = 0$.

Therefore, when adding any $-\beta \leq s < 0$, a'_1 either remains the same or maps to $a_1 = a'_1 - 1 \pmod{\delta}$. Moreover, if $-\alpha/2 - s = a'_0 \wedge a'_1 = 0$, we have $a_1 = a'_1 = 0$, which preserves the probability that $a_1 = 0$. □

Algorithm 12: Hybrids for Theorem 2

$\underline{\mathsf{Trans}_0(\mathsf{sk} = (\rho, \mathbf{s}, \mathbf{e}, \mathbf{t}_1, \mathbf{t}_0))}$

$\mathbf{A} := \mathsf{ExpandA}(\rho)$
$(\mathbf{y}, \mathbf{e}_w) \leftarrow \tilde{S}^{\ell}_{\gamma_1} \times S^k_{\eta}$
$\mathbf{w} = \mathbf{A}\mathbf{y} + \mathbf{e}_w$
$(\mathbf{w}_1, \mathbf{w}_0) = \mathsf{DecomposeMod}_q(\mathbf{w}, 2\gamma_2)$
$c \leftarrow C$
$\mathbf{z} = c\mathbf{s} + \mathbf{y}$
$\bar{\mathbf{w}}_0 = \mathbf{w}_0 - c\mathbf{e}$
if $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$ **then return** $(\mathbf{w}_1, c, \perp)$
if $\|\bar{\mathbf{w}}_0\|_\infty \geq \gamma_2 - \beta$ **then return** $(\mathbf{w}_1, c, \perp)$
$\mathbf{h} := \mathsf{MakeHint}_q(-c\mathbf{t}_0, \mathbf{A}\mathbf{z} - c\mathbf{t} + c\mathbf{t}_0, 2\gamma_2)$
return $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}))$.

$\underline{\mathsf{Trans}_1(\mathsf{sk} = (\rho, \mathbf{s}, \mathbf{e}, \mathbf{t}_1, \mathbf{t}_0))}$

$\mathbf{A} := \mathsf{ExpandA}(\rho)$
$(\mathbf{y}, \mathbf{e}_w) \leftarrow \tilde{S}^{\ell}_{\gamma_1} \times S^k_{\eta}$
$\mathbf{w} = \mathbf{A}\mathbf{y} + \mathbf{e}_w$
$(\mathbf{w}_1, \mathbf{w}_0) = \mathsf{DecomposeMod}_q(\mathbf{w}, 2\gamma_2)$
$c \leftarrow C$
$\mathbf{z} = c\mathbf{s} + \mathbf{y}$
$(\mathbf{w}'_1, \mathbf{w}'_0) = \mathsf{DecomposeMod}_q(\mathbf{w} - c\mathbf{e}, 2\gamma_2)$
if $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$ **then return** $(\mathbf{w}_1, c, \perp)$
if $\left\|\mathbf{w}'_0\right\|_\infty \geq \gamma_2 - \beta$ **then return** $(\mathbf{w}_1, c, \perp)$
$\mathbf{h} := \mathsf{MakeHint}_q(-c\mathbf{t}_0, \mathbf{A}\mathbf{z} - c\mathbf{t} + c\mathbf{t}_0, 2\gamma_2)$
return $(\mathbf{w}'_1, c, (\mathbf{z}, \mathbf{h}))$.

$\underline{\mathsf{Trans}_2(\mathsf{sk} = (\rho, \mathbf{s}, \mathbf{e}, \mathbf{t}_1, \mathbf{t}_0))}$

$\mathbf{A} := \mathsf{ExpandA}(\rho)$
$(\mathbf{y}, \mathbf{e}_w) \leftarrow \tilde{S}^{\ell}_{\gamma_1} \times S^k_{\eta}$
$c \leftarrow C$
$\mathbf{z} = c\mathbf{s} + \mathbf{y}$
if $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$ **then**
  $\mathbf{w}' = \mathbf{A}\mathbf{z} - c\mathbf{t} + \mathbf{e}_w$
  $(\mathbf{w}'_1, \mathbf{w}'_0) = \mathsf{DecomposeMod}_q(\mathbf{w}', 2\gamma_2)$
  if $\left\|\mathbf{w}'_0\right\|_\infty \geq \gamma_2 - \beta$ **then**
    $\mathbf{w} = \mathbf{w}' + c\mathbf{e}$
    $(\mathbf{w}_1, \mathbf{w}_0) = \mathsf{DecomposeMod}_q(\mathbf{w}, 2\gamma_2)$
    return $(\mathbf{w}_1, c, \perp)$
  $\mathbf{h} := \mathsf{MakeHint}_q(-c\mathbf{t}_0, \mathbf{A}\mathbf{z} - c\mathbf{t} + c\mathbf{t}_0, 2\gamma_2)$
  return $(\mathbf{w}'_1, c, (\mathbf{z}, \mathbf{h}))$
else
  $\mathbf{w} := \mathbf{A}\mathbf{y} + \mathbf{e}_w$
  $(\mathbf{w}_1, \mathbf{w}_0) = \mathsf{DecomposeMod}_q(\mathbf{w}, 2\gamma_2)$
  return $(\mathbf{w}_1, c, \perp)$

$\underline{\mathsf{Trans}_3(\mathsf{sk} = (\rho, \mathbf{s}, \mathbf{e}, \mathbf{t}_1, \mathbf{t}_0))}$

$\mathbf{A} := \mathsf{ExpandA}(\rho)$
$\mathbf{e}_w \leftarrow S^k_{\eta}$
$c \leftarrow C$
$\rho' \leftarrow [0, 1)$
if $\rho' \leq p_{\mathbf{z}}$ **then** // Accepting $\mathbf{z}$.
  $\mathbf{z} \leftarrow [-\gamma_1 + \beta + 1, \gamma_1 - \beta - 1]^{N\ell}$
  $\mathbf{w}' = \mathbf{A}\mathbf{z} - c\mathbf{t} + \mathbf{e}_w$
  $(\mathbf{w}'_1, \mathbf{w}'_0) = \mathsf{DecomposeMod}_q(\mathbf{w}', 2\gamma_2)$
  if $\left\|\mathbf{w}'_0\right\|_\infty \geq \gamma_2 - \beta$ **then**
    $\mathbf{w} = \mathbf{A}\mathbf{z} - c\mathbf{t} + c\mathbf{e}$
    $(\mathbf{w}_1, \mathbf{w}_0) = \mathsf{DecomposeMod}_q(\mathbf{w}, 2\gamma_2)$
    return $(\mathbf{w}_1, c, \perp)$
  $\mathbf{h} := \mathsf{MakeHint}_q(-c\mathbf{t}_0, \mathbf{A}\mathbf{z} - c\mathbf{t} + c\mathbf{t}_0, 2\gamma_2)$
  return $(\mathbf{w}'_1, c, (\mathbf{z}, \mathbf{h}))$
else
  $\mathbf{y} \leftarrow \{\mathbf{y}' \in \tilde{S}^{\ell}_{\gamma_1} : \|c\mathbf{s} + \mathbf{y}'\|_\infty \geq \gamma_1 - \beta\}$
  $\mathbf{w} := \mathbf{A}\mathbf{y} + \mathbf{e}_w$
  $(\mathbf{w}_1, \mathbf{w}_0) = \mathsf{DecomposeMod}_q(\mathbf{w}, 2\gamma_2)$
  return $(\mathbf{w}_1, c, \perp)$

Algorithm 13: Hybrids for Theorem 2 (continued)

$\underline{\mathsf{Trans}_4(\mathsf{sk}=(\rho,\mathbf{s},\mathbf{e},\mathbf{t}_1,\mathbf{t}_0))}$

$\mathbf{A} := \mathsf{ExpandA}(\rho)$
$\mathbf{e}_w \leftarrow S_\eta^k$
$c \leftarrow C$
$\rho' \leftarrow [0,1)$
if $\rho' \le p_{\mathbf{z}}$ **then** // Accepting $\mathbf{z}$.
  $\mathbf{z} \leftarrow [-\gamma_1+\beta+1, \gamma_1-\beta-1]^{N\ell}$
  $\mathbf{w}' = \mathbf{Az} - c\mathbf{t} + \mathbf{e}_w$
  $(\mathbf{w}_1', \mathbf{w}_0') = \mathsf{DecomposeMod}_q(\mathbf{w}', 2\gamma_2)$
  if $\|\mathbf{w}_0'\|_\infty \ge \gamma_2 - \beta$ **then**
    $\mathbf{w} = \mathbf{w}' + c\mathbf{e}$
    $(\mathbf{w}_1, \mathbf{w}_0) = \mathsf{DecomposeMod}_q(\mathbf{w}, 2\gamma_2)$
    return $(\mathbf{w}_1, c, \perp)$
  $\mathbf{h} := \mathsf{MakeHint}_q(-c\mathbf{t}_0, \mathbf{Az} - c\mathbf{t} + c\mathbf{t}_0, 2\gamma_2)$
  return $(\mathbf{w}_1', c, (\mathbf{z}, \mathbf{h}))$
else
  $\mathbf{w} \leftarrow \mathcal{R}_q^k$
  $(\mathbf{w}_1, \mathbf{w}_0) = \mathsf{DecomposeMod}_q(\mathbf{w}, 2\gamma_2)$
  return $(\mathbf{w}_1, c, \perp)$

$\underline{\mathsf{Trans}_5(\mathsf{sk}=(\rho,\mathbf{s},\mathbf{e},\mathbf{t}_1,\mathbf{t}_0))}$

$\mathbf{A} := \mathsf{ExpandA}(\rho)$
$\mathbf{e}_w \leftarrow S_\eta^k$
$c \leftarrow C$
$\rho' \leftarrow [0,1)$
if $\rho' \le p_{\mathbf{z}}$ **then** // Accepting $\mathbf{z}$.
  $\mathbf{z} \leftarrow [-\gamma_1+\beta+1, \gamma_1-\beta-1]^{N\ell}$
  $\mathbf{w}' := \mathbf{Az} - c\mathbf{t} + \mathbf{e}_w$
  $(\mathbf{w}_1', \mathbf{w}_0') = \mathsf{DecomposeMod}_q(\mathbf{w}', 2\gamma_2)$
  if $\|\mathbf{w}_0'\|_\infty \ge \gamma_2 - \beta$ **then**
    $\mathbf{w} = \mathbf{w}' + c\mathbf{e}$
    $(\mathbf{w}_1, \mathbf{w}_0) = \mathsf{DecomposeMod}_q(\mathbf{w}, 2\gamma_2)$
    return $(\mathbf{w}_1, c, \perp)$
  else // Sample a passing signature.
    $\mathbf{w}_0' := (\gamma_2 - \beta, \ldots, \gamma_2 - \beta)$
    $i := 0$
    while $\|\mathbf{w}_0'\|_\infty \ge \gamma_2 - \beta$ **or** $i < 4\lambda$ **do**
      $i := i + 1$
      $\mathbf{z} \leftarrow [-\gamma_1+\beta+1, \gamma_1-\beta-1]^{N\ell}$
      $\mathbf{e}_w \leftarrow S_\eta^k$
      $\mathbf{w}' = \mathbf{Az} - c\mathbf{t} + \mathbf{e}_w$
      $(\mathbf{w}_1', \mathbf{w}_0') = \mathsf{DecomposeMod}_q(\mathbf{w}', 2\gamma_2)$
    $\mathbf{h} := \mathsf{MakeHint}_q(-c\mathbf{t}_0, \mathbf{Az} - c\mathbf{t} + c\mathbf{t}_0, 2\gamma_2)$
    return $(\mathbf{w}_1', c, (\mathbf{z}, \mathbf{h}))$
else
  $\mathbf{w} \leftarrow \mathcal{R}_q^k$
  $(\mathbf{w}_1, \mathbf{w}_0) = \mathsf{DecomposeMod}_q(\mathbf{w}, 2\gamma_2)$
  return $(\mathbf{w}_1, c, \perp)$

$\underline{\mathsf{Trans}_6(\mathsf{sk}=(\rho,\mathbf{s},\mathbf{e},\mathbf{t}_1,\mathbf{t}_0))}$

$\mathbf{A} := \mathsf{ExpandA}(\rho)$
$c \leftarrow C$
$\rho' \leftarrow [0,1)$
if $\rho' \le p_{\mathbf{z}}$ **then** // Accepting $\mathbf{z}$.
  $\mathbf{w}' \leftarrow \mathcal{R}_q^k$
  $(\mathbf{w}_1', \mathbf{w}_0') = \mathsf{DecomposeMod}_q(\mathbf{w}', 2\gamma_2)$
  if $\|\mathbf{w}_0'\|_\infty \ge \gamma_2 - \beta$ **then**
    $\mathbf{w} := \mathbf{w}' + c\mathbf{e}$
    $(\mathbf{w}_1, \mathbf{w}_0) = \mathsf{DecomposeMod}_q(\mathbf{w}, 2\gamma_2)$
    return $(\mathbf{w}_1, c, \perp)$
  else
    $\mathbf{w}_0' := (\gamma_2 - \beta, \ldots, \gamma_2 - \beta)$
    $i := 0$
    while $\|\mathbf{w}_0'\|_\infty \ge \gamma_2 - \beta$ **or** $i < 4\lambda$ **do**
      $i := i + 1$
      $\mathbf{z} \leftarrow [-\gamma_1+\beta+1, \gamma_1-\beta-1]^{N\ell}$
      $\mathbf{e}_w \leftarrow S_\eta^k$
      $\mathbf{w}' = \mathbf{Az} - c\mathbf{t} + \mathbf{e}_w$
      $(\mathbf{w}_1', \mathbf{w}_0') = \mathsf{DecomposeMod}_q(\mathbf{w}', 2\gamma_2)$
    $\mathbf{h} := \mathsf{MakeHint}_q(-c\mathbf{t}_0, \mathbf{Az} - c\mathbf{t} + c\mathbf{t}_0, 2\gamma_2)$
    return $(\mathbf{w}_1', c, (\mathbf{z}, \mathbf{h}))$
else
  $\mathbf{w} \leftarrow \mathcal{R}_q^k$
  $(\mathbf{w}_1, \mathbf{w}_0) = \mathsf{DecomposeMod}_q(\mathbf{w}, 2\gamma_2)$
  return $(\mathbf{w}_1, c, \perp)$

$\underline{\mathsf{Trans}_7(\mathsf{sk}=(\rho,\mathbf{s},\mathbf{e},\mathbf{t}_1,\mathbf{t}_0)) = \mathsf{Sim}(\mathsf{pk}=(\rho,\mathbf{t}_1,\mathbf{t}_0))}$

$\mathbf{A} := \mathsf{ExpandA}(\rho)$
$c \leftarrow C$
$\rho' \leftarrow [0,1)$
if $\rho' \le p_{\mathbf{z}}$ **then** // Accepting $\mathbf{z}$.
  $\mathbf{w}' \leftarrow \mathcal{R}_q^k$
  $(\mathbf{w}_1', \mathbf{w}_0') = \mathsf{DecomposeMod}_q(\mathbf{w}', 2\gamma_2)$
  if $\|\mathbf{w}_0'\|_\infty \ge \gamma_2 - \beta$ **then**
    for $i \in [kN]$ **do**
      if $|w_{0,i}'| < \gamma_2 - \beta$ **then**
        $w_{1,i} \leftarrow \mathcal{U}$ // Uniform in $\mathbb{Z}_\delta$
      else
        $w_{1,i} \leftarrow \tilde{\mathcal{U}}$ // See (2)
    $\mathbf{w}_1 := (w_{1,1}, \ldots, w_{1,kN})$
    return $(\mathbf{w}_1, c, \perp)$
  else
    $\mathbf{w}_0' := (\gamma_2 - \beta, \ldots, \gamma_2 - \beta)$
    $i := 0$
    while $\|\mathbf{w}_0'\|_\infty \ge \gamma_2 - \beta$ **or** $i < 4\lambda$ **do**
      $i := i + 1$
      $\mathbf{z} \leftarrow [-\gamma_1+\beta+1, \gamma_1-\beta-1]^{N\ell}$
      $\mathbf{e}_w \leftarrow S_\eta^k$
      $\mathbf{w}' = \mathbf{Az} - c\mathbf{t} + \mathbf{e}_w$
      $(\mathbf{w}_1', \mathbf{w}_0') = \mathsf{DecomposeMod}_q(\mathbf{w}', 2\gamma_2)$
    $\mathbf{h} := \mathsf{MakeHint}_q(-c\mathbf{t}_0, \mathbf{Az} - c\mathbf{t} + c\mathbf{t}_0, 2\gamma_2)$
    return $(\mathbf{w}_1', c, (\mathbf{z}, \mathbf{h}))$
else
  $\mathbf{w} \leftarrow \mathcal{R}_q^k$
  $(\mathbf{w}_1, \mathbf{w}_0) = \mathsf{DecomposeMod}_q(\mathbf{w}, 2\gamma_2)$
  return $(\mathbf{w}_1, c, \perp)$

B.2 Proof of Theorem 1

Quantum Random Oracle Model In the QROM [BDF+11], adversaries are allowed to evaluate a random oracle $\mathsf{H}: X \to Y$ on quantum states. Formally, we say that the adversary $\mathcal{A}$ has quantum access to the random oracle H, denoted by $\mathcal{A}^{|\mathsf{H}\rangle}$, if the oracle performs the unitary operation $|x\rangle|y\rangle \mapsto |x\rangle|y \oplus \mathsf{H}(x)\rangle$ for any $x \in X$ and $y \in Y$ as the computational basis. A classical version of the random oracle H can be obtained by measuring the first register storing x of the input state in the computational basis before applying the above unitary operation.

Adaptive Reprogramming in the QROM Before proving Theorem 1, we state the adaptive reprogramming lemma from [GHHM21], which allows us to transition between hybrid games of which one reprograms the random oracle on input $(\mu, \mathbf{w}_1)$, while the other does not.

Lemma 9 (Adaptive Reprogramming (Proposition 2 of [GHHM21]))**.** *Let X_1, X_2, X', Y be finite sets. and let $\mathcal{D}$ be a distribution on $X_1 \times X'$. Let $\mathcal{A}$ be a distinguisher, issuing queries Q many queries to the random oracle H_b, and R classical queries to the reprogramming oracle in the adaptive reprogramming game (Algorithm 14). Then*

$$|\Pr[\mathsf{AR}_1 = 1] - \Pr[\mathsf{AR}_0 = 1]| \leq \frac{3R}{2}\sqrt{Q \cdot 2^{-\alpha}}.$$

where α is the min-entropy of the first component of $\mathcal{D}$.

Algorithm 14: Adaptive Reprogramming Game

$\underline{\mathsf{AR}_b}$
1: $\mathsf{H}_0 \leftarrow \mathcal{U}(Y^{X_1 \times X_2})$
2: $\mathsf{H}_1 := \mathsf{H}_0$
3: $b' \leftarrow \mathcal{A}^{|\mathsf{H}_b\rangle, \mathsf{Reprogram}}$
4: **return** b'

$\underline{\mathsf{Reprogram}(x_1)}$
1: $(x_2, x') \leftarrow \mathcal{D}$
2: $y \leftarrow \mathcal{U}(Y)$
3: $\mathsf{H}_1 := \mathsf{H}_1^{(x_1,x_2)\mapsto y}$
4: **return** (x_2, x')

Proving UF-CMA$_\perp$ Security of Modified ML-DSA We adapt general results on Fiat-Shamir signatures from [GHHM21, Theorem 3] and [DFPS23, Theorem 10]. Note that [DFPS23] analyzes the UF-CMA security in which the signing oracle does not release rejected w and repeats the signing process B times, while still requiring scHVZK. As the analysis of [DFPS23, Theorem 10] with $B = 1$ coincides with [GHHM21, Theorem 3] in which an entire transcript always gets released, we can rely on their result to conclude the (standard) UF-CMA security of modified ML-DSA. Moreover, we additionally observe that it is possible to prove the UF-CMA$_\perp$ security, because the signing oracle can be even simulated without reprogramming the random oracle whenever the transcript simulator outputs $\mathbf{z} = \perp$. Overall, we obtain the following result.

Theorem 1 (detailed) *The signature scheme presented in Algorithm 1 is* UF-CMA$_\perp$ *secure in the quantum random oracle model, assuming the* UF-NMA *security of the original ML-DSA and under the* $\mathsf{MLWE}_{q,N,k,\ell,\tilde{S}^\ell_{\gamma_1},\eta}$ *and* $\mathsf{MLWE}_{q,N,k,\ell,S^\ell_{\gamma_1-\beta-1},\eta}$ *assumptions. Concretely, for any $(\varepsilon_{\mathcal{A}}, T_{\mathcal{A}})$-adversary $\mathcal{A}$ against the* UF-CMA$_\perp$ *security that makes at most Q_s queries to the signing oracle and Q_h queries to the random oracle, there exists $(\varepsilon_{\mathcal{B}}, T_{\mathcal{B}})$-adversary $\mathcal{B}$ against the* UF-NMA *security of the original ML-DSA such that*

$$\begin{aligned}\varepsilon_{\mathcal{A}} &\leq \varepsilon_{\mathcal{B}} + Q_s \cdot (\varepsilon_{\mathcal{B}_1} + p_{\mathbf{z}} \cdot (\varepsilon_{\mathcal{B}_2} + \varepsilon_{\mathcal{B}_3}) + 2^{-2\lambda}) \\ &\quad + \frac{9Q_s}{2}\sqrt{(Q_s + Q_h + 1) \cdot 2^{-\alpha}} + O(Q_h^3/2^{512}) \\ T_{\mathcal{B}} &\approx T_{\mathcal{A}} + O(Q_s \cdot T_{\mathsf{Sim}} + (Q_h + Q_s) \cdot Q_s)\end{aligned}$$

where $\alpha \geq 1028$ and $\varepsilon_{\mathcal{B}_1}$, $\varepsilon_{\mathcal{B}_2}$, and $\varepsilon_{\mathcal{B}_3}$ are the same as in Theorem 2 detailed in Appendix B.1. If $\mathcal{B}$ is allowed to use a quantum random access memory (QRAM), the running time can be improved to $T_{\mathcal{B}} \approx T_{\mathcal{A}} + O(Q_s \cdot T_{\mathsf{Sim}} + (Q_h + Q_s) \cdot \log Q_s)$.

Algorithm 15: Game G_0

$\underline{\mathsf{G}_0(1^\lambda)}$
1: $M := \varnothing$
2: $T[*] := \perp$ // Empty table
3: $i = 0$
4: $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Gen}(1^\lambda)$
5: $(\mathsf{m}^*, \sigma^*) \leftarrow \mathcal{A}^{O, |\mathsf{H}\rangle}(\mathsf{pk})$
6: **return** $\mathsf{Ver}(\mathsf{pk}, \mathsf{m}^*, \sigma^*) = 1 \wedge \mathsf{m}^* \notin M$

$\underline{O(\mathsf{m})}$
1: $\mu := \mathsf{H}(tr, \mathsf{m})$;
2: $i := i + 1$
3: $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h})) \leftarrow \mathsf{GetTrans}_0(\mu, i)$
4: **if** $\mathbf{z} \neq \perp$ **then**
5: $\quad M := M \cup \{\mathsf{m}\}$
6: **return** $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}))$

$\underline{\mathsf{GetTrans}_0(\mu, i)}$
1: $(\mathbf{w}_1, \mathsf{st}) \leftarrow \mathsf{P}_1(\mathsf{sk})$
2: $c = \mathsf{H}(\mu, \mathbf{w}_1)$
3: $(\mathbf{z}, \mathbf{h}) \leftarrow \mathsf{P}_2(\mathsf{st}, c)$
4: **return** $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}))$

Proof of Theorem 1. We prove the UF-CMA$_\perp$ security of modified ML-DSA via a sequence of games, detailed in Algorithm 17. For each game G_i below, we assume that the signing oracle O calls $\mathsf{GetTrans}_i$ as a subroutine.

Game G_0: This is identical to the UF-CMA$_\perp$ security experiment $\mathsf{Exp}^{\mathsf{cma}_\perp}_{\mathcal{A}}$, except with some additional variables for later hybrids. See Algorithm 15.

Game G_1: This is identical to the previous game except that it aborts if $\mathsf{m}^* \notin M$ and $\mu^* = \mathsf{H}(tr, \mathsf{m}^*) = \mathsf{H}(tr, \mathsf{m}) \in \{0,1\}^{512}$

for some $\mathsf{m} \neq \mathsf{m}^*$ that has been queried to the signing oracle O.[12] The probability that this event occurs can be bounded by the generic collision search probability in the QROM [Zha19, Corollary 2], and thus we have

$$|\Pr[\mathsf{G}_1(1^\lambda) = 1] - \Pr[\mathsf{G}_0(1^\lambda) = 1]| \leq O(Q_h^3/2^{512}).$$

Game G_2: This is identical to G_1 except that the signing oracle O makes a call to $\mathsf{GetTrans}_2$, which reprograms the random oracle output to a fresh challenge. This jump is identical to G_0-G_1 of [GHHM21, Theorem 3], which is justified by applying the adaptive reprogramming (Lemma 9). Thus, we have that

$$\begin{aligned}&\left|\Pr[\mathsf{G}_2(1^\lambda) = 1] - \Pr[\mathsf{G}_1(1^\lambda) = 1]\right| \\ &\leq \frac{3Q_s}{2}\sqrt{(Q_s + Q_h + 1)\cdot 2^{-\alpha}}\end{aligned}$$

where α is the min-entropy of $\mathbf{w}_1$, which is at least 1028 for all ML-DSA parameters [BBD+23].

Game G_3^j for $j = 1, \ldots, Q_s$: This is identical to G_2 except that the signing oracle O makes a call to $\mathsf{GetTrans}_3^j$, which runs the simulator Sim (Algorithm 11) to obtain a transcript for the first j invocations, while the remaining transcripts are generated by the real prover algorithm $\mathsf{P} = (\mathsf{P}_1, \mathsf{P}_2)$. Here, we denote by $\mathsf{Sim}(\mathsf{pk}; c)$ the invocation of Sim with public key pk and challenge c as input, where c is uniformly sampled from C by $\mathsf{GetTrans}_3^j$. By the scHVZK property of the underlying identification scheme (Theorem 2), we can construct a distinguisher $\mathcal{D}$ that distinguishes a real transcript from simulated one such that

$$\left|\Pr[\mathsf{G}_3^j(1^\lambda) = 1] - \Pr[\mathsf{G}_3^{j-1}(1^\lambda) = 1]\right| \leq \varepsilon_{\mathcal{D}}$$

where $\mathsf{G}_3^0 = \mathsf{G}_2$. The advantage $\varepsilon_{\mathcal{D}}$ can then be bounded by the advantage of breaking LWE by Theorem 2. Note that such a distinguisher $\mathcal{D}$ can successfully simulate the view of the adversary in G_3^j because $\mathcal{D}$ receives sk as input, and thus can generate real transcripts for the remaining $Q_s - j$ signing queries.

Game G_4: This is identical to game $\mathsf{G}_3^{Q_s}$ except that $\mathsf{GetTrans}_4$ now runs different modes of the simulator Sim depending on a random value $\rho' \in [0, 1)$ sampled uniformly at the beginning of the game. Moreover, following Sim, it runs either mode 1-A or mode 1-B depending on the norm of $\mathbf{w}_0'$. This is merely a syntactic modification, and thus we have

$$\Pr[\mathsf{G}_4(1^\lambda) = 1] = \Pr[\mathsf{G}_3^{Q_s}(1^\lambda) = 1]$$

[12] We note that ruling out this event is also necessary to prevent a trivial forgery of original ML-DSA, because such a collision allows an adversary to query the signing oracle with m, obtain a signature σ, and output σ on m^* as a valid forgery. To the best of our knowledge, this was not considered explicitly in the previous QROM analysis of ML-DSA.

Game G_5: This is identical to the previous game except that the random oracle responses are reprogrammed only if $\mathsf{GetTrans}_5$ runs mode 1-B of the simulator. Note that this modification is valid because the simulator in mode 1-A or mode 2 does not need any challenge at all. We can apply the adaptive reprogramming lemma (Lemma 9) to this jump twice, once for mode 1-A and once for mode 2. For completeness, in Algorithm 16 we present a distinguisher $\mathcal{D}$ against the reprogramming game, where the $\mathsf{Reprogram}$ oracle internally runs $\mathsf{Sim}_{1\text{-A}}$.[13] The reduction against the reprogramming game for mode 2 can be defined analogously.

By (2), the min-entropy of $\mathbf{w}_1$ in mode 1-A can be calculated as

$$\tilde{H}_\infty(\mathbf{w}_1) = -kN \cdot \log_2\left(\frac{2\beta + 2}{2(\beta+1)\delta + 1}\right).$$

In mode 2, the min-entropy can be calculated as

$$\tilde{H}_\infty(\mathbf{w}_1) = -kN \cdot \log_2\left(\frac{2\gamma_2 + 1}{q}\right).$$

In both cases, the min-entropy is at least $\alpha \geq 1028$ for all ML-DSA parameters. Thus, by applying Lemma 9 twice, we have

$$\begin{aligned}&\left|\Pr[\mathsf{G}_5(1^\lambda) = 1] - \Pr[\mathsf{G}_4(1^\lambda) = 1]\right| \\ &\leq 3Q_s\sqrt{(Q_s + Q_h + 1)\cdot 2^{-\alpha}}\end{aligned}$$

Game G_6: This is identical to the previous game except that the random oracle responses are simulated either by returning reprogrammed values recorded in the table T, or by using a private random oracle H'. This is merely a syntactic modification, and thus we have

$$\Pr[\mathsf{G}_6(1^\lambda) = 1] = \Pr[\mathsf{G}_5(1^\lambda) = 1]$$

Game G_7: This is identical to the previous game except that it aborts if $\mathsf{m}^* \notin M$ and $\mathsf{H}(\mathbf{w}_1^*, \mu^*) \neq \mathsf{H}'(\mathbf{w}_1^*, \mu^*)$, where $\mathbf{w}_1^*$ is reconstructed from the forgery $\sigma^* = (c^*, (\mathbf{z}^*, \mathbf{h}^*))$ and $\mu^* = \mathsf{H}(tr, \mathsf{m}^*)$. This event may potentially occur only if one of the following holds: 1) for some $\mathsf{m} \neq \mathsf{m}^*$ queried to O, it holds that $\mathsf{H}(tr, \mathsf{m}^*) = \mathsf{H}(tr, \mathsf{m})$, or 2) m^* has been queried to O, while m^* is not recorded in M. Event 1) implies that a collision has been found for the random oracle H, which is already ruled out in G_1. Assuming 1) does not occur, we can guarantee 2) does not occur. Observe that O skips recording m^* in M only if $\mathsf{GetTrans}_6$ outputs $\mathbf{z} = \perp$. In that case, $\mathsf{GetTrans}_6$ does not introduce any inconsistency between H and H' since it does

[13] While the behavior of $\mathsf{Sim}_{1\text{-A}}$ in Algorithm 11 depends on rejected $\mathbf{w}'$, one can define an equivalent subroutine that freshly samples $\mathbf{w}' \leftarrow \{\mathbf{x} \in \mathcal{R}_q^k \mid \left\|\mathsf{LowBits}_q(\mathbf{x}, 2\gamma_2)\right\|_\infty \geq \gamma_2 - \beta\}$ and samples each coefficinet of $\mathbf{w}_1$ as in the original simulator. In this manner, one can define a reprogramming oracle $\mathsf{Reprogram}$ drawing $\mathbf{w}_1$ from a fixed disribution, allowing to invoke Lemma 9.

not reprogram the random oracle at this stage. Thus, G_7 and G_6 are equivalent:

$$\Pr[\mathsf{G}_7(1^\lambda) = 1] = \Pr[\mathsf{G}_6(1^\lambda) = 1]$$

Reduction to UF-NMA**:** Finally, we can construct a reduction to the UF-NMA security of the original ML-DSA scheme. A reduction $\mathcal{B}$, upon receiving an ML-DSA public key pk and with access to a random oracle H', emulates G_7 in the eyes of $\mathcal{A}$. If the adversary $\mathcal{A}$ finds a valid signature on $\mathsf{m}^* \notin M$, then the random oracle has not been reprogrammed using $\mu^* = \mathsf{H}'(tr, \mathsf{m}^*)$ thanks to the abort condition in G_7. Hence, if $\mathcal{A}$ wins G_7, $\mathcal{B}$ can always output a valid signature in the UF-NMA game w.r.t. H', and thus we have

$$\Pr[\mathsf{G}_7(1^\lambda) = 1] \leq \Pr[\mathsf{Exp}^{\mathrm{nma}}(1^\lambda) = 1]$$

The running time of the reduction $\mathcal{B}$ follows from [DFPS23, Theorem 10]; as the database T of reprogrammed input-outputs has at most Q_s entries, each query to H can be answered in time $O(Q_s)$ via an exhaustive search (or $O(\log Q_s)$ with QRAM via a search over the sorted database). □

Algorithm 16: Reprogramming distinguisher $\mathcal{D}$

$\underline{\mathsf{Reprogram}(\mu)}$

$\mathbf{w}_1 \leftarrow \mathsf{Sim}_{1\text{-A}}(\mathsf{pk})$
$c \leftarrow C$
$\mathsf{H}_1 := \mathsf{H}_1^{(\mu,\mathbf{w}_1)\mapsto c}$
return $\mathbf{w}_1$

$\underline{\mathcal{D}^{|\mathsf{H}'_b\rangle}}$

$M := \varnothing$
$T[*] := \perp$ // Empty table
$(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Gen}(1^\lambda)$
$(\mathsf{m}^*, \sigma^*) \leftarrow \mathcal{A}^{O,|\mathsf{H}\rangle}(\mathsf{pk})$
return $\mathsf{Ver}(\mathsf{pk}, \mathsf{m}^*, \sigma^*) = 1 \wedge \mathsf{m}^* \notin M$

$\underline{O(\mathsf{m})}$

$\mu := \mathsf{H}(tr, \mathsf{m})$;
$\rho' \leftarrow [0,1)$
$(\mathbf{z}, \mathbf{h}) := (\perp, \perp)$
if $\rho' \leq p_{\mathbf{z}}$ **then**
  $\mathbf{w}' \leftarrow \mathcal{R}_q^k$
  $(\mathbf{w}'_1, \mathbf{w}'_0) = \mathsf{Decompose}(\mathbf{w}', 2\gamma_2)$
  if $||\mathbf{w}'_0|| \geq \gamma_2 - \beta$ **then**
    $\mathbf{w}_1 \leftarrow \mathsf{Reprogram}(\mu)$
    $c := \mathsf{H}(\mu, \mathbf{w}_1)$
  else
    $c \leftarrow C$
    $(\mathbf{w}_1, (\mathbf{z}, \mathbf{h})) \leftarrow \mathsf{Sim}_{1\text{-B}}(\mathsf{pk}; c)$
    $T[\mu, \mathbf{w}_1] := c$
else
  $c \leftarrow C$
  $\mathbf{w}_1 \leftarrow \mathsf{Sim}_2(\mathsf{pk})$
  $T[\mu, \mathbf{w}_1] := c$
if $\mathbf{z} \neq \perp$ **then**
  $M := M \cup \{\mathsf{m}\}$
return $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}))$

$\underline{\mathsf{H}(\mu, \mathbf{w}_1)}$

if $T[\mu, \mathbf{w}_1] \neq \perp$ **then**
  return $T[\mu, \mathbf{w}_1]$
else
  return $\mathsf{H}'_b(\mu, \mathbf{w}_1)$

Algorithm 17: Hybrid algorithms for $\mathsf{G}_1, \mathsf{G}_2, \mathsf{G}_3^1, \ldots, \mathsf{G}_3^{Q_s}, \mathsf{G}_4, \mathsf{G}_5, \mathsf{G}_6$

$\underline{\mathsf{GetTrans}_1(\mu, i)}$

$(\mathbf{w}_1, \mathsf{st}) \leftarrow \mathsf{P}_1(\mathsf{sk})$
$c = \mathsf{H}(\mu, \mathbf{w}_1)$
$(\mathbf{z}, \mathbf{h}) \leftarrow \mathsf{P}_2(\mathsf{st}, c)$
return $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}))$

$\underline{\mathsf{GetTrans}_2(\mu, i)}$

$(\mathbf{w}_1, \mathsf{st}) \leftarrow \mathsf{P}_1(\mathsf{sk})$
$c \leftarrow C$
$(\mathbf{z}, \mathbf{h}) \leftarrow \mathsf{P}_2(\mathsf{st}, c)$
$\mathsf{H} := \mathsf{H}^{(\mu, \mathbf{w}_1) \mapsto c}$
return $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}))$

$\underline{\mathsf{GetTrans}_3^j(\mu, i)}$

$c \leftarrow C$
if $i \leq j$ **then**
  $(\mathbf{w}_1, (\mathbf{z}, \mathbf{h})) \leftarrow \mathsf{Sim}(\mathsf{pk}; c)$
else
  $(\mathbf{w}_1, \mathsf{st}) \leftarrow \mathsf{P}_1(\mathsf{sk})$
  $(\mathbf{z}, \mathbf{h}) \leftarrow \mathsf{P}_2(\mathsf{st}, c)$
$\mathsf{H} := \mathsf{H}^{(\mu, \mathbf{w}_1) \mapsto c}$
return $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}))$

$\underline{\mathsf{GetTrans}_4(\mu, i)}$

$c \leftarrow C$
$\rho' \leftarrow [0, 1)$
$(\mathbf{z}, \mathbf{h}) := (\perp, \perp)$
if $\rho' \leq p_{\mathbf{z}}$ **then**
  $\mathbf{w}' \leftarrow \mathcal{R}_q^k$
  $(\mathbf{w}_1', \mathbf{w}_0') = \mathsf{Decompose}(\mathbf{w}', 2\gamma_2)$
  if $||\mathbf{w}_0'|| \geq \gamma_2 - \beta$ **then**
    $\mathbf{w}_1 \leftarrow \mathsf{Sim}_{\text{1-A}}(\mathsf{pk})$
  else
    $(\mathbf{w}_1, (\mathbf{z}, \mathbf{h})) \leftarrow \mathsf{Sim}_{\text{1-B}}(\mathsf{pk}; c)$
else
  $\mathbf{w}_1 \leftarrow \mathsf{Sim}_2(\mathsf{pk})$
$\mathsf{H} := \mathsf{H}^{(\mu, \mathbf{w}_1) \mapsto c}$
return $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}))$

$\underline{\mathsf{GetTrans}_5(\mu, i)}$

$\rho' \leftarrow [0, 1)$
$(\mathbf{z}, \mathbf{h}) := (\perp, \perp)$
if $\rho' \leq p_{\mathbf{z}}$ **then**
  $\mathbf{w}' \leftarrow \mathcal{R}_q^k$
  $(\mathbf{w}_1', \mathbf{w}_0') = \mathsf{Decompose}(\mathbf{w}', 2\gamma_2)$
  if $||\mathbf{w}_0'|| \geq \gamma_2 - \beta$ **then**
    $\mathbf{w}_1 \leftarrow \mathsf{Sim}_{\text{1-A}}(\mathsf{pk})$
    $c = \mathsf{H}(\mu, \mathbf{w}_1)$
  else
    $c \leftarrow C$
    $(\mathbf{w}_1, (\mathbf{z}, \mathbf{h})) \leftarrow \mathsf{Sim}_{\text{1-B}}(\mathsf{pk}; c)$
    $\mathsf{H} := \mathsf{H}^{(\mu, \mathbf{w}_1) \mapsto c}$
else
  $\mathbf{w}_1 \leftarrow \mathsf{Sim}_2(\mathsf{pk})$
  $c = \mathsf{H}(\mu, \mathbf{w}_1)$
return $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}))$

$\underline{\mathsf{GetTrans}_6(\mu, i)}$

$\rho' \leftarrow [0, 1)$
$(\mathbf{z}, \mathbf{h}) := (\perp, \perp)$
if $\rho' \leq p_{\mathbf{z}}$ **then**
  $\mathbf{w}' \leftarrow \mathcal{R}_q^k$
  $(\mathbf{w}_1', \mathbf{w}_0') = \mathsf{Decompose}(\mathbf{w}', 2\gamma_2)$
  if $||\mathbf{w}_0'|| \geq \gamma_2 - \beta$ **then**
    $\mathbf{w}_1 \leftarrow \mathsf{Sim}_{\text{1-A}}(\mathsf{pk})$
    $c = \mathsf{H}(\mu, \mathbf{w}_1)$
  else
    $c \leftarrow C$
    $(\mathbf{w}_1, (\mathbf{z}, \mathbf{h})) \leftarrow \mathsf{Sim}_{\text{1-B}}(\mathsf{pk}; c)$
    $T[\mu, \mathbf{w}_1] := c$
else
  $\mathbf{w}_1 \leftarrow \mathsf{Sim}_2(\mathsf{pk})$
  $c = \mathsf{H}(\mu, \mathbf{w}_1)$
return $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}))$

$\underline{\mathsf{H}(\mu, \mathbf{w}_1)}$

if $T[\mu, \mathbf{w}_1] \neq \perp$ **then**
  return $T[\mu, \mathbf{w}_1]$
else
  return $\mathsf{H}'(\mu, \mathbf{w}_1)$

C Supplementary Material for Section 4

C.1 Missing Functionalities

Functionality 3: $\mathcal{F}_{\text{T-MLDSA}}$

Setup (DKG): On receiving (GEN) from all parties:

1: $\rho \xleftarrow{\$} \{0,1\}^{256}$
2: $\mathbf{A} := \mathsf{ExpandA}(\rho)$
3: $(\mathbf{s},\mathbf{e}) \xleftarrow{\$} S_\eta^\ell \times S_\eta^k$
4: $\mathbf{t} := \mathbf{As}+\mathbf{e} \bmod q$
5: $(\mathbf{t}_1,\mathbf{t}_0) := \mathsf{Power2Round}_q(\mathbf{t},d)$
6: $tr := \mathsf{H}(\rho,\mathbf{t}_1)$
7: $(\mathsf{pk},\mathsf{sk}) := ((\rho,\mathbf{t}_1),(\rho,tr,\mathbf{s},\mathbf{e},\mathbf{t}_1,\mathbf{t}_0))$
8: Internally store sk
9: Send $(\mathsf{pk},tr,\mathbf{t},\mathbf{t}_0)$ to the adversary and then output pk to all parties

Prep: Upon receiving (PREP) as input:

1: $\mathbf{y} \xleftarrow{\$} \tilde{S}_{\gamma_1}^\ell$ // $\tilde{S}_{\gamma_1} = S_{\gamma_1} \setminus \{x \in S_{\gamma_1} : \exists x_i \in x, x_i = -\gamma_1\}$
2: $\mathbf{e}_w \leftarrow S_\eta^k$
3: $\mathbf{w} = \mathbf{Ay}+\mathbf{e}_w \bmod q$
4: $(\mathbf{w}_1,\mathbf{w}_0) = \mathsf{DecomposeMod}_q(\mathbf{w},2\gamma_2)$
5: Internally store $(\mathbf{w}_0,\mathbf{w}_1,\mathbf{y})$

Sign: Upon receiving (SIGN, m) as input:

1: $\mu := \mathsf{H}(tr,\mathsf{m})$
2: $c = \mathsf{H}(\mu,\mathbf{w}_1)$
3: $\mathbf{z} = c\mathbf{s}+\mathbf{y}$
4: $\bar{\mathbf{w}}_0 = \mathbf{w}_0 - c\mathbf{e}$
5: **if** $\|\mathbf{z}\|_\infty \geq \gamma_1-\beta$ or $\|\bar{\mathbf{w}}_0\|_\infty \geq \gamma_2-\beta$ **then**
6: $\quad(\mathbf{z},\mathbf{h}) = (\perp,\perp)$
7: **else**
8: $\quad\mathbf{h} = \mathsf{MakeHint}_q(-c\mathbf{t}_0, \mathbf{Az}-c\mathbf{t}+c\mathbf{t}_0, 2\gamma_2)$
9: $\quad$**if** $\|c\mathbf{t}_0\|_\infty \geq \gamma_2$ or $\mathsf{HW}(\mathbf{h}) > \omega$ **then**
10: $\quad\quad\mathbf{h} = \perp$
11: Send $(\mathbf{w}_1,c,\mathbf{z},\mathbf{h})$ to the adversary and then output the same to all callers

Functionality 4: $\mathcal{F}_{\text{LTC}}[B]$

On receiving shares $[\![x]\!]$ from the honest parties:

1: Reconstruct $x \leftarrow [\![x]\!]$ and the shares of the corrupt parties, and send the shares to the adversary.
2: let $b = (x < B)$
3: Receive t shares from the adversary, complete $[\![b]\!]_2$ from these and send shares to the honest parties.

Functionality 5: $\mathcal{F}_{\text{BitDec}}$

On receiving shares of $[\![\mathbf{w}]\!]$ from the honest parties:

1: Reconstruct $\mathbf{w}$ and the shares of the corrupted parties, $\mathbf{w}^j$ for $j \in \mathcal{C}$.
2: Send $\{\mathbf{w}^j\}_j$ to the adversary.
3: **for** $i = 1,\ldots,m$ **do**
4: $\quad$Receive from the adversary corrupted parties' shares $\mathbf{w}^j$, for $j \in \mathcal{C}$
5: $\quad$Using $\{\mathbf{w}^j\}_j$ and $\mathbf{w}$, compute sharing $[\![\mathbf{w}]\!]_2$
6: Output to the parties their shares of $[\![\mathbf{w}_1]\!]_2,\ldots[\![\mathbf{w}_m]\!]_2$

Functionality 6: $\mathcal{F}_{\text{RejSamp}}$

On receiving shares of $[\![\mathbf{z}]\!]$ and $[\![\bar{\mathbf{w}}_0]\!]$ from the honest parties:

1: Reconstruct $\mathbf{z} \leftarrow [\![\mathbf{z}]\!]$ and $\bar{\mathbf{w}}_0 \leftarrow [\![\bar{\mathbf{w}}_0]\!]$, and the shares of the corrupt parties, and send the latter to the adversary.
2: **if** $\|\mathbf{z}\|_\infty < \gamma_1-\beta$ and $\|\bar{\mathbf{w}}_0\|_\infty < \gamma_2-\beta-\eta$ **then** set $b \leftarrow 0$
3: **else** set $b \leftarrow 1$
4: Send bit b to the adversary.
5: Once the adversary replies with continue, output b to the parties.

C.2 Instantiating Secure Comparison

Now we address the missing piece of securely comparing a private and a public value. Fortunately, this operation is a fundamental primitive in a wide range of MPC applications, and hence we are able to leverage an ample set of techniques from the literature. For our particular use-case, we prioritize *simple* protocols with *low communication complexity*, and for this we determine an adaptation of Rabbit [MRVW21] to be the best candidate. For this protocol, a few functionalities are needed. First, it requires edabits (Functionality $\mathcal{F}_{\text{edabits}}$). Second, a *simpler* secure comparison functionality is needed, which compares $x < B$, where B is public, and instead of having x secret-shared arithmetically as in $\mathcal{F}_{\text{LTC}}$, every bit of x is secret-shared as $[\![x_1]\!]_2,\ldots,[\![x_m]\!]_2$.

Consider the following functionality.

Functionality 7: $\mathcal{F}_{\text{BitLTC}}[B]$

The functionality takes in m binary shares $[\![x_1]\!]_2,\ldots,[\![x_m]\!]_2$ from the honest parties, and proceeds as follows:

1: Reconstruct $x_1,\ldots,x_m$ alongside the corrupt parties' shares, and send the shares to the adversary.
2: Compute $x := \sum_{i=1}^m x_i 2^{i-1}$.
3: Compute $b := (B < x)$
4: Receive from the adversary corrupted parties' shares b^j for $j \in \mathcal{C}$.
5: Sample $[\![b]\!]_2$ based on b and $\{b^j\}_j$ and send the parties their shares.

Given $\mathcal{F}_{\text{BitLTC}}$, we instantiate $\mathcal{F}_{\text{LTC}}$ by adapting Rabbit [MRVW21], as in Protocol Π_{LTC} below.

Protocol 18: $\Pi_{\mathsf{LTC}}[B]$, [MRVW21]

The protocol takes as input a shared value $[\![x]\!]$.

1: Call $([\![r]\!];[\![r_1]\!]_2,\ldots,[\![r_m]\!]_2) \leftarrow \mathcal{F}_{\mathsf{edabits}}[m]$.
2: Locally compute $[\![a]\!] := [\![x+r]\!] \leftarrow [\![x]\!] + [\![r]\!]$.
3: Call $a \leftarrow \mathcal{F}_{\mathsf{Open}}([\![a]\!])$.
4: Compute $b \leftarrow (a-B) \mod q$.
5: // The two $\mathcal{F}_{\mathsf{BitLTC}}$ calls are run in parallel.
6: $[\![w_1]\!]_2 \leftarrow \mathcal{F}_{\mathsf{BitLTC}}[a]([\![r_1]\!]_2,\ldots,[\![r_m]\!]_2)$ // $a < r$
7: $[\![w_2]\!]_2 \leftarrow \mathcal{F}_{\mathsf{BitLTC}}[b]([\![r_1]\!]_2,\ldots,[\![r_m]\!]_2)$ // $b < r$
8: $w_3 = (b \geq q - B)$
9: **return** $[\![w_1]\!]_2 \oplus [\![w_2]\!]_2 \oplus w_3$

Lemma 10. *$\Pi_{\mathsf{LTC}}[B]$ UC-realizes $\mathcal{F}_{\mathsf{LTC}}[B]$ in the $(\mathcal{F}_{\mathsf{edabits}}, \mathcal{F}_{\mathsf{Open}}, \mathcal{F}_{\mathsf{BitLTC}})$-hybrid model.*

Proof. First we provide a simulator $\mathcal{S}$ for $\Pi_{\mathsf{LTC}}[B]$. $\mathcal{S}$ receives from $\mathcal{F}_{\mathsf{LTC}}[B]$ the corrupted parties' shares of $[\![x]\!]$. For its emulation of $\mathcal{F}_{\mathsf{edabits}}$, it receives from the adversary the corrupted parties' shares of $([\![r]\!];[\![r_1]\!]_2,\ldots,[\![r_m]\!]_2)$. It can thus compute the corrupted parties' shares of $[\![a]\!]$. For its emulation of $\mathcal{F}_{\mathsf{Open}}$, if the adversary does not send the same shares as above, it sends abort to $\mathcal{F}_{\mathsf{LTC}}[B]$. Otherwise, it sends random $a \in \mathbb{F}_q$ to the adversary. For its emulations of $\mathcal{F}_{\mathsf{BitLTC}}$ if the adversary does not send the same shares as $\mathcal{S}$ has, it sends abort to $\mathcal{F}_{\mathsf{LTC}}[B]$. Otherwise, it receives from the adversary the corrupted parties' shares of $[\![w_1]\!]_2$ and $[\![w_2]\!]_2$. Finally, it can locally compute and send to $\mathcal{F}_{\mathsf{LTC}}[B]$ the corrupted parties' shares of $1 \oplus [\![w_1]\!]_2 \oplus [\![w_2]\!]_2 \oplus w_3$.

To show that the ideal world is identical to the real world, first note that $[\![r]\!]$ is random conditioned on the corrupted parties' shares, and thus so too is $[\![a]\!]$. Thus, in both the real world and ideal world, the adversary receives a random a from $\mathcal{F}_{\mathsf{Open}}$ and $\mathcal{S}$'s emulation of it, respectively. All other operations are local or emulations of $\mathcal{F}_{\mathsf{BitLTC}}$ which only take information in from the adversary. Therefore, the two worlds are identical.

Now we show correctness. First, for any integer x, let $(x)_q := x \mod q$. Define the function over the integers:

$$\mathsf{LT}(x,y) = \begin{cases} 1 & x < y \\ 0 & \text{o.w.} \end{cases}$$

For any integers $x, y \in \mathbb{Z}$, it is easy to see that

$$\begin{aligned}(x+y)_q &= (x)_q + (y)_q - q \cdot \mathsf{LT}((x+y)_q, (x)_q) \\ &= (x)_q + (y)_q - q \cdot \mathsf{LT}((x+y)_q, (y)_q)\end{aligned}$$

Now note that $b \equiv_q a + (q-B)$. Let $c = q - B$ and $d = (x+c)_q$. From above, we can see that the integer $b = (a+c)_q$ can be written as $b = a + c - q \cdot \mathsf{LT}(b,c)$. Furthermore, $a = (x+r)_q$ can be written as $a = x + r - q \cdot \mathsf{LT}(a,r)$. Thus,

$$b = x + r - q \cdot \mathsf{LT}(a,r) + c - q \cdot \mathsf{LT}(b,c). \tag{3}$$

Written differently, the integer $b = (d+r) \mod q$ can be written as $b = d + r - q \cdot \mathsf{LT}(b,r)$. Furthermore, $d = (x+c)_q$ can be written as $d = x + c - q \cdot \mathsf{LT}(d,c)$ Thus,

$$b = x + c - q \cdot \mathsf{LT}(d,c) + r - q \cdot \mathsf{LT}(b,r). \tag{4}$$

Taking the RHS of Eq. 3 and Eq. 4, we get:

$$\mathsf{LT}(a,r) + \mathsf{LT}(b,c) = \mathsf{LT}(d,c) + \mathsf{LT}(b,r)$$

Now observe that $(x < B)$ iff $\overline{\mathsf{LT}(d,c)}$. Indeed, if $x < B$ then $q > (x+q-B)_q \geq q - B$ and so $d \geq c$. On the other hand, if $x \geq B$ then $0 \leq (x+q-B)_q < q - B$ and so $d < c$. Furthermore, we have from above that $\mathsf{LT}(d,c) = \mathsf{LT}(a,r) + \mathsf{LT}(b,c) - \mathsf{LT}(b,r)$. Since $\mathsf{LT}(d,c) \in \{0,1\}$ and the above identity is over the integers, it must be the case that we cannot simultaneously have $(\mathsf{LT}(a,r) = \mathsf{LT}(b,c) = 0$ and $\mathsf{LT}(b,r) = 1)$ OR $(\mathsf{LT}(a,r) = \mathsf{LT}(b,c) = 1$ and $\mathsf{LT}(b,r) = 0)$. Thus, we can also write $\mathsf{LT}(d,c) = \mathsf{LT}(a,r) \oplus \mathsf{LT}(b,c) \oplus \mathsf{LT}(b,r)$.

Putting the above together, we have

$$\begin{aligned}(x < B) &= 1 \oplus \mathsf{LT}(a,r) \oplus \mathsf{LT}(b,c) \oplus \mathsf{LT}(b,r) \\ &= \mathsf{LT}(a,r) \oplus \mathsf{LT}(b,r) \oplus (b \geq c),\end{aligned}$$

which is exactly the shared value that Π_{LTC} returns, since $(1 \oplus \mathsf{LT}(r, a+1)) = (r \geq a+1) = \mathsf{LT}(a,r)$ and $(1 \oplus \mathsf{LT}(r, b+1)) = (r \geq b+1) = \mathsf{LT}(b,r)$. □

C.3 Instantiating Bitwise Comparison

It only remains to see how to instantiate $\mathcal{F}_{\mathsf{BitLTC}}$. Let x be the integer corresponding to $x_1,\ldots,x_m$. We want to compute $s = (B < x)$. Let $d = 2^m + B - x$. Observe that $0 < d < 2^{m+1}$ and let $d = d_{m+1},\ldots,d_1$ be its binary representation. If $B - x < 0$ then $d_{m+1} = 0$ and if $B - x \geq 0$ then $d_{m+1} = 1$. Therefore, $s = 1 - d_{m+1}$. Let $x' = \neg x$, where $\neg x$ is the bitwise negation of x, and observe that $2^m - x = x' + 1$. Therefore, the bit d_k can be found by computing the carry-out bit resulting from binary addition of $x' + 1$ and B. This is what Π_{BitLTC} does, with the help of recursive procedure $\pi_{\mathsf{CarryOutAux}}$ to compute the carry-out bit and $\pi_{\mathsf{CarryProp}}$ to compute the carry propagation operator. Explanation of these latter procedures can be found in, e.g., [EL03].

Procedure 19: $\pi_{\mathsf{CarryProp}}$ ([CH10])

Takes in binary shares $([\![p_1]\!]_2, [\![g_1]\!]_2)$ and $([\![p_2]\!]_2, [\![g_2]\!]_2)$.

1: $[\![p]\!]_2 \leftarrow \mathcal{F}_{\mathsf{Mult}}([\![p_1]\!]_2, [\![p_2]\!]_2)$
2: $[\![x]\!]_2 \leftarrow \mathcal{F}_{\mathsf{Mult}}([\![p_2]\!]_2, [\![g_1]\!]_2)$
3: $[\![y]\!]_2 \leftarrow \mathcal{F}_{\mathsf{Mult}}([\![g_2]\!]_2, [\![x]\!]_2)$
4: $[\![g]\!]_2 \leftarrow [\![g_2]\!]_2 \oplus [\![x]\!]_2 \oplus [\![y]\!]_2$
5: **return** $([\![p]\!]_2, [\![g]\!]_2)$

Procedure 20: $\pi_{\mathsf{CarryOutAux}}$ ([CH10])

Takes in m pairs of binary shares $([\![p_1]\!]_2,[\![g_1]\!]_2),\ldots,([\![p_m]\!]_2,[\![g_m]\!]_2)$.

1: **if** $m=1$ **then**
2: **return** $([\![p_1]\!]_2,[\![g_1]\!]_2)$
3: **else**
4: **for** $i\in[m/2]$ **do**
5: $([\![p'_i]\!]_2,[\![g'_i]\!]_2) \leftarrow \pi_{\mathsf{CarryProp}}(([\![p_{2i}]\!]_2,[\![g_{2i}]\!]_2),([\![p_{2i-1}]\!]_2,[\![g_{2i-1}]\!]_2))$
6: **return** $\pi_{\mathsf{CarryOutAux}}((([\![p'_i]\!]_2,[\![g'_i]\!]_2)_{i=1}^{m/2})$

Protocol 21: $\Pi_{\mathsf{BitLTC}}[B]$ ([CH10])

Takes in m binary shares $[\![x_1]\!]_2,\ldots,[\![x_m]\!]_2$. Let $x=\sum_{i=1}^m x_i\cdot 2^{i-1}$. Let b_i be the i^{th} bit of $B=\sum_{i=1}^m b_i\cdot 2^{i-1}$.

1: **for** $i\in[m]$ **do**
2: $[\![x'_i]\!]_2 \leftarrow 1\oplus[\![x_i]\!]_2$
3: $[\![p_i]\!]_2 \leftarrow b_i\oplus[\![x'_i]\!]_2$
4: $[\![g_i]\!]_2 \leftarrow b_i[\![x'_i]\!]_2$
5: $[\![g_1]\!]_2 \leftarrow [\![g_1]\!]_2\oplus[\![p_1]\!]_2$
6: $([\![p]\!]_2,[\![g]\!]_2) \leftarrow \pi_{\mathsf{CarryOutAux}}((([\![p_i]\!]_2,[\![g_i]\!]_2)_{i=1}^m)$
7: **return** $1\oplus[\![g]\!]_2$

Lemma 11. *$\Pi_{\mathsf{BitLTC}}[B]$ UC-realizes $\mathcal{F}_{\mathsf{BitLTC}}[B]$ in the $\mathcal{F}_{\mathsf{Mult}}$-hybrid world.*

The above lemma is straightforward to prove since $\Pi_{\mathsf{BitLTC}}[B]$ consists of only local operaitons and invocations of $\mathcal{F}_{\mathsf{Mult}}$; correctness is explained above.

C.3.1 Costs of Π_{BitLTC}

The interaction in Protocol Π_{BitLTC} happens in Procedure $\pi_{\mathsf{CarryOutAux}}$. This procedure involves calling Procedure $\pi_{\mathsf{CarryProp}}$ a total of $\approx m$ times, distributed in $\lceil\log m\rceil$ rounds. Each call to $\pi_{\mathsf{CarryProp}}$ requires 3 calls to $\mathcal{F}_{\mathsf{Mult}}$, distributed across 2 rounds.

C.4 Instantiating Bit Decomposition

Our goal is to instantiate Functionality $\mathcal{F}_{\mathsf{BitDec}}$, which is used by the preprocessing protocol Π_{Prep}.

For this, a binary adder functionality is required. We model this as Functionality $\mathcal{F}_{\mathsf{BitAdd}}$.

Functionality 8: $\mathcal{F}_{\mathsf{BitAdd}}$

Takes 2 length-m vectors of binary shares, $([\![x_1]\!]_2,\ldots,[\![x_m]\!]_2)$ and $([\![y_1]\!]_2,\ldots,[\![y_m]\!]_2)$, and outputs $m+1$ shared bits representing the sum of the two integers $x+y$.

1: Reconstruct $x_i=\mathsf{Recover}([\![x_i]\!]_2)$ and $y_i=\mathsf{Recover}([\![y_i]\!]_2)$ from honest parties' shares, for $i\in[m]$.
2: Find $z_1,\ldots,z_{m+1}\in\{0,1\}$ such that $\sum_{j=1}^{m+1}2^{j-1}z_j=(\sum_{i=1}^m 2^{i-1}x_i)+(\sum_{i=1}^m 2^{i-1}y_i)$
3: For each $i\in[m+1]$, receive from the adversary corrupted parties' shares z_i^j, for $j\in C$.
4: Using $\{z_i^j\}_j$ and z_i, compute sharing $[\![z_i]\!]_2$ for all $i\in[m+1]$.
5: **return** $[\![z_1]\!]_2,\ldots,[\![z_{m+1}]\!]_2$ to the parties.

To instantiate $\mathcal{F}_{\mathsf{BitAdd}}$, we use generic MPC based on additions and multiplications over $\mathbb{F}_2$ to compute a binary adder circuit, such as the textbook ripple-carry adder or the carry-lookahead adder [WS58].

Now we are ready to instantiate $\mathcal{F}_{\mathsf{BitDec}}$.

Protocol 22: Π_{BitDec} ([NO07])

Parameters: This protocol takes in arithmetic shares $[\![\mathbf{w}]\!]$.

1: **for** $i=1\ldots,k$ **do**
2: The parties call $([\![r]\!],[\![r_1]\!]_2,\ldots,[\![r_m]\!]_2) \leftarrow \mathcal{F}_{\mathsf{edabits}}[m]$
3: The parties compute $[\![c]\!]\leftarrow[\![\mathbf{w}_i]\!]-[\![r]\!]$ and open $c\leftarrow\mathcal{F}_{\mathsf{Open}}([\![c]\!])$. Let $c_1,\ldots,c_m$ be the bit representation of c.
4: **if** $c=0$ **then**, set $(\mathbf{w}_{1,i},\ldots,\mathbf{w}_{m,i})\leftarrow([\![r_1]\!]_2,\ldots,[\![r_m]\!]_2)$ (since then $\mathbf{w}_i=r$) and continue to next loop iteration, $i+1$.
5: **else** ($c\neq 0$) the parties continue this loop iteration, i (below).
6: The parties compute $[\![p]\!]_2\leftarrow\mathcal{F}_{\mathsf{BitLTC}}[q-c]([\![r_1]\!]_2,\ldots,[\![r_m]\!]_2)$ and $[\![\hat{p}]\!]_2\leftarrow 1\oplus[\![p]\!]_2$.
7: Let $f_1,\ldots,f_m$ be the bit representation of 2^m+c-q. The parties compute $[\![g_j]\!]_2\leftarrow(f_j\oplus c_j[\![\hat{p}]\!]_2\oplus c_j$ for $j\in[m]$.
8: The parties compute $[\![\mathbf{w}_{1,i}]\!]_2,\ldots,[\![\mathbf{w}_{m+1,i}]\!]_2\leftarrow\mathcal{F}_{\mathsf{BitAdd}}(([\![r_1]\!]_2,\ldots,[\![r_m]\!]_2),([\![g_1]\!]_2,\ldots,[\![g_m]\!]_2))$ and discard $[\![\mathbf{w}_{m+1,i}]\!]_2$
9: **return** $[\![\mathbf{w}_1]\!]_2,\ldots,[\![\mathbf{w}_m]\!]_2$

Lemma 12. *Π_{BitDec} UC-realizes $\mathcal{F}_{\mathsf{BitDec}}$ in the $(\mathcal{F}_{\mathsf{edabits}},\mathcal{F}_{\mathsf{Open}},\mathcal{F}_{\mathsf{BitLTC}},\mathcal{F}_{\mathsf{BitAdd}})$-hybrid model.*

Proof. We first provide a simulator $\mathcal{S}$ for Π_{BitDec}. $\mathcal{S}$ first receives from $\mathcal{F}_{\mathsf{BitDec}}$ the corrupted parties' shares of $[\![\mathbf{w}]\!]$. Then for each $i\in[k]$: $\mathcal{S}$ then emulates $\mathcal{F}_{\mathsf{edabits}}$ and receives from the adversary the corrupted parties' shares of $[\![r]\!],[\![r_1]\!]_2,\ldots,[\![r_m]\!]_2$. For the opening of $[\![c]\!]$, $\mathcal{S}$ computes the corrupted parties' shares of $[\![\mathbf{w}_i]\!]-[\![r]\!]$ and if these are not what the adversary gives to $\mathcal{S}$, $\mathcal{S}$ sends abort to $\mathcal{F}_{\mathsf{BitDec}}$. Otherwise, for its emulation of $\mathcal{F}_{\mathsf{Open}}$, $\mathcal{S}$ samples random c and sends it to the adversary. If $c=0$, $\mathcal{S}$ will input to $\mathcal{F}_{\mathsf{BitDec}}$ for this i, the corrupted parties' shares of $[\![r_1]\!]_2,\ldots,[\![r_m]\!]_2$. If $c\neq 0$, $\mathcal{S}$ can forward the corrupted parties' shares of $[\![\mathbf{w}_{1,i}]\!]_2,\ldots,[\![\mathbf{w}_{m+1,i}]\!]_2$ received from the adversary during its emulation of $\mathcal{F}_{\mathsf{BitAdd}}$ to $\mathcal{F}_{\mathsf{BitDec}}$. Indeed, note that the computation of each $[\![g_j]\!]_2$ is done locally, and $\mathcal{S}$ only receives from

the adversary during its emulations of $\mathcal{F}_{\mathsf{BitLTC}}$ and $\mathcal{F}_{\mathsf{BitAdd}}$; thus no additional simulation is needed.

Since $\mathcal{S}$ emulates $\mathcal{F}_{\mathsf{edabits}}$ exactly, to show that the real world and ideal world are identically distributed, we only need to argue that the openings of $[\![c]\!]$ is the same in the two worlds. The honest parties' shares define the sharings $[\![\mathbf{w}_i]\!], [\![r]\!]$, so in both worlds if the adversary does not input the correct corrupted parties' shares of $[\![c]\!]$, the parties will abort. If this is not the case, in both worlds, due to the properties of the secret sharing scheme, the distribution of $[\![r]\!]$ is random conditioned on the corrupted parties' shares. Thus, so too is $[\![c]\!]$, and particularly c. Since $\mathcal{S}$ samples c randomly, the two worlds are identically distributed.

For correctness, if $c = 0$, this means that $\mathbf{w}_i = r$ and therefore, the parties can output the binary shares $[\![r_1]\!]_2, \ldots, [\![r_m]\!]_2$ of the edabit as the binary shares of $\mathbf{w}_i$. Otherwise, the parties need to check if over the integers $\mathbf{w}_i - r < 0$, meaning that there is overflow mod q when computing $c = (\mathbf{w}_i - r) \bmod q$. One way to check this is to see if $c + r \geq q$ or in otherwords, $\overline{r < q - c}$, which is exactly what is computed in the sharing $[\![\hat{p}]\!]_2$. Now, note that $\mathbf{w}_i$ can be represented as $\mathbf{w}_i = c + r - \hat{p}q$ over the integers. Therefore, we have $2^m + \mathbf{w}_i = 2^m + c - \hat{p}q + r$ over the integers. Consider $g = (2^m + c - \hat{p}q) \mod 2^m$ and its bit representation $(g_1, \ldots, g_m)$. Let $(f_1, \ldots, f_m)$ be the bit representation of $2^m + c - q$. Also, let $(c_1, \ldots, c_m)$ be the bit representation of c. Then the bits g_j of g can be computed as $(f_j \oplus c_j) \cdot \hat{p} \oplus c_j$. This is because $g = c$ if $\hat{p} = 0$ and $g = 2^m + c - q$ if $\hat{p} = 1$. Indeed, this is exactly how we compute each $[\![g_j]\!]_2$. Finally, adding together r and g over the integers (done by $\mathcal{F}_{\mathsf{BitAdd}}$) will yield $c + r = \mathbf{w}_i$ if $\hat{p} = 0$ or $2^m + c - q + r = 2^m + \mathbf{w}_i$ if $\hat{p} = 1$. Thus, removing the high bit of each $[\![\mathbf{w}_{1,i}]\!]_2, \ldots, [\![\mathbf{w}_{m,i}]\!]_2, [\![\mathbf{w}_{m+1,i}]\!]_2$ gives sharings of the bits of each $\mathbf{w}_i$. □

C.5 Missing Proofs

Proof of Theorem 4. We first provide a simulator $\mathcal{S}$ for Π_{Prep}. $\mathcal{S}$ emulates both calls to $\mathcal{F}_{\mathsf{Rand}}$ and receives the corrupted parties' shares from the adversary. $\mathcal{S}$ can then compute the corrupted parties' shares of $[\![\mathbf{s}]\!]$ and for its emulation of $\mathcal{F}_{\mathsf{BitDec}}$, send these to the adversary. $\mathcal{S}$, for its emulation $\mathcal{F}_{\mathsf{BitDec}}$, will then receive from the adversary either abort, in which case it sends abort to $\mathcal{F}_{\mathsf{Prep}}$, or the corrupted parties' shares of $[\![\mathbf{s}_1]\!]_2, \ldots, [\![\mathbf{s}_m]\!]_2$. Next, $\mathcal{S}$ emulates $\mathcal{F}_{\mathsf{dabits}}$ and receives the corrupted parties' shares of $([\![\mathbf{r}_1]\!]; [\![\mathbf{r}_1]\!]_2), \ldots, ([\![\mathbf{r}_d]\!], [\![\mathbf{r}_d]\!]_2)$. For the $\mathbf{c}_i$ openings, $\mathcal{S}$ computes the corrupted parties' shares of $[\![\mathbf{s}_i]\!]_2 \oplus [\![\mathbf{r}_i]\!]_2$ and if these are not what the adversary gives to $\mathcal{S}$, $\mathcal{S}$ sends abort to $\mathcal{F}_{\mathsf{Prep}}$. Otherwise, for its emulation of $\mathcal{F}_{\mathsf{Open}}$, $\mathcal{S}$ samples random $\mathbf{c}_i$ and sends it to the adversary. $\mathcal{S}$ then computes the corrupted parties' shares of $[\![\mathbf{w}_0]\!]$ and then $[\![\mathbf{w}_1]\!]$ based on $\mathbf{c}_i$ and the corrupted parties' shares of $[\![\mathbf{r}_i]\!]$. Finally, $\mathcal{S}$ inputs to $\mathcal{F}_{\mathsf{Prep}}$ the corrupted parties' shares of $[\![\mathbf{w}_0]\!]$, $[\![\mathbf{w}_1]\!]$, and $[\![\mathbf{y}]\!]$.

Since $\mathcal{S}$ exactly emulates $\mathcal{F}_{\mathsf{Rand}}$, $\mathcal{F}_{\mathsf{BitDec}}$, and $\mathcal{F}_{\mathsf{dabits}}$, to show that the real world and ideal world are identically distributed, we only need to argue that the openings of $[\![\mathbf{c}_i]\!]_2$ for $i \in [m-d]$ is the same in the two worlds. This follows from a similar argument as the one in Lemma 12, since each $[\![r_i]\!]_2$ is random conditioned on corrupted parties' shares. □

Proof of Theorem 5. First we provide a simulator $\mathcal{S}$. $\mathcal{S}$ receives from $\mathcal{F}_{\mathsf{Rand}}$ the corrupted parties' shares of $[\![\mathbf{y}]\!]$ and $[\![\mathbf{e}_w]\!]$. For its emulation of $\mathcal{F}_{\mathsf{edabits}}$, it receives from the adversary the corrupted parties' shares of $([\![\mathbf{r}]\!]; [\![\mathbf{r}_1]\!]_2, [\![\mathbf{r}_m]\!]_2)$. It can thus compute the corrupted parties' shares of $[\![s+r]\!]$. For its emulation of $\mathcal{F}_{\mathsf{Open}}$, if the adversary does not send the same shares as above, it sends abort to $\mathcal{F}_{\mathsf{Prep}}$. Otherwise, it sends random $a \in \mathbb{F}_q$ to the adversary. For its emulation of $\mathcal{F}_{\mathsf{BitLTC}}$ if the adversary does not send the same shares as $\mathcal{S}$ has, it sends abort to $\mathcal{F}_{\mathsf{Prep}}$. Otherwise, it receives from the adversary the corrupted parties' shares of $[\![\mathbf{b}]\!]_2$. For its emulation of $\mathcal{F}_{\mathsf{dabits}}$, it receives from the adversary the corrupted parties' shares of $([\![\mathbf{u}]\!]; [\![\mathbf{u}]\!]_2)$. For its emulation of $\mathcal{F}_{\mathsf{Open}}$, if the adversary does not send the same shares as above, it sends abort to $\mathcal{F}_{\mathsf{Prep}}$. Inside $\pi_{\mathsf{Mod\text{-}44}}$, For its emulation of $\mathcal{F}_{\mathsf{dabits}}$, it receives from the adversary the corrupted parties' shares of each $([\![\mathbf{s}_i]\!]; [\![\mathbf{s}_i]\!]_2)$. For its emulation of $\mathcal{F}_{\mathsf{Open}}$, if the adversary does not send the same shares as above, it sends abort to $\mathcal{F}_{\mathsf{Prep}}$. For its emulation of $\mathcal{F}_{\mathsf{Mult}}$ if the adversary does not send the same shares as $\mathcal{S}$ has, it sends abort to $\mathcal{F}_{\mathsf{Prep}}$. Otherwise, it receives from the adversary the corrupted parties' shares of $[\![\mathbf{u}_j]\!]_2$. Same with the multiplications from the computation of $F(x)$. In the end, $\mathcal{S}$ can locally compute and send to $\mathcal{F}_{\mathsf{Prep}}$ the corrupted parites' shares of $[\![\mathbf{w}_0]\!]$, $[\![\mathbf{w}_1]\!]$, and $[\![\mathbf{y}]\!]$.

It is easy to show that the ideal world is identical to the real world since $[\![\mathbf{r}]\!]$, $[\![\mathbf{u}]\!]$, and all $[\![\mathbf{s}_i]\!]_2$ are all uniformly random. All other operations are local or emulations of subfunctionaltiies, whicih only take information in from the adversary. Therefore, the two worlds are identical.

Correctness follows from the discussion above that $\pi_{\mathsf{Mod\text{-}44}}$ gives shares of $[\![\mathbf{r} \bmod \delta]\!]$, as well as the fact that computing $\mathbf{w}_1 := \mathbf{s} \bmod \delta$ is equivalent to computing the *integer* $(\mathbf{s} + \mathbf{r}) \bmod \delta$ and then subtracting $\mathbf{r} \bmod \delta$. The *integer* $(\mathbf{s} + \mathbf{r}) \bmod \delta \equiv ((\mathbf{s} + \mathbf{r}) \bmod q + (1 - \mathbf{b}) \cdot q) \bmod \delta$, where $\mathbf{b}$ is 0 for coordinates where $\mathbf{s} + \mathbf{r}$ overflows mod q and 1 otherwise, which is exactly what $\mathcal{F}_{\mathsf{BitLTC}}$ gives us. □

Proof of Theorem 3. First we provide a simulator $\mathcal{S}$ for $\Pi_{\mathsf{T\text{-}MLDSA}}$. Upon receiving (GEN), $\mathcal{S}$ receives from $\mathcal{F}_{\mathsf{T\text{-}MLDSA}}$, $\mathsf{pk} := (\rho, \mathbf{t}_1)$, *tr*, $\mathbf{t}$, and $\mathbf{t}_0$. $\mathcal{S}$ emulates $\mathcal{F}_{\mathsf{Coin}}$ by sending ρ to the adversary. $\mathcal{S}$ emulates $\mathcal{F}_{\mathsf{Rand}}$ by receiving from the adversary the corrupted parties' shares of $[\![\mathbf{s}]\!]$ and $[\![\mathbf{e}]\!]$. $\mathcal{S}$ can then compute the corrupted parties' shares of $[\![\mathbf{t}]\!]$. For its emulation of $\mathcal{F}_{\mathsf{Open}}$, if the adversary inputs shares that are different from what $\mathcal{S}$ computed, $\mathcal{S}$ sends abort to $\mathcal{F}_{\mathsf{T\text{-}MLDSA}}$. Otherwise, $\mathcal{S}$ sends $\mathbf{t}$ to the adversary. It is clear that this is a perfect simulation of the real world.

Upon receiving PREP, $\mathcal{S}$ emulates $\mathcal{F}_{\mathsf{Prep}}$ by receiving from the adversary the corrupted parties' shares of $[\![\mathbf{w}_0]\!]$, $[\![\mathbf{w}_1]\!]$, $[\![\mathbf{y}]\!]$.

Upon receiving (SIGN, m), $\mathcal{S}$ receives from $\mathcal{F}_{\text{T-MLDSA}}$, $(\mathbf{w}_1, c, \mathbf{z}, \mathbf{h})$. For its emulation of $\mathcal{F}_{\text{Open}}$, if the adversary inputs shares that are different from that above, $\mathcal{S}$ sends abort to $\mathcal{F}_{\text{T-MLDSA}}$. Otherwise, $\mathcal{S}$ sends $\mathbf{w}_1$ to the adversary. $\mathcal{S}$ can then compute c and from the corrupted parties' shares of $[\![\mathbf{s}]\!]$, $[\![\mathbf{y}]\!]$, $[\![\mathbf{w}_0]\!]$, and $[\![\mathbf{e}]\!]$, it can compute their shares of $[\![\mathbf{z}]\!]$ and $[\![\overline{\mathbf{w}}_0]\!]$. For its emulation of $\mathcal{F}_{\text{RejSamp}}$, if the adversary inputs shares that are different from that above, $\mathcal{S}$ sends abort to $\mathcal{F}_{\text{T-MLDSA}}$. Otherwise, $\mathcal{S}$ checks if $\mathbf{z} = \mathbf{h} = \perp$ and if so outputs 1 to the adversary; else outputs 0. For its emulation of $\mathcal{F}_{\text{Open}}$, it again checks that the adversary's input shares match that computed above. Then, sends $\mathbf{z}$ to the adversary if so; aborts if not. This concludes the simulation. It is clear that it is a perfect simulation of the real world. □

Proof of Lemma 1. First, we start with a simulation for $\Pi_{\text{BatchedOR}}$. $\mathcal{S}$ receives from $\mathcal{F}_{\text{BatchedOR}}$ the corrupted parties' shares of $[\![\mathbf{v}]\!]_2$ and bit $b \in \{0,1\}$. $\mathcal{S}$ emulates $\mathcal{F}_{\text{Coin}}$ by sampling random $s^{(i)} \in \{0,1\}^{k_x}$, for $i \in [\kappa]$ and sending them to the adversary. Based on the above, $\mathcal{S}$ can compute the corrupted parties' shares of each $[\![b^{(i)}]\!]_2$. For its emulation of $\mathcal{F}_{\text{Rand}}$, $\mathcal{S}$ receives from the adversary the corrupted parties' shares of $[\![a^{(i)}]\!]_2$ for $i \in [\kappa]$. Based on the above, $\mathcal{S}$ can compute the corrupted parties' shares of $[\![\beta]\!]_2$ and $[\![\alpha]\!]_2$. If the adversary inputs any shares that do not match these to $\mathcal{F}_{\text{Mult}}$, then $\mathcal{S}$ sends abort to $\mathcal{F}_{\text{RejSamp}}$. Otherwise, if $b = 0$, $\mathcal{S}$ emulates $\mathcal{F}_{\text{Open}}$ by sending $\mathbf{0}$ to the adversary for $\mathbf{c}$. If $b = 1$, $\mathcal{S}$ emulates $\mathcal{F}_{\text{Open}}$ by sending random $\mathbf{c} \in \{0,1\}^{\kappa}$ to the adversary. Finally, $\mathcal{S}$ inputs continue to $\mathcal{F}_{\text{BatchedOR}}$.

It is clear that $\mathcal{S}$ exactly simulates the real world up until $\mathcal{F}_{\text{Open}}$. We now show that the rest of the simulation is identical and that the honest parties' outputs in the real and ideal worlds match with all-but-negligible probability. If $\mathbf{v} = \mathbf{0}$, every shared $b^{(i)} = 0$, so $\mathbf{b} \cdot \mathbf{a} = \mathbf{0}$; therefore, $\mathcal{S}$'s emulation of $\mathcal{F}_{\text{Open}}$ is identical to the real world and the honest parties will return $b = 0$ in both worlds.

Otherwise, there exists w.l.o.g., $j^* \in [\ell]$ such that $[\![v_{j^*}]\!]_2$ shares $v_{j^*} = 1$. Thus, for a single $i \in [\kappa]$, we can write

$$\left(\bigoplus_{j=1}^{\ell} s_j^{(i)} \cdot v_j\right) = 0 \iff s_{j^*}^{(i)} \cdot v_{j^*} = \left(\bigoplus_{j \neq j^*} s_j^{(i)} \cdot v_j\right)$$
$$\iff s_{j^*}^{(i)} = \left(\bigoplus_{j \neq j^*} s_j^{(i)} \cdot v_j\right),$$

which happens with probability $1/2$ since $s_{j^*}^{(i)}$ is sampled randomly. Since the $s_{j^*}^{(i)}$ are sampled independently for each $i \in [\kappa]$, we get that every $b^{(i)} = 0$ with probability $1/2^{\kappa}$; otherwise, there exists some $b^{(i_0)} \neq 0$. Now, since each $a^{(i)}$ is sampled independently and randomly, $\mathbf{a} \in \mathbb{F}_{2^{\kappa}}$ is uniformly random and thus so too is $\mathbf{c}$ in the real world; therefore $\mathcal{S}$'s simulation is identical. However, with probability $1/2^{\kappa}$, we could have $\mathbf{a} = \mathbf{0}$ and thus $\mathbf{c} = \mathbf{0}$, so that the parties in the real world output 0 but in the ideal world they output 1.

All together, the real and ideal world distributions are $2^{-\kappa+1}$-close. This concludes the proof. □

Proof of Lemma 2. First, we start with a simulation for Π_{RejSamp}. For its emulation of $\mathcal{F}_{\text{LTC}}$, $\mathcal{S}$ receives from the adversary the corrupted parties' shares of $[\![u_i]\!]_2$ for $i \in [k_x]$ and $[\![u_i']\!]_2$ for $i \in [k_y]$. If the adversary inputs any shares that do not match these to $\mathcal{F}_{\text{BatchedOR}}$, then $\mathcal{S}$ sends abort to $\mathcal{F}_{\text{RejSamp}}$. Otherwise, $\mathcal{S}$ inputs continue to $\mathcal{F}_{\text{RejSamp}}$. It is clear that $\mathcal{S}$ exactly simulates the real world. Correctness follows from the analysis in Section 4.1. □

D Cost of Generating Edabits

Here we estimate the cost of generating N edabits of length m (Functionality $\mathcal{F}_{\text{edabits}}$) using the techniques from [EGK+20]. At a high level their approach, adapted to our honest majority setting, consists of the following:

1. Each party P_i for $i = 1, \ldots, t+1$ distributes a *private edabit*, which is an edabit where the associated party knows the underlying secret. To ensure active security, these edabits are checked for consistency, which involves certain *cut-and-choose* procedure.

2. These $t+1$ private edabits are added up to obtain the final edabit, which is secret and unknown to any party. To remove the $\log(t+1)$ carry bits that happen in this addition, $\log(t+1)$ binary-to-arithmetic conversions are performed, which make use of dabits [RW19].

In what follows we analyze the communication costs of each one of these steps. Note that we do not focus on the round complexity, mainly due to the fact that their protocols are optimized for communication given that this can be run in parallel in an offline phase.

Before we start, recall the following basic MPC costs, which will be useful later on:

- Robustly opening an ℓ-bit secret-shared value costs $4n \cdot \ell$ bits, using the king-based reconstruction outlined in Section A.3.1.

- Multiplying two ℓ-bit secret-shared values takes $5.5n \cdot \ell$ bits, according to Section A.3.4.

- Multiplying two secret-shared ℓ-bit values using a preprocessed triples costs two openings, which amounts to $2 \cdot 4n\ell = 8n\ell$.

D.1 Cost of Generating Dabits

In this section we analyze the cost of producing a dabit, which can be seen as an edabit of length 1, that is, $([\![b]\!], [\![b]\!]_2)$. For this, we use the techniques from [RW19]. Below, we analyze the complexity of Protocol $\Pi_{\text{daBits+MPC}}$ from [RW19, Fig. 5

Parameter	Description	Constraints
κ	statistical security parameter	
λ	computational security parameter	
N	# of EdaBits generated	
m	bit-length of EdaBit	
n	number of parties	$t < \frac{n}{2}$
t	corruption threshold	
B	cut-and-choose parameter	
C	cut-and-choose parameter	$C^B \cdot \binom{B\ell}{B} > 2^\kappa$
ℓ	# of output daBits	

Table 8: Parameters for DaBit generation.

& 6], and the steps below corresponds to the protocol in that paper. Note that, since we are in the honest majority setting, we use $t+1$ parties for the aggregation of dabits (and we make a similar modification to generate edabits later on).

The protocol from [RW19] generates ℓ dabits at once.

Generate daBits. First, the following is performed by $t+1$ parties. In steps 2.b and 2.c, the party secret-shares $CB\ell$ bits in both the arithmetic and the binary domain. This costs $CB\ell(t+1)(n-1)(\log q+1)$ bits in total. Running secret-sharing in parallel, this phase takes 1 round.

Cut and Choose. In step 1 a call to a public coin sampling functionality is made. We ignore these costs since a seed can be expanded with a PRF. Then in steps 3 and 4 the parties open $(t+1)(C-1)B\ell$ secret-shared values both in arithmetic and binary domains, which cost $4n(t+1)(C-1)B\ell(\log q+1)$ bits in total. As $\mathcal{F}_{\mathsf{Rand}}$ takes 1 round, followed by $\mathcal{F}_{\mathsf{Open}}$ which takes either 1 round. Overall, this phase takes 2 rounds.

Combine. The parties XOR the $t+1$ bits together, $(C-1)B\ell$ times. For the arithmetic part, this costs t secure multiplications, which amounts to $t(C-1)B\ell \cdot 5.5n\log q$ bits in total. This phase takes 4 rounds due to $\mathcal{F}_{\mathsf{Mult}}$, including 3 rounds for generating the triples.

Consistency Check. Another call to a coin-sampling functionality is made, whose complexity we ignore. Step 2 is repeated for ℓ buckets. For 2.b the parties need to add B arithmetically-shared bits, which costs $B-1$ products, so $(B-1)5.5n\log q$ bits. For 2.c and 2.d the parties open $B-1$ bits in both the arithmetic and binary domains, which costs $(B-1)4n(\log q+1)$ bits. Adding up and multiplying by the ℓ buckets, we get[14]

$$\ell(B-1)9.5n\log q+(B-1)4n.$$

This phase involves $\mathcal{F}_{\mathsf{Rand}}$ (1 round), $\mathcal{F}_{\mathsf{Mult}}$ (1 round), and $\mathcal{F}_{\mathsf{Open}}$ (1 round), for a total of 3 rounds.

[14]Note that steps 3 and 4 correspond to a MAC check that we do not need in our honest majority protocol (and whose complexity is independent of the number of dabits anyway, so its complexity amortizes away).

D.1.1 Concrete Parameters for Generating Dabits

Adding the rounds above we obtain a total of 10 rounds to generate ℓ dabits. Adding the communications costs above we obtain

$$CB\ell(t+1)(n-1)(\log q+1)+4n(t+1)(C-1)B\ell(\log q+1)$$
$$+5.5t(C-1)B\ell n\log q+\ell(B-1)9.5n\log q+(B-1)4n. \quad (5)$$

In [RW19] the parameters C and B are chosen so that $C^B \cdot \binom{B\ell}{B} > 2^\kappa$, where ℓ is the desired number of dabits, and κ is the statistical security parameter. We set $\kappa = 40$. In [RW19] the authors choose $C=5$ and $B=4$, and with these parameters we need a mild minimum of $\ell > 113$ dabits in parallel for the expression above to hold. We allow for many more dabits in parallel, and choose $C=2$, $B=2$, which requires $\ell \approx 370$k dabits in parallel. With these parameters, we simplify Eq. (5), and divide it by ℓ to obtain a total cost per dabit of

$$\leq 23tn(\log q+1)+9.5n\log q+4n. \quad (6)$$

D.2 Generating Private Edabits

Protocol $\Pi_{\mathsf{edaBitsPriv}}$ in [EGK+20, Fig. 4] generates N private edabits known to a given party P_i. The costs are the following (we the step labels below correspond to the steps of the protocol in [EGK+20]):

- In Step 1, P_i distributes $m\cdot(NB+C)$ binary shares. Recall that in our setting these are shares over $\mathbb{F}_{2^k}$ with $k=\log(n)$. This costs $m(NB+C)\log n\cdot(n-1)$ bits, where the $n-1$ factor comes from sending the shares to the remaining $n-1$ parties. This phase takes 1 round.
- In Step 2 P_i distributes $NB+C$ arithmetic shares. In our setting these are defined over $\mathbb{Z}_q$. This costs $(NB+C)(n-1)\log q$.
- In Step 3 P_i distributes $(N(B-1)+C')m$ bit-triples. This costs $3\cdot(N(B-1)+C')m(n-1)\log(n)$. Step 2 and 3 executed in parallel take 1 round.

The next step in the protocol is the cut-and-choose check, which we analyze next.

D.3 Cost of the Cut-and-Choose

Protocol $\Pi_{\mathsf{CutNChoose}}$ in [EGK+20, Fig. 5] produces N private edabits, and has the following costs:

1. Step 1 is a call to $\mathcal{F}_{\mathsf{Coin}}$ [15] which we ignore as the cost amortizes away. This step takes 2 rounds.

[15]This is called $\mathcal{F}_{\mathsf{Rand}}$ in [EGK+20] but it actually corresponds to public coins.

2. Step 2 requires C arithmetic openings, which costs $C \cdot 4n\log q$ bits, and $C'm$ binary openings, which costs $C'm \cdot 4n\log n$ bits. This step takes 1.

3. Items 4.b, 4.c and 4.d are repeated $B-1$ in every bucket, and there are N/B buckets. Hence, the costs below are multiplied by $\frac{N}{B}(B-1)$. The costs of these steps are the following:

   (a) Step 4.b is a binary adder, which the authors in [EGK+20] set to have m multiplication (*i.e.* AND) gates. The parties use the triples to compute this, which takes $m \cdot 8n\log n$ bits. This step takes m rounds.

   (b) Step 4.c involves calling ConvertB2A using a dabit. This consists of masking and opening in the binary domain, which costs one binary reconstruction, that is $4n\log n$, plus the cost of generating one dabit, which we denote by D, and can be found in Eq. (6). This step takes 1 round.

   (c) Step 4.d involves opening an edabit, which costs $4n\log q$ for the arithmetic part, and $m \cdot 4n\log n$ for the binary part. This step takes 1 round.

With all the above, N private edabits known to a party P_i are produced. Summing the costs we see that the communication complexity of this is upper-bounded by

$$n\log n\left(m\left(4NB+C+5C'+12N\right)+4N\right) + n\log q\left((NB+C)+4N\right)+ND \quad (7)$$

Overall, $\Pi_{\mathsf{edaBitsPriv}}$ and $\Pi_{\mathsf{CutNChoose}}$ take $7+m$ rounds.

D.4 From Private to (Normal) Edabits

In [EGK+20], $t+1$ parties generate N private edabits each with the method above, whose cost corresponds to Eq. (7) multiplied by $t+1$. Then, these private edabits are combined via addition into N edabits where no party knows the underlying secrets. This is captured in Protocol Π_{edaBits} in [EGK+20, Fig. 3]. We analyze below the cost per edabit (that is, without multiplying by N)

1. Step 3 involves a $(t+1)$-input m-bit binary adder, which has $\log(m)\cdot\log(t+1)$ layers. Upper-bounding complexity, this can be implemented via t $(m+\log(t+1))$-bit adders, which have in total $t\cdot(m+\log(t+1))$ binary multiplication gates. This costs $t(m+\log(t+1))\cdot 5.5n\log n$ bits. This step takes $\log(m)\cdot\log(t+1)$ rounds.

2. In Step 4 ConvertB2A is called $\log(t+1)$ times, which costs $\log(t+1)\cdot 4n\log n$ bits, plus the cost of generating $\log(t+1)$ dabits, which is $D\cdot\log(t+1)$. This step takes 1 round.

The total cost for a single edabit is then

$$n\log n\,(5.5tm+\log(t+1)(4+5.5t))+\log(t+1)\cdot D \quad (8)$$

Π_{edaBits} assumes Π_{daBits}, $\Pi_{\mathsf{edaBitsPriv}}$, and $\Pi_{\mathsf{CutNChoose}}$, overall, it takes $10+7+m+\log(m)\cdot\log(t+1)+1 = 18+m+\log(m)\cdot\log(t+1)$ rounds.

If we add to this the amortized cost of $t+1$ parties generating one private edabit each, which is obtained by multiplying Eq. (7) by $(t+1)/N$, we see that the final cost of generating one edabit is

$$\begin{aligned}(t+1)n\log n\left(m\left(4B+\frac{C}{N}+5\frac{C'}{N}+12\right)+4\right)\\ +(t+1)n\log q\left((B+\frac{C}{N})+4\right)\\ +n\log n\,(5.5tm+\log(t+1)(4+5.5t))\\ +\left((t+1)+\frac{t+1}{N}\cdot\log(t+1)\right)\cdot D. \quad (9)\end{aligned}$$

D.5 Concrete Choice of Parameters

According to [EGK+20, Theorem 2], the edabit generation is secure except with probability $2^{-\kappa}$, if one sets the parameters as $N \geq 2^{\frac{\kappa}{B-1}}$, $C=C'=B$, and $B\in\{3,4,5\}$. We set $\kappa=40$ as the statistical security parameter, and take $C=C'=B=3$, which imposes the restriction that $N\geq 2^{40/2}\approx 1M$ edabits must be generated in parallel. With these parameters, Eq. (9) simplifies to the following. We ignore the terms that have N in the denominator which amortize away as N grows. We also replace D from Eq. (6), to obtain:

$$\begin{aligned}(t+1)n\log n\,(24m+4)+7(t+1)n\log q\\ +n\log n\,(5.5tm+\log(t+1)(4+5.5t))\\ +(t+1)(23tn(\log q+1)+9.5n\log q+4n). \quad (10)\end{aligned}$$